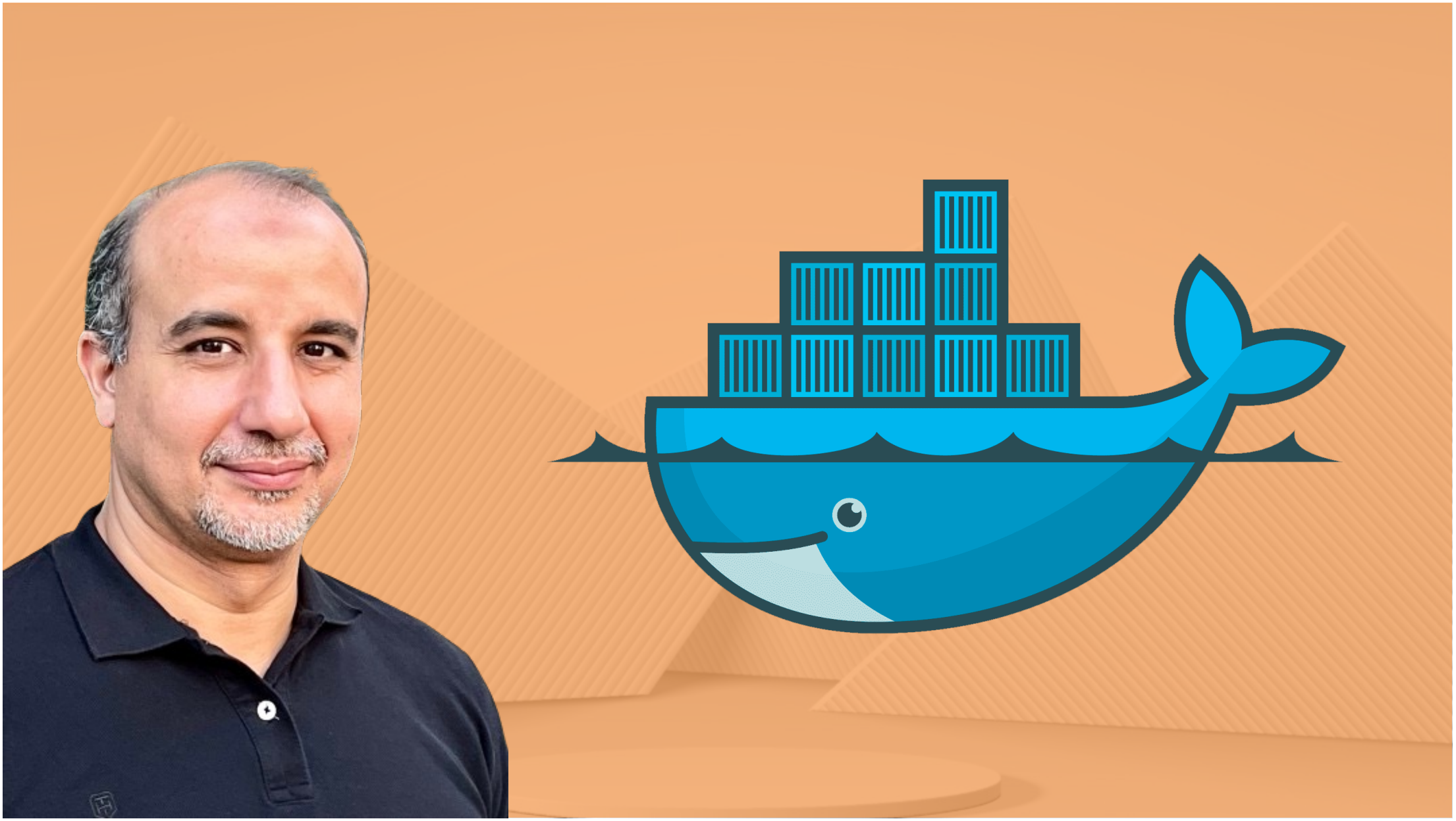
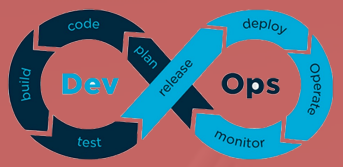
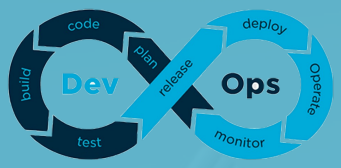






Course Introduction



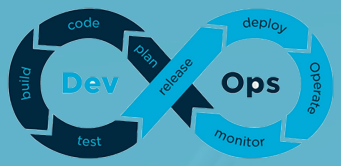


Course Audience

Intended Audience

- Newcomers to IT
- IT professionals including :
  - Software Developers
  - MLOps engineers
  - AI Engineers
  - Data Engineers
  - Data Analysts
  - DevOps Engineers
  - Cloud Engineers
  - Cyber Security Professionals

© DolfinED All rights reserved

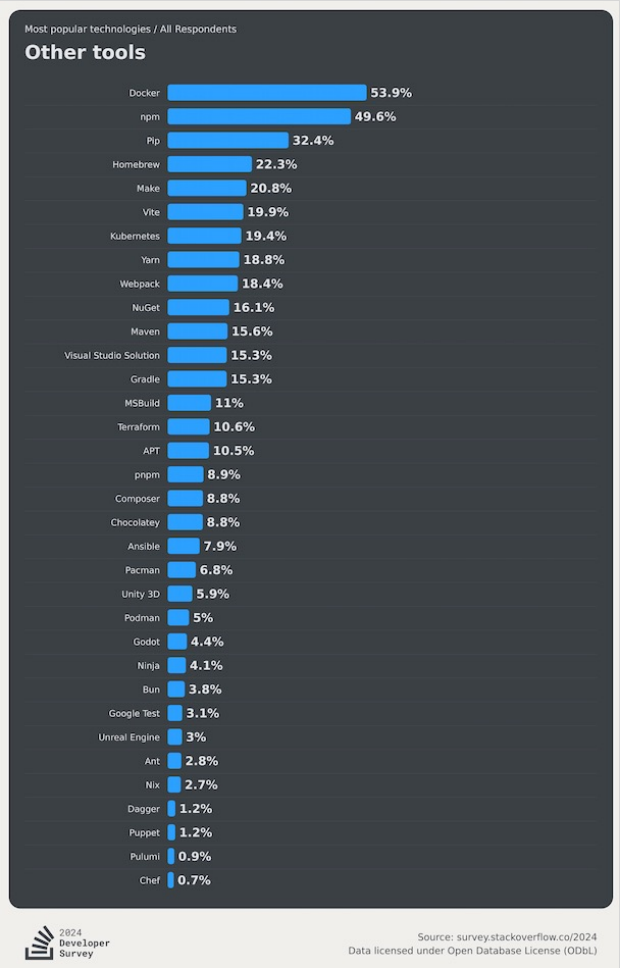

code
deploy
plan
release
Dev
Ops
operate
build
test
monitor



Why Docker?

Why Docker?

Developers



Stack Overflow Survey – Popular Developer Tools

https://survey.stackoverflow.co/2024/technology#1-other-tools

© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

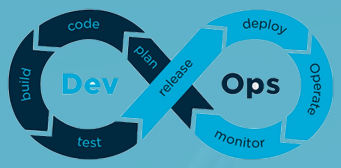
Why Docker?

- **Environment reproducibility**: Everyone gets the *exact* same setup.
- **Dependency management**: No Python hell, Conda weirdness, or OS-specific issues.
- **Portability**: Run your container on any machine with Docker (dev laptop, server, cloud instance).
- **Version control for environments**: Dockerfiles are text = versionable in Git.

Stack Overflow Survey – Popular Developer Tools

https://survey.stackoverflow.co/2024/technology#1-other-tools

© DolfinED All rights reserved



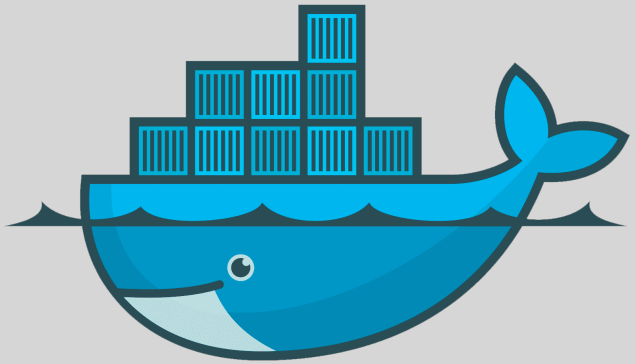


Course Highlights

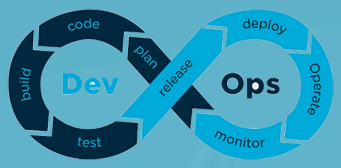
Course Highlights



- 15+ hours
- 100+ Lectures
- 40+ Demos
- 2 Capstone Projects



© DolfinED All rights reserved





Course Curriculum

Section 2

Servers and Virtualization Overview

- Compute Device Components
- What is a Server?
  - Server Components
  - Scaling Servers
  - Hosting Servers in Datacenters
- Server Resource Optimization & Multiple Apps on a Server
- Introduction to Virtualization
  - Virtualization Components
  - Popular Hypervisors Providers
  - Types of Hypervisors

Section 3

From Virtualization to Containers - The background

- Containers Overview
- Docker Containers – From a Dockerfile to an Application

Section 4

Docker Architecture

- Docker Architecture
- Docker Hosts - How Docker Works
- Namespaces and Cgroups

Section 5

Environment Setup

- Learning on real Cloud VM vs. a VM on your laptop
- Demo - Create a Free AWS Account
- Demo - Limiting your spending on AWS
- Demo - Understand AWS Free Tier Services
- Demo - Launch an Ubuntu AWS EC2 instance
- Demo - Connect to the Instance from Linux, MaxOS and Windows Clients
- Demo - installing Docker

Section 6

Basic Docker Commands

- Basic Docker commands
- Demo – Basic Docker Commands

Section 7

Dockerfile

- Setup VS Code – Installing Docker Extension
- Docker File
- Demo – Docker file
- Dockerfile – ENTRYPOINT
- Demo – Docker file

Section 8

Docker Image and Docker Registry

- Docker Images
- Demo – Docker Images
- Docker Registry
- Demo – Creating a Docker Hub Account
- Demo – Docker Registry

Section 9

Docker Networking

- Network Bridges Overview
- Docker Drivers and Networking Modes
- Demo - Docker Networking Modes – Bridge Mode
- Demo – Docker Bridge Network – Container Port Mapping
- Docker Networking Modes (cont.)
- Demo - Docker Networking Modes – Host and None

Section 10

Docker Volumes (Persisting Data)

- Docker Volumes
- Demo – Docker Volumes
- Create a Docker Volume using a Dockerfile
- Demo - Create a Docker Volume using a Dockerfile
- Docker Bind Mounts
- Demo - Bind Mounts

Section 11

Docker Compose

- Introduction to Docker Compose
- Demo - Install Docker Compose
- Docker Compose File Structure
- Demo – Docker Compose Configuration File

Section 12

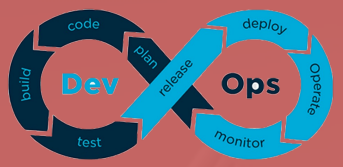
Capstone Projects

- Project 1 – Deploy A Simple Website Using Nginx
- Project 2 – Deploying a Website Visitor Counter Application (Flask and Redis Database) using Docker Compose

Section 13

Bonus/Advanced Topics

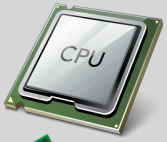
- Name Resolution In Docker (DNS)
- Demo – Name Resolution in Docker
- Dockerfiles – Multi-Stage Builds
- Demo – Dockerfiles Multi-Stage Builds
- .dockerignore file
- Docker Best Practices
- Monolith vs Microservices
- Introduction to Docker Swarm



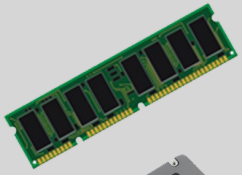


Servers and Virtualization Overview

Compute Device Components



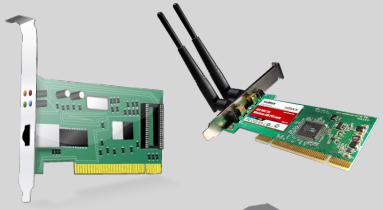
Central Processing Unit (The Brain).
Is involved in all operations that run on the compute device



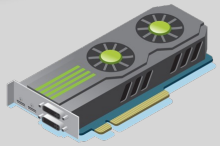
Random Access Memory (RAM)
Used to run applications and hold data during processing.



Hard Disk Drives(HDD) and Solid-State Drives (SSD).
Used to provide persistent storage for OS, Data and Applications.



Network Interface Cards
Connect the computing device to the outside world. Can be wireless (WiFi) or wired (uses Ethernet/LAN cables).



Graphic cards or GCPUs are used to accelerate video operations (gaming, rendering) on the compute device. Can be built-in or a card.

© DolfinED All rights reserved

What Is A Server?



Laptop



Physical Server

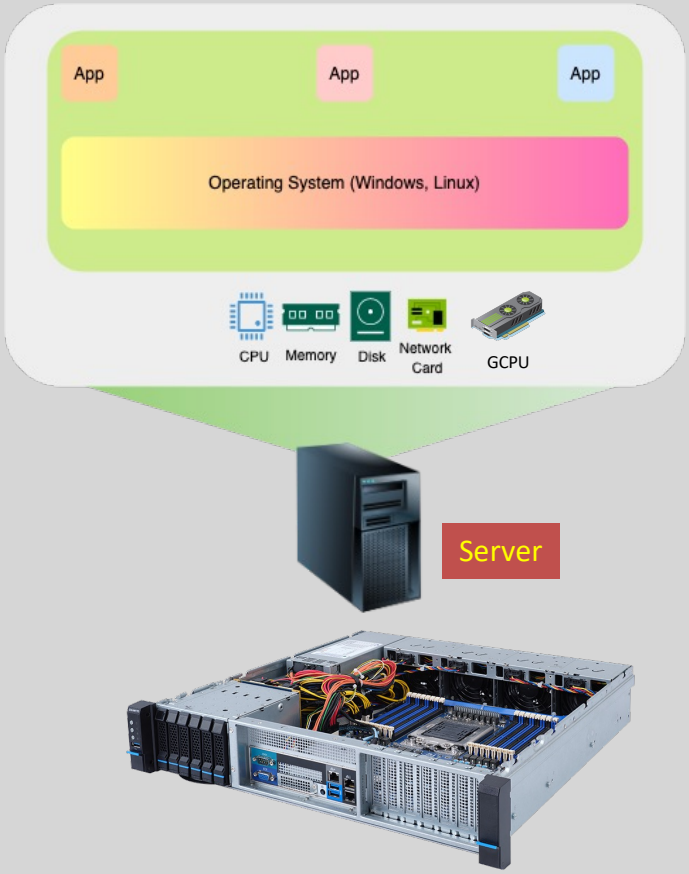
© DolfinED All rights reserved

Server Components

A server is basically a powerful computing device with:

- A good size memory (RAM)
- One or more CPU(s)
- One or more disk drives to persistently store the operating system, applications, and data
- One or more network interfaces
- One or more GCPUs
- An operating system (Windows, Linux, or MacOS

We can install one or more applications on a server



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Scaling Servers

As the number of users increases,

- The server performance may degrade
- We need to increase (scale) the number of servers



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Server Hosting in Datacenters

A data center is an air conditioned, secure, and monitored space



© DolfinED All rights reserved

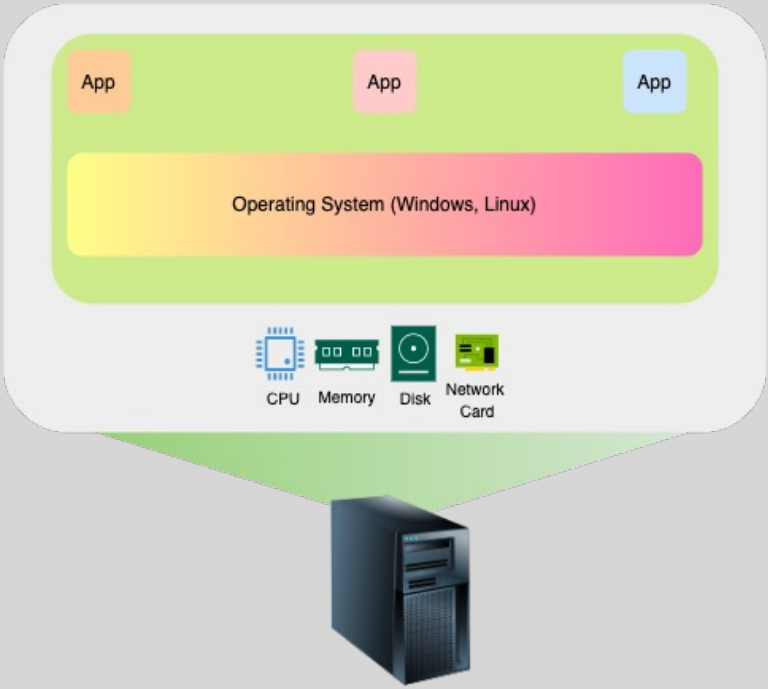
Server Resource Optimization & Multiple Apps on a Server

To optimize resource usage, multiple applications could be run on the same physical server

- Each application may require its own runtime version, dependencies, database..etc

Drawbacks:

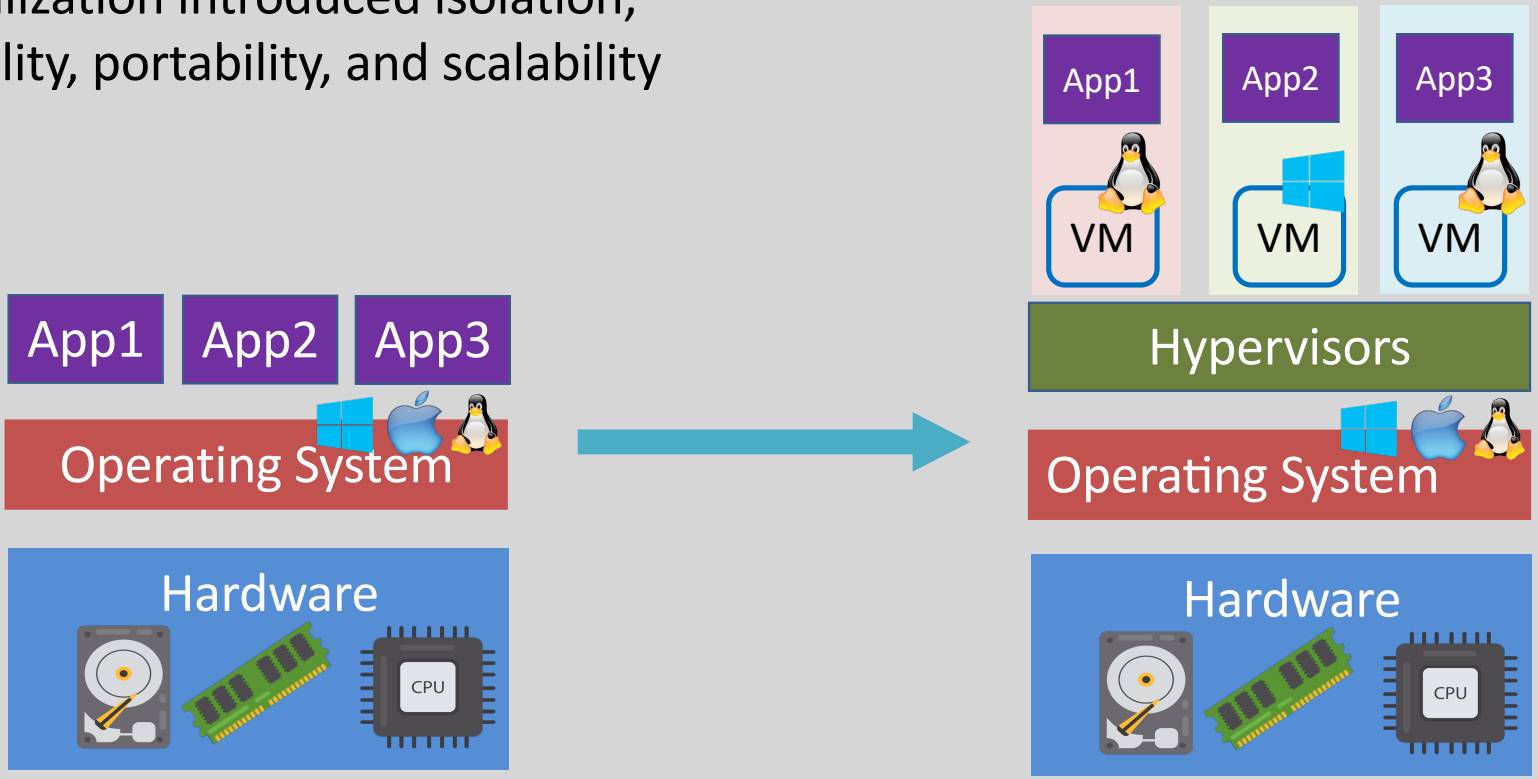
- Resource contention and inefficiency
- Dependency conflicts
- Lack of strong isolation - Security risks
- Scaling is painful
- Deployment & maintenance hell
- Lack of portability



© DolfinED All rights reserved

Introduction to Virtualization

Virtualization introduced isolation, flexibility, portability, and scalability



Physical (bare metal) Server

Virtualization

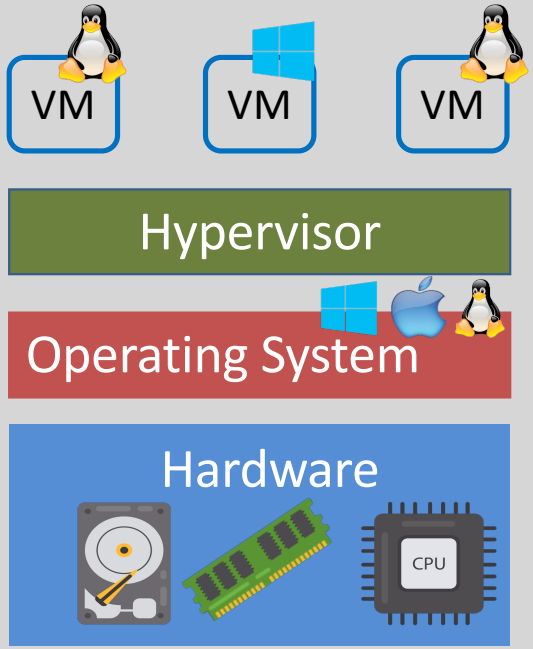
© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Introduction to Virtualization (cont.)

Virtualization enables us to run more than one virtual machine, multiple operating systems, and applications on a single physical server

- Virtual machines are isolated
- We can take a snapshot of the VM (a backup)
  - A VM is a set of files.



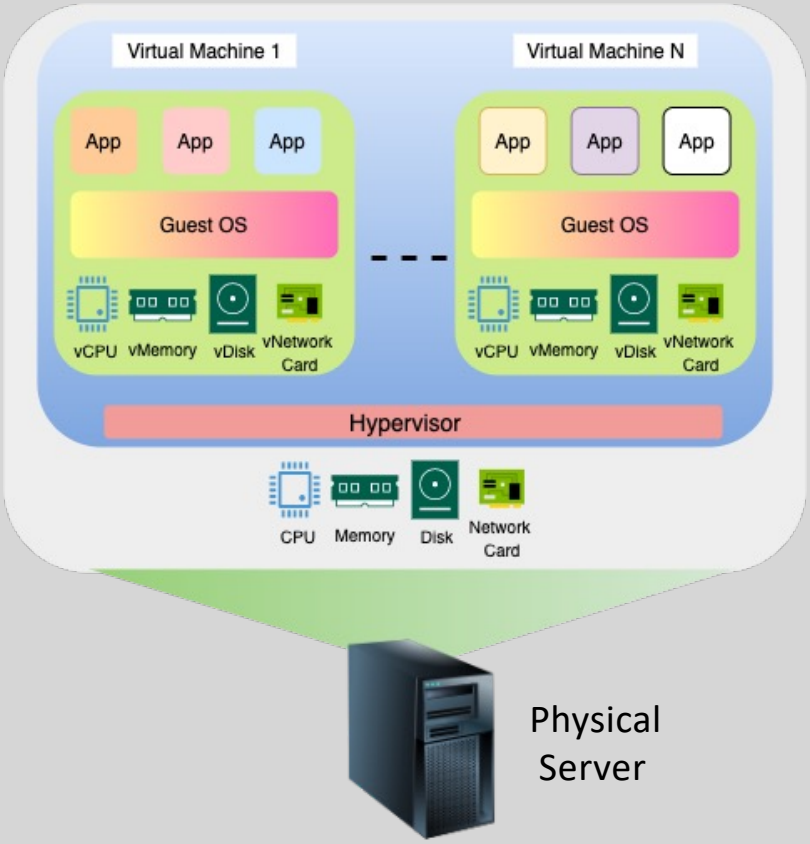
© DolfinED All rights reserved

Virtual Machine (VM) Resources

The hypervisor shares the physical server resources among the virtual machine (VMs)

Each VM gets its:

- Virtual Network Interface Card (vNIC)
- Virtual Disk (vDisk)
- Virtual RAM (vRAM)
- Virtual CPU(s) (vCPU)



© DolfinED All right reserved

Popular Hypervisors Providers

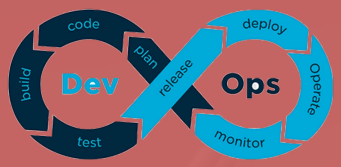
- VMware vSphere
- Xen
- Microsoft Hyper-V
- Linux KVM
- Oracle VM VirtualBox
- VMware Workstation

© DolfinED All rights reserved

Virtual Machines (VMs) - Issues

- Heavyweight (slow to start)
- Limited Scalability
- Poor Dev/Test/Prod parity
- Redundant OS overhead
- Inefficient Image Management
- Low portability

© DolfinED All rights reserved





From Virtualization to Containers – The Background

Linux Software Processes Overview

A process is, a running instance of a program that has its own memory, CPU context, and system resources, managed by the Linux kernel

When you run a program (like bash, nginx, or python), the Linux kernel:
1. Loads the program's code into memory
2. Allocates it a **unique Process ID (PID)**
3. Creates a **process** to execute the code

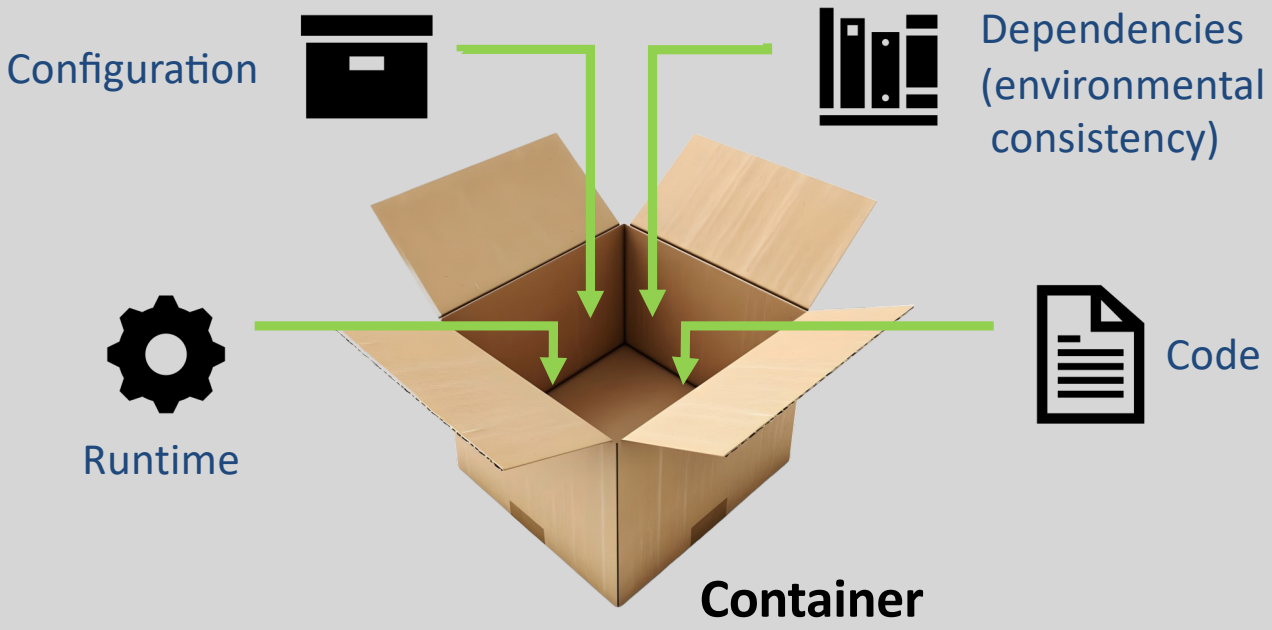
© DolfinED All rights reserved

Containers Overview

A **container** is a **lightweight, isolated process** running on a shared operating system kernel, packaged with everything it needs to run, including code, libraries, environment variables, runtime, and configuration files.

- We can run applications inside containers





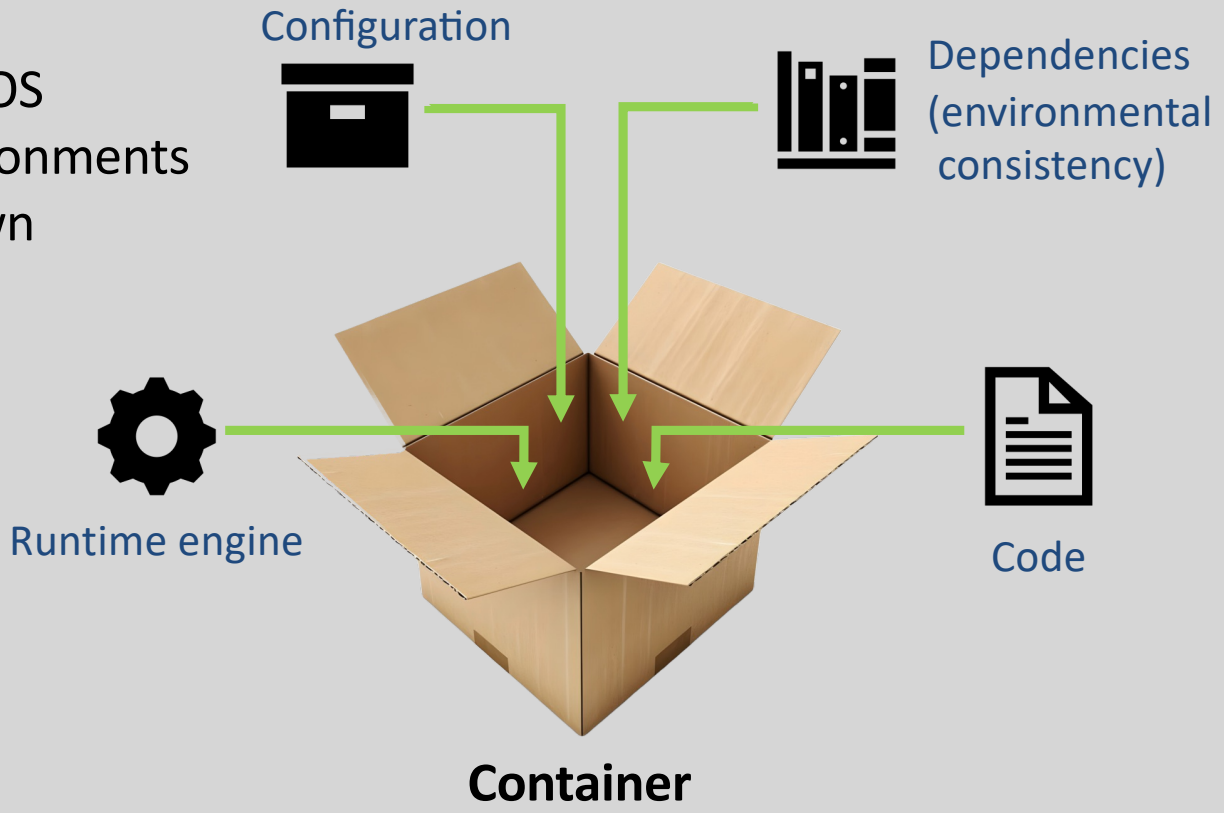
© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Containers - Features

Container features

- Does not require a separate OS
- Isolated, self-contained environments
- Faster to create and tear down
- Repeatable
- Portable
- Easily scalable

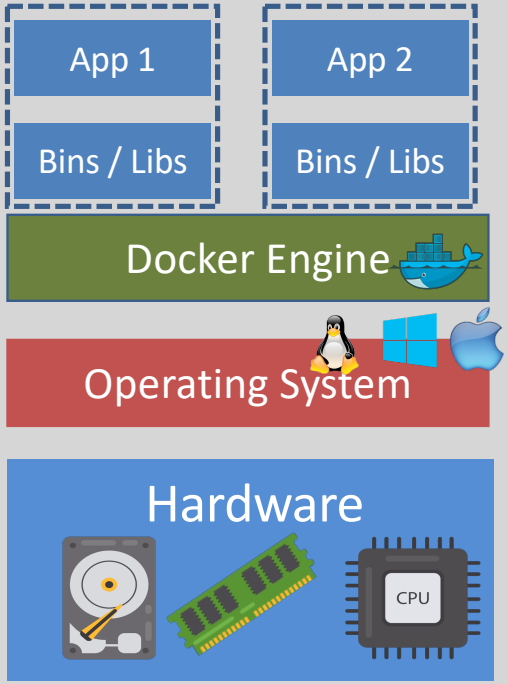


© DolfinED All rights reserved

Containerization Overview

To build, ship and run containers, we need a container runtime engine be installed on the host's operating system

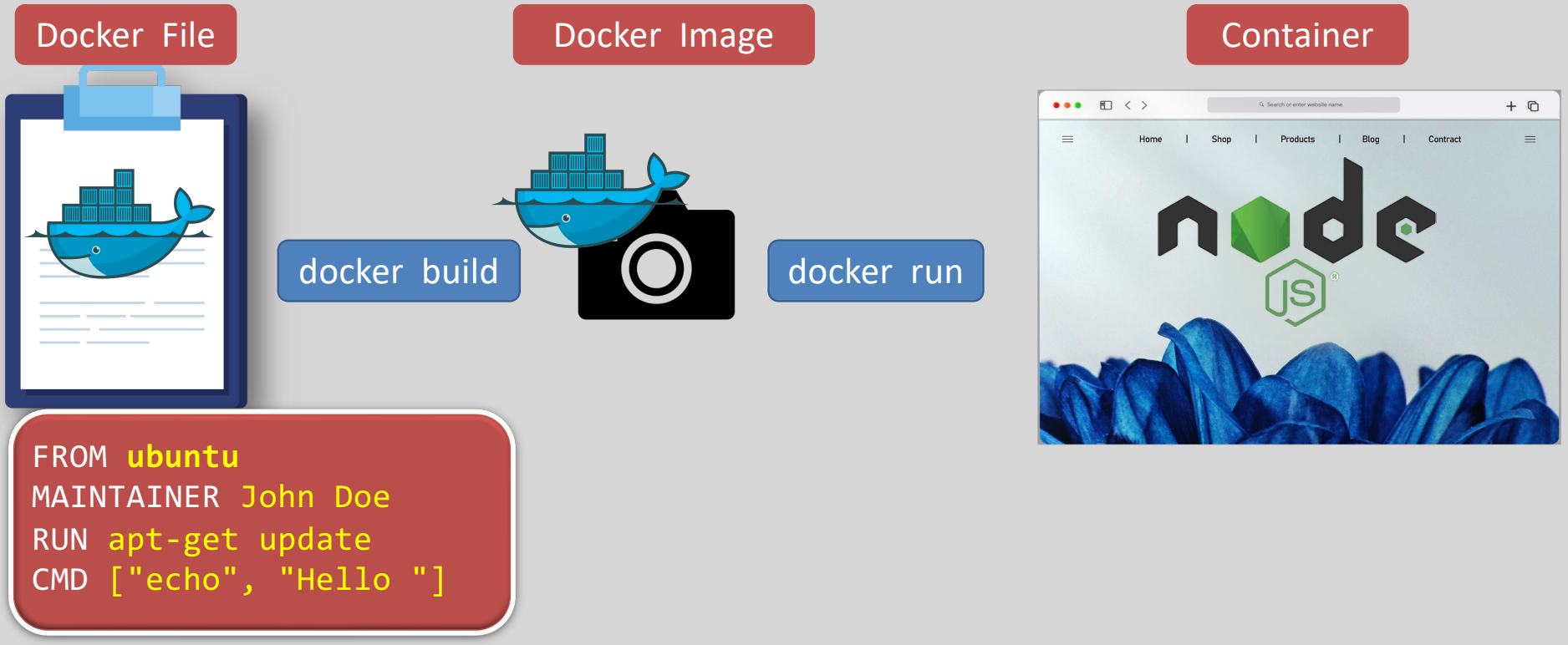
- The engine shares the Host's OS kernel among the containers
- Containerized applications are "isolated"
- Docker is the most popular container engine
  - It comes in Community Edition (CE) which is free and a paid Enterprise Edition (EE)



© DolfinED All rights reserved

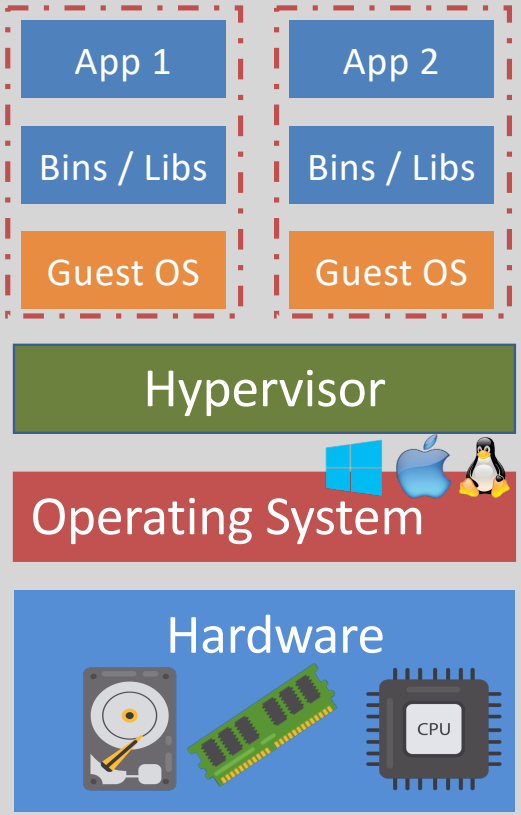
Docker Containers – From a Dockerfile to an Application

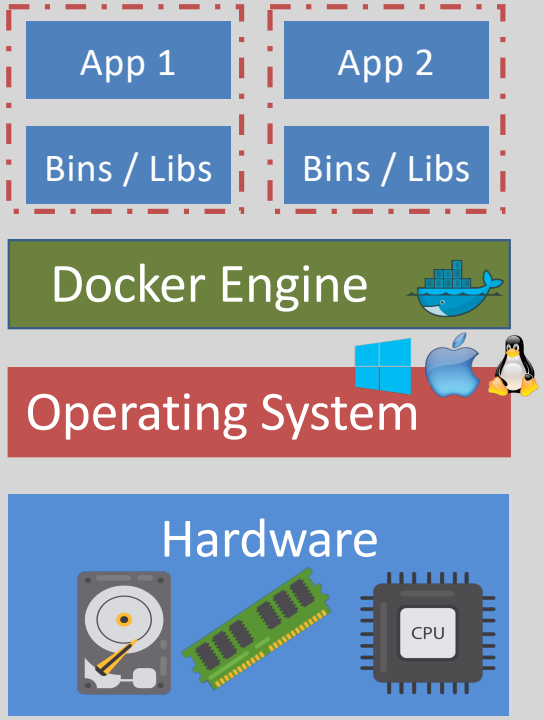
Starting with a Dockerfile, we can build a container image that can be used to consistently run an application across different environments.



© DolfinED All rights reserved

Virtual Machines (VMs) vs. Containers



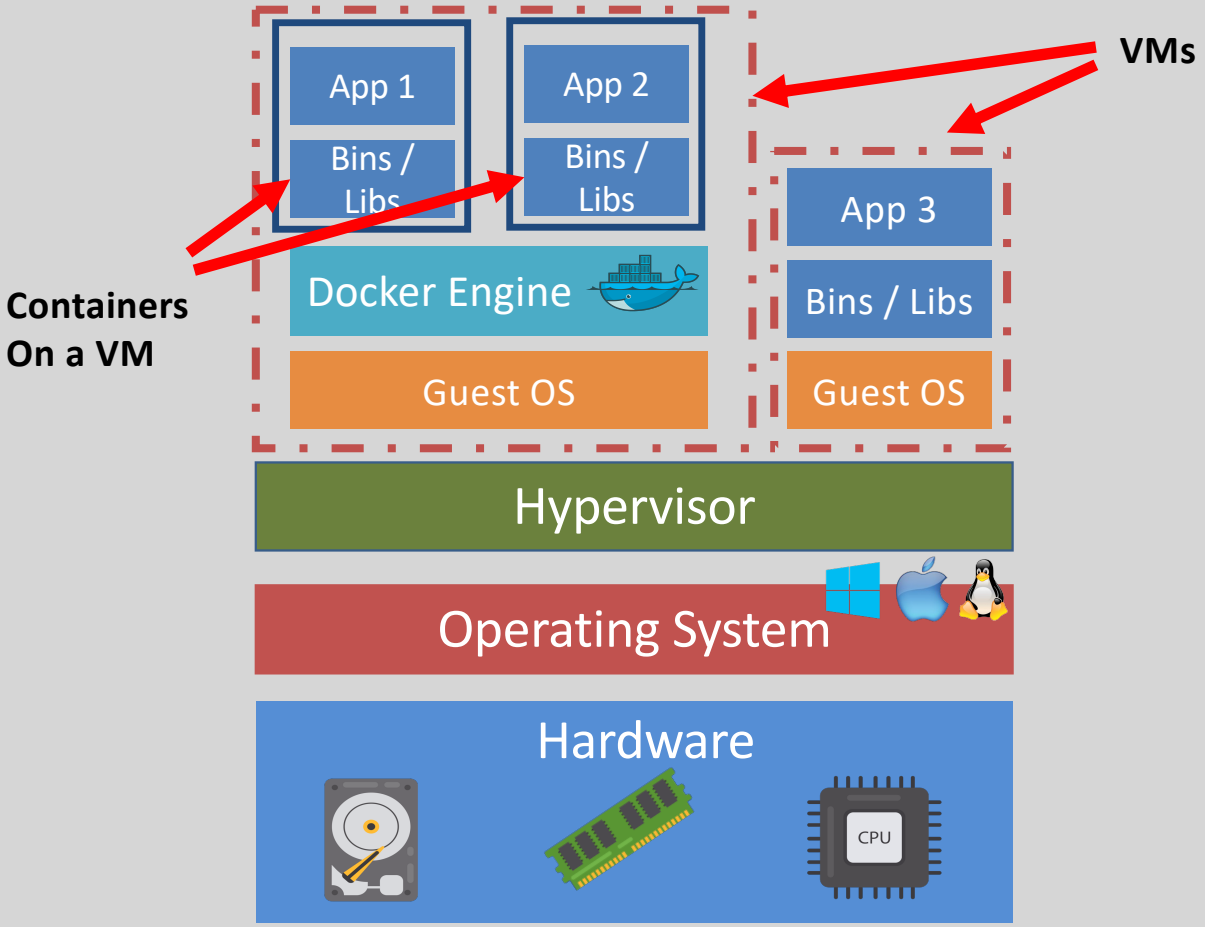


© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Virtual Machines (VMs) vs. Containers

We can run containers on Virtual Machines



© DolfinED All rights reserved

Virtual Machines (VMs) vs. Containers (cont.)

Virtual Machines (VMs):

- Heavyweight (slow to start)
- Limited Scalability
- Low portability
- Redundant OS overhead
- Inefficient Image Management
- Poor Dev/Test/Prod parity

Containers:

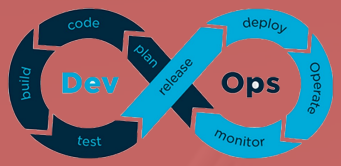
- Lightweight (start in milliseconds)
- Faster scaling
- Excellent portability
- Containers share the host OS kernel
- Efficient image management
- Containers are excellent for CI/CD

© DolfinED All rights reserved

Popular Container Engines & Runtimes

- Docker
- Linux containers – LXC (System Level containerization framework)
- Containerd – geared towards Kubernetes deployments
- CRI-O – geared towards Kubernetes deployments

© DolfinED All rights reserved



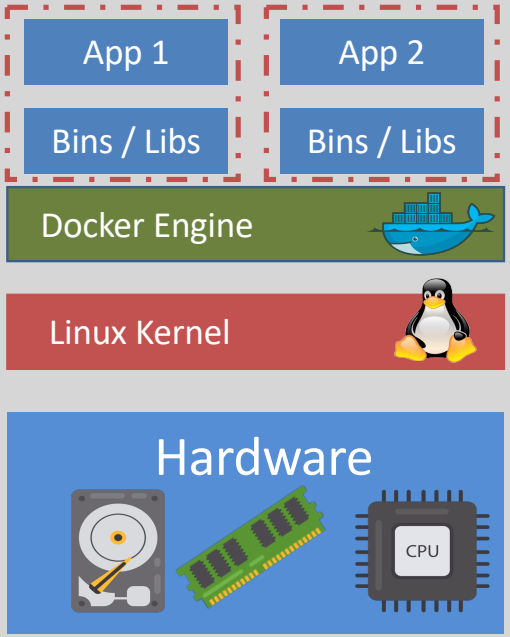


Docker Architecture

Docker - Overview

Docker is a software platform that simplifies the process of building, running, managing, and distributing applications using containers.

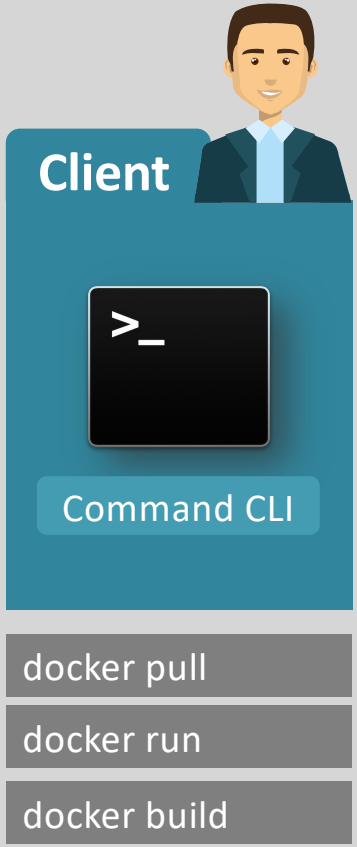
- Developed by Docker Inc.
- Docker is open source
- Docker virtualizes the host OS (containers use the same kernel)
- Applications run in containers as isolated environments
- There are two types of Docker Editions: docker-CE, docker EE



© DolfinED All rights reserved

Docker Architecture - Docker Client

- It enables developers/users to interact with Docker
- When you use docker commands, the docker client sends these commands to the Docker daemon (dockerd)



© DolfinED All rights reserved

Docker Architecture – Docker Host

The server or virtual machine on which Docker Engine is installed

- The **Docker Daemon** (dockerd) is the heart of Docker



© DolfinED All rights reserved

Docker Architecture – Docker Images

Images are read-only binary templates used to build containers

- They contain the **application code**, **libraries**, and **dependencies** required to run an application



© DolfinED All rights reserved

Docker Architecture – Docker Daemon (Service)

The **Docker Daemon** (dockerd) is the heart of Docker

- It listens to Docker API requests (from the Docker client) and manages containers, images, networks, and volumes
- It builds container images as requested by the client
- It interfaces with docker registries to pull or publish images as requested by the client
- It manages the lifecycle of the containers (start, stop, remove)



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Docker Architecture – Docker Containers

Containers are encapsulated environments in which you run applications

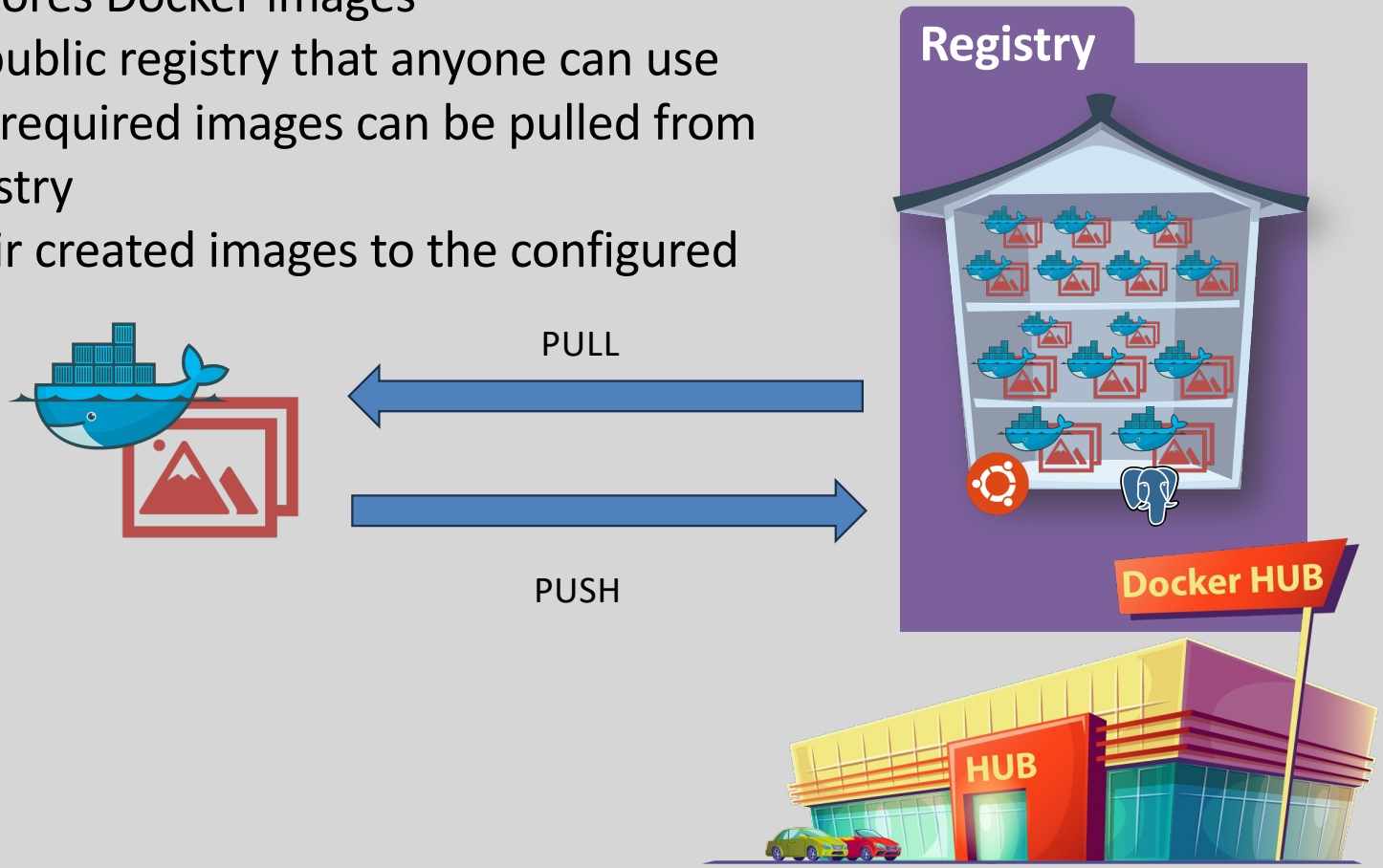
- A container is an instance of a Docker image
- A container runs as a process in an isolated environment
- It packages the application and its dependencies into a single executable unit



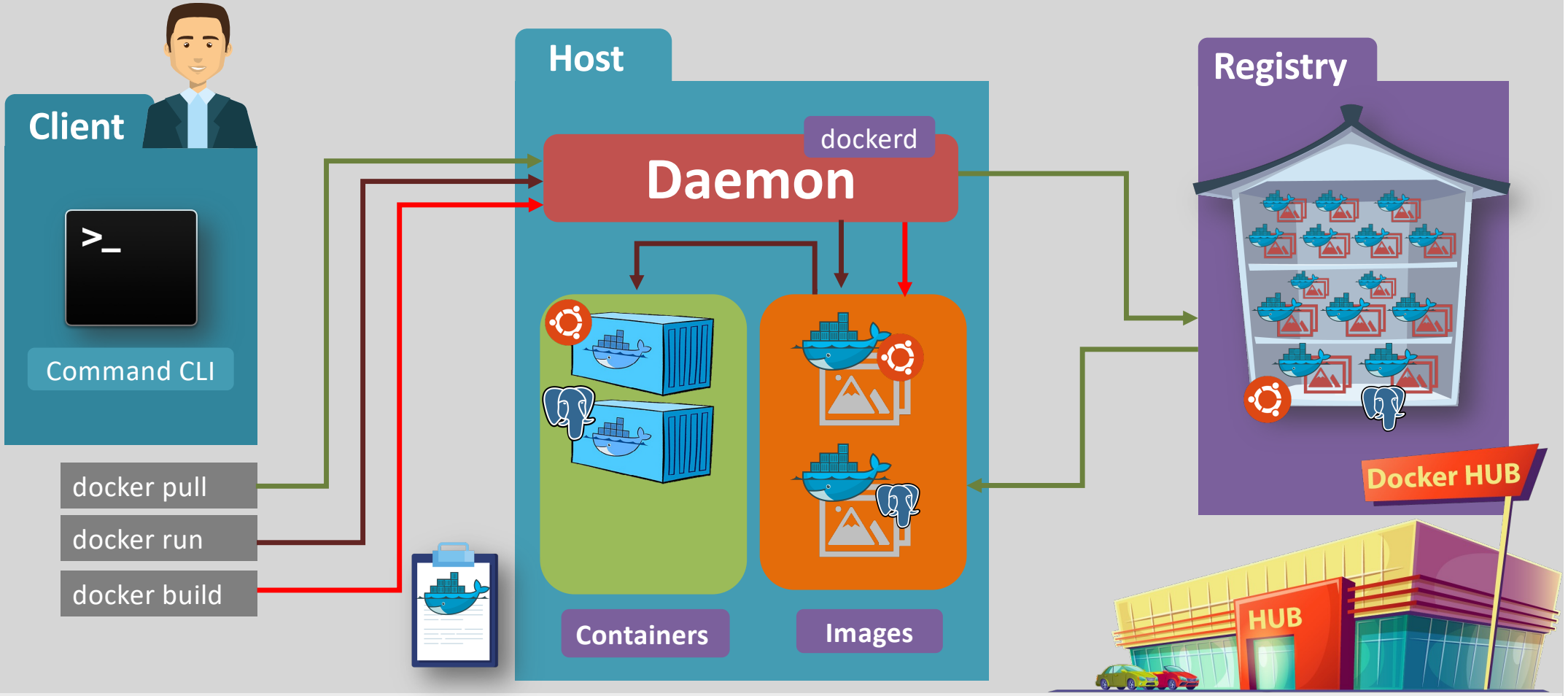
© DolfinED All rights reserved

Docker Architecture – Docker Registries

- A Docker registry stores Docker images
- Docker Hub is the public registry that anyone can use
- When needed, the required images can be pulled from the configured registry
- Users can push their created images to the configured registry

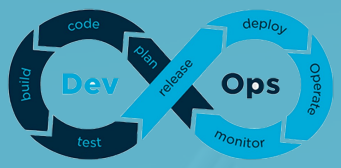


Docker Architecture – Connecting the Dots



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning





Docker Hosts – How Docker Works

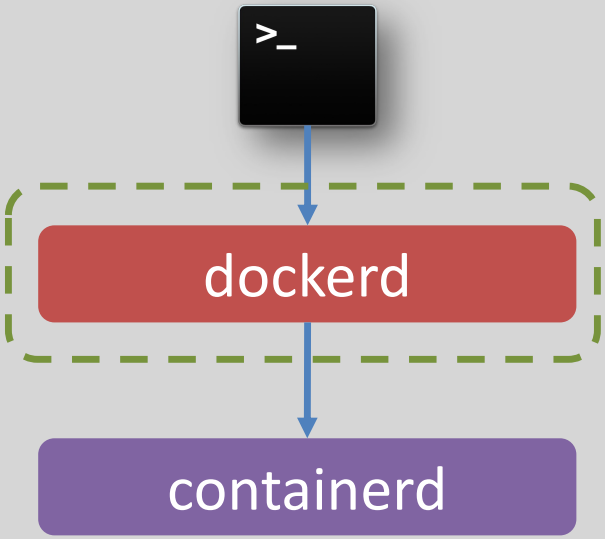
Understanding How Docker Works

Docker CLI

- Docker CLI communicates with the Docker Daemon/server, dockerd

dockerd

- dockerd processes the Docker API requests and utilizes **containerd** functionality to manage the container's life-cycle.



© DolfinED All rights reserved

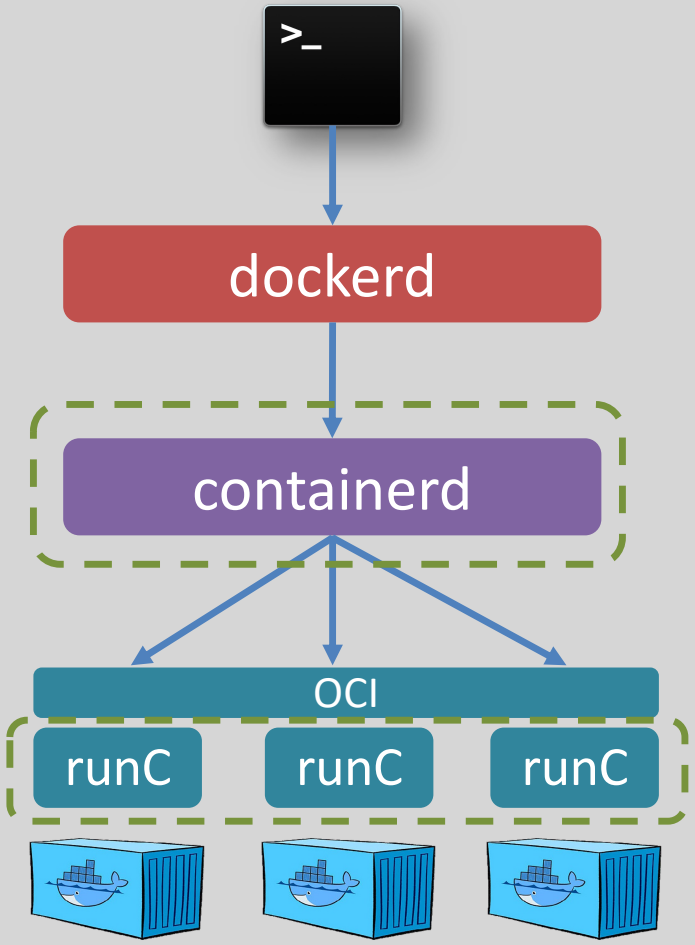
Understand how to running containers (cont.)

Containerd

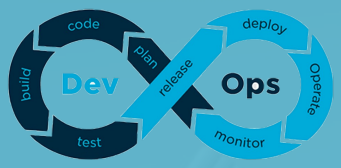
- Manages containers, Storage, and Networking
- Pushes and pulls images

runC

- Creates and runs containers.



© DolfinED All rights reserved

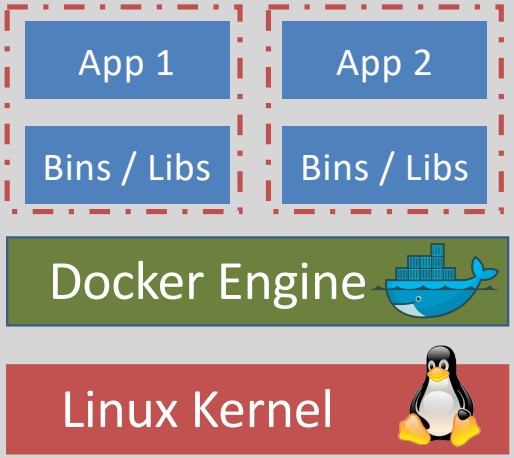




Namespaces and Cgroups

Containers – Isolation and Resource Management

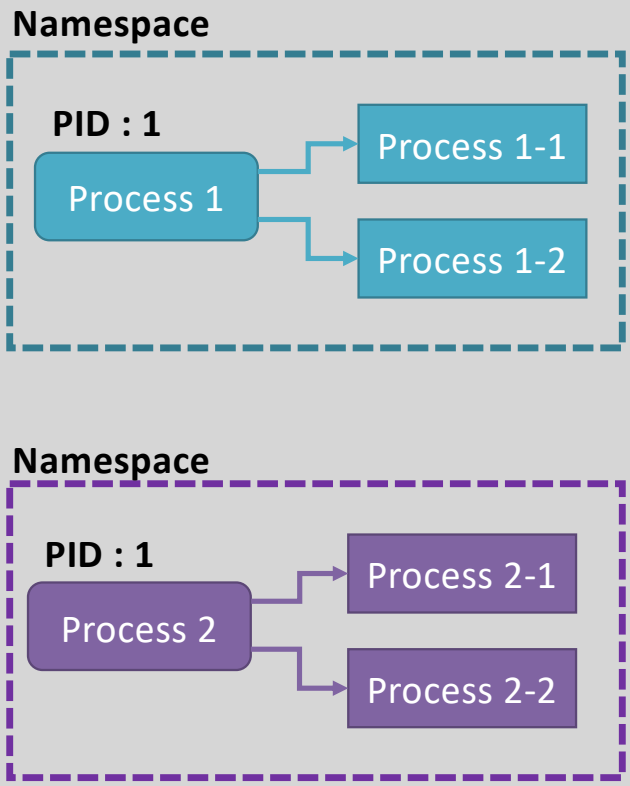
- When you run a program or command in Linux, Linux creates a process for it
- Each process in Linux is assigned a unique identifier called the **Process ID** (PID)
- Each process gets an allocation of CPU, Memory and other system resources
- Containers run as software processes
- Namespaces and Cgroups are core Linux kernel technologies



© DolfinED All rights reserved

Namespaces

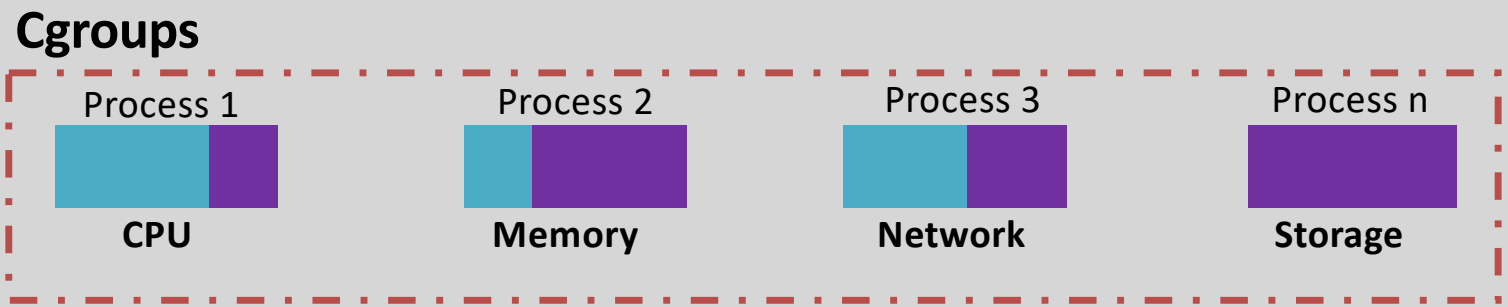
- Namespaces is a kernel security feature that was introduced in 2002
- A Namespace isolates resources (hard drive, networking, hostnames, users, etc) for **a particular process** on a machine
- Each process can be assigned to a namespace and can only see the resources connected to that namespace



© DolfinED All rights reserved

Control Groups (Cgroups)

- Cgroups were originally implemented by the Engineers at Google in 2006
- Cgroups limit, prioritize, and isolate resource usage (CPU, Disk, memory, I/O, and network bandwidth) of a group of processes
- Cgroups is used to achieve fine-grained control over how much resources a process (such as a container) can use



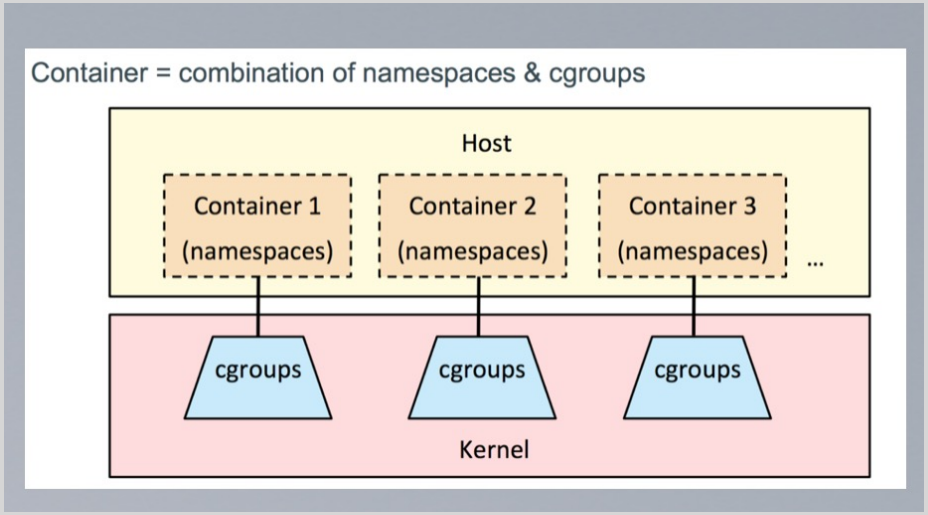
© DolfinED All rights reserved

Docker Containers - Namespaces & Cgroups

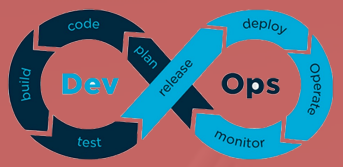
Both are fundamental to how containers work, providing the necessary isolation and resource management needed to run containers efficiently on a single host

When you run a container,

- **Namespaces** gives the container its isolated environment
- **Cgroups** ensures that the container consumes only the resources it is allocated



© DolfinED All rights reserved





Lab Environment Setup

Why Are We Using A Cloud VM for Learning?

- Practice in **Real-World** Environments
- Faster and cleaner to setup and delete
- Build Confidence in Working Remotely & Collaboratively
- Master the "Glue" Between Tools
- Linux OS
- Resume & Interview Ready – You practiced using the cloud
- **It is free!**

AWS Console

EC2 Instance

vs.



Your Laptop

© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Cannot create an AWS Account – Alternatives?

- Docker Desktop Personal (for Windows, Mac, and Linux)
https://www.docker.com/products/docker-desktop/

- Virtual Machine - Oracle Virtual Box (Windows, Mac, Linux)
https://www.oracle.com/ae/virtualization/technologies/vm/downloads/virtualbox-downloads.html

- Virtual Machine - VMware Workstation (Windows and Linux)
https://www.vmware.com/products/desktop-hypervisor/workstation-and-fusion

Time will be wasted before you are ready to go!

© DolfinED All rights reserved

Hands-on Labs (HoLs)

Creating an AWS Account



© DolfinED All rights reserved

Hands-on Labs (HoLs)

AWS Cost Budget & Admin User



© DolfinED All rights reserved

Hands-on Labs (HoLs)

AWS Free Tier



© DolfinED All rights reserved

Hands-on Labs (HoLs)

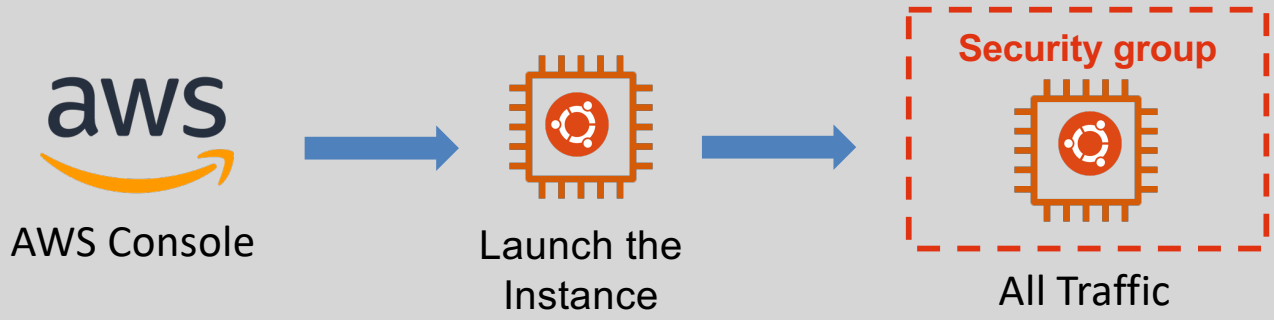
Launch an AWS EC2 Instance - Ubuntu Linux



© DolfinED All rights reserved

Launch an AWS ubuntu EC2 Instance - Steps

- Open the AWS console
- Launch a new Ubuntu EC2 instance (Virtual machine)
- Configure the security Group, Allow - All Traffic inbound



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Hands-on Labs (HoLs)

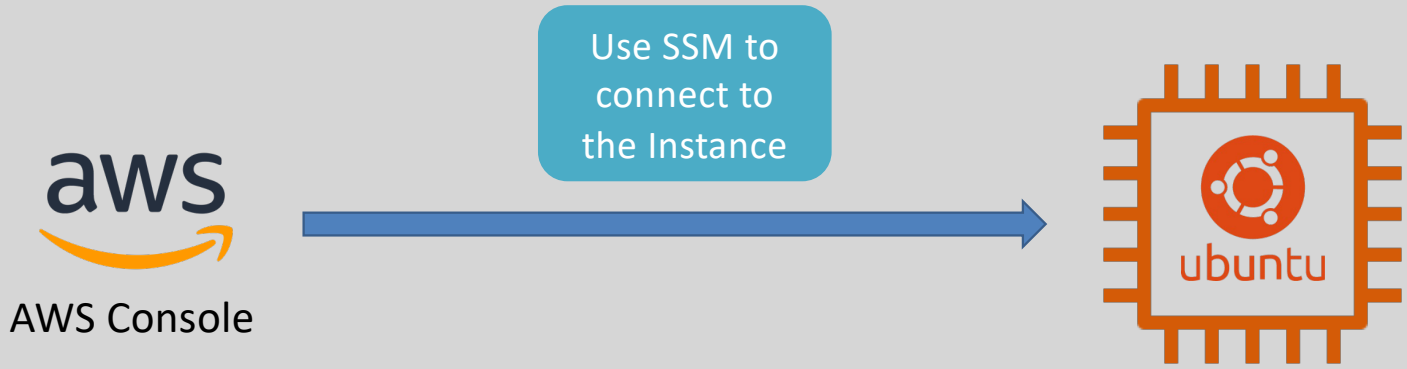
Use AWS SSM To Connect to the Ubuntu EC2 Instance on AWS



© DolfinED All rights reserved

Use SSM Session Manager to connect

- Open the AWS console
- Connect to the EC2 Instance from Session Manager



© DolfinED All rights reserved

Hands-on Labs (HoLs)

Use MobaXterm on Windows to Connect to the Ubuntu EC2 Instance



© DolfinED All rights reserved

Use MobaXterm to connect

- Download and install MobaXterm
- Use MobaXterm to connect to the Instance using SSH



© DolfinED All rights reserved

Hands-on Labs (HoLs)

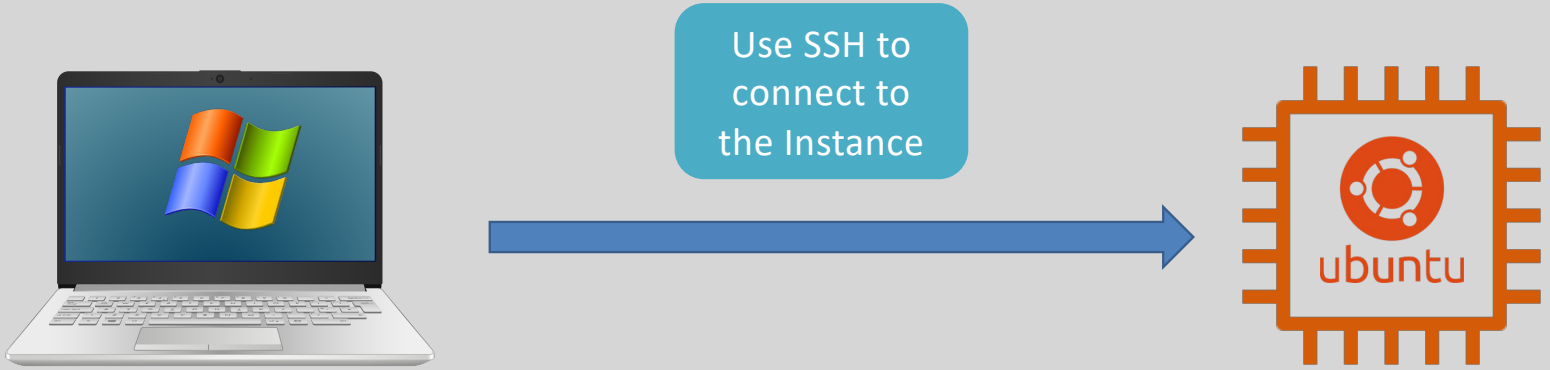
Use SSH on Windows To Connect to the Ubuntu EC2 Instance



© DolfinED All rights reserved

Use Windows 10+ OpenSSH to connect

- Verify the OpenSSH service is up and running
- Use OpenSSH in Windows CMD to connect



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Hands-on Labs (HoLs)

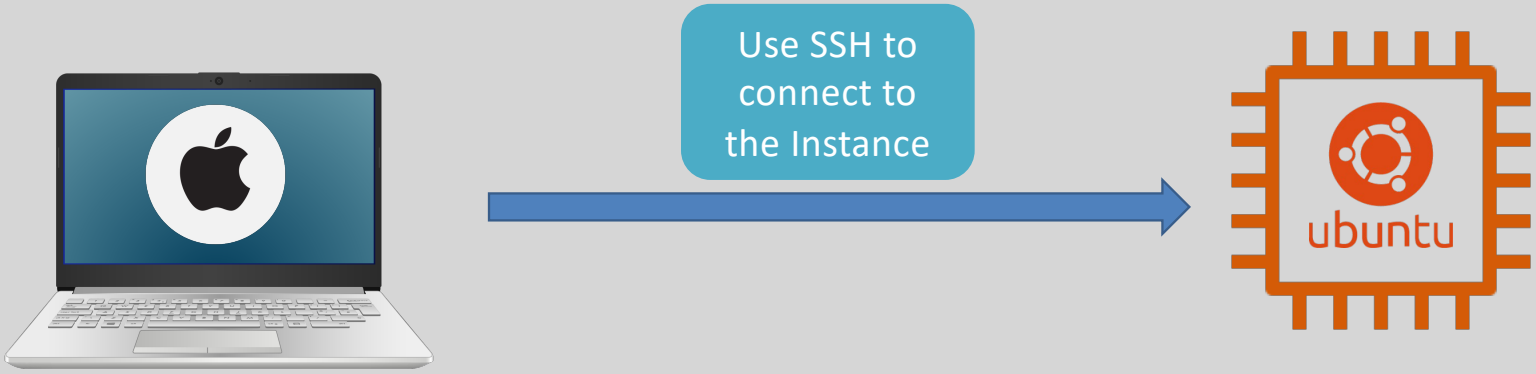
Use SSH on MacOS Devices
To Connect to the Ubuntu EC2 Instance



© DolfinED All rights reserved

Use MacOS Terminal App to connect

- Open the AWS console
- Use Terminal app to connect to the instance using SSH



© DolfinED All rights reserved

Hands-on Labs (HoLs)

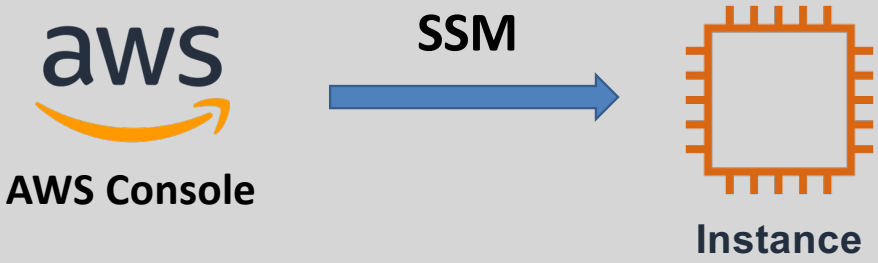
Installing Docker



© DolfinED All rights reserved

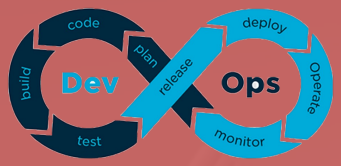
Demo - Installing Docker On An AWS Linux EC2 Instance

- Open the AWS console
- Login to the EC2 instance using AWS Systems Manager (SSM)



https://docs.docker.com/engine/install/ubuntu/

© DolfinED All rights reserved





Basic Docker Commands

docker pull

docker = dockerd

Syntax : docker pull [OPTIONS] IMAGE[:TAG|@DIGEST]

Download a docker image from a registry (default is Docker Hub)
:TAG Specific version

docker pull ubuntu	Pulls the latest version of the image ubuntu
docker pull ubuntu:20.04	Pulls version 20.04 of the image ubuntu
docker pull --all-tags ubuntu	Pulls all versions(tags) of the image ubuntu

© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

docker images

Syntax : docker images [OPTIONS]

List the available Docker images on the local Docker host

docker images	List all images on the local host
docker images -q	List all the available image IDs

docker ps

Syntax : docker ps [OPTIONS]

List the running containers on ther Docker host.
- It provides detailed information about each container's status, including its ID, name, status, and more

docker ps	List the running docker containers on the system
docker ps –a (or --all)	List all containers (running and stopped)
docker ps –q (or –quiet)	Only list the IDs of the running containers

© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

docker create

Syntax : docker create [OPTIONS] IMAGE [COMMAND] [ARG...]

Create a container in a stopped state (does not start it)

```
docker create --name my-container nginx
```

Create a container where its name is my-container and image is nginx
- If the image is not available locally, it will attempt to download it from Docker hub

```
docker create --name my-container ubuntu:20.04
```

Creates a container where its name is my-container and image is ubuntu release 20.04.

© DolfinED All rights reserved

docker start

Syntax : docker start [OPTIONS] CONTAINER [CONTAINER...]

Start an already created container that is not running
CONTAINER : name or id of the container to start

docker start my-container	Start the specified container (by name)
docker start abc1234	Start the specified container (by id)
docker start container1 container2 ...	Start multiple containers

© DolfinED All rights reserved

docker stop

Syntax : docker stop [OPTIONS] CONTAINER [CONTAINER...]

Stop a running Docker container gracefully
- If the process doesn't terminate within a certain timeout period (by default 10 seconds),

docker stop my-container	Stop the container my-container
docker stop -t 30 my-container	Stop the container my-container and allow it 30 seconds to stop gracefully
docker stop container1 container2	Stop the two specified containers

© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

A Container is a Software Process – Lets Prove It!

Syntax : ps [OPTIONS]

ps a	Show processes for **all users** (not just yours)
ps -A	Show **all processes**
ps x	Show processes **not attached to a terminal** (e.g., background services)
ps u	Show process **owner (user)** and display in **user-friendly** format

© DolfinED All rights reserved

docker exec

Syntax : docker exec [OPTIONS] CONTAINER COMMAND [ARG...]

Used to execute a command inside **a running** Docker container.

- It can be used to interact with a container's file system, processes, and environment.

```
docker exec my-container ls /usr/src/app
```

Run the “ls” command inside the container my-container to list the contents of the /usr/src/app directory

- Use it with non-interactive commands that do not expect user input like ls, cat, echo

© DolfinED All rights reserved

docker exec : -t Option

Syntax : docker exec [OPTIONS] CONTAINER COMMAND [ARG...]

```
docker exec -t my-container ls /app
```

This command allocates a terminal (tty) to run interactive commands (like bash) within the container

- The --tty (or -t) flag attaches a pseudo-TTY to the container
- Hence, you get access to input and output feature that TTY devices provide.

© DolfinED All rights reserved

docker exec : -i Option

Syntax : docker exec [OPTIONS] CONTAINER COMMAND [ARG...]

```
docker exec -i my-container cat /etc/hostname
```

i (or –interactive) : It keeps the container's Stdin open. This allows you send input (commands or else) to the container through the standard input
- Use it with interactive commands that expect user input like vim, bash, sh or applications that require input

© DolfinED All rights reserved

docker exec –it – Opening a Terminal Inside the Container

```
docker exec -it my-container /bin/bash
```

The two flags (-i and –t) are often used together to bind the I/O streams of the container to a pseudo terminal, creating an interactive terminal session

- /bin/bash specifies the type of the shell as bash (options bash, sh, Zsh, …etc)

```
docker exec -it my-container cat /etc/shells
```

Use this command to find out which shells are available for your container

© DolfinED All rights reserved

docker run

Syntax : docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Create and start a container from a specified image in a single step

docker run nginx	Create and start a container using latest nginx image
docker run --name cont1 ubuntu:20.04	Create and start a container from the ubuntu:20.04 image and assign it a name cont1
docker run -d nginx	-d: detached mode Create and start a container from image nginx in the background

© DolfinED All rights reserved

docker run -it Options

```
docker run -it ubuntu /bin/bash
```

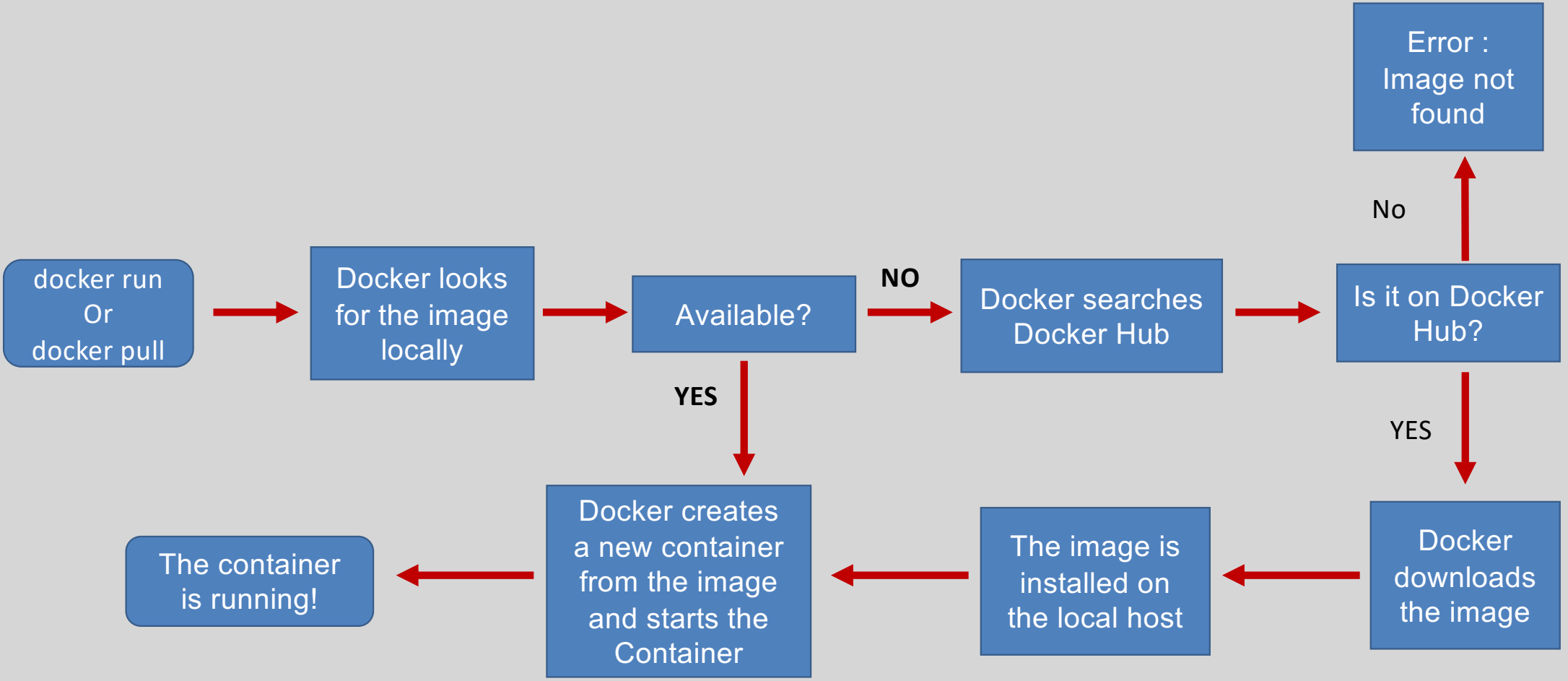
Create and run an ubuntu container and open an interactive shell of type Bash
- The container will run in interactive mode with a pseudo-TTY
- The command will also enter the shell (will move to the container prompt)

```
docker run -it ubuntu  bash
```

This will do the same if the bash shell is included in the container $PATH

© DolfinED All rights reserved

Locating The Docker Image



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

docker run --rm Options

```
docker run --rm ubuntu echo “this will self-destruct”
```

Create and run an ubuntu container, echo the sentence “this will self-destruct”, then delete the container

- Use it for one off containers such testing (curl, ping, ..etc)
- Do not use for long term containers (websites, databases..etc)

```
docker run -it --rm ubuntu bash
```

Create and run an ubuntu container, open an interactive shell into the container. Once the user exits from the shell, delete the container

© DolfinED All rights reserved

docker inspect

Syntax : docker inspect [OPTIONS] OBJECT [OBJECT...]

Use to retrieve detailed, low-level information about containers, images, volumes, or networks in JSON format.
- You can run docker inspect on both running and stopped containers

docker inspect my-container	Returns detailed information about the container (name, ID, Network, Config…)
docker inspect nginx:latest	Returns information about the image named nginx with the latest tag, including: Image ID, Creation time and more

docker commit

Syntax : docker commit [OPTIONS] CONTAINER IMAGE[:TAG]

The docker commit command is used to create a new image that captures the container's current state.

docker commit my-container new-image	Create a new image new-image from the container my-container
docker commit my-container new-image:1.0	Create the image new-image with the tag 1.0 from the container my-container

Installing Some Useful Utilities – Ping, Vim, Nano, Curl, IP address

Some of the docker image versions (Ubuntu:20.04 and Ubuntu:22.04 lack essential IP tools (such as ping, curl, vim, nano, ip address) that are handy in testing and troubleshooting

It is a good idea to start a container, install these tools and build your docker image that has these tools installed.

You can install these using the following commands

```
apt update
apt install -y iputils-ping curl nano vim iproute2
```

Update the container OS and install the required tools

docker commit (cont.)

Syntax : docker commit [OPTIONS] CONTAINER IMAGE[:TAG]

```
docker commit -a "John Doe <john.doe@example.com>" -m "Updated web server configuration" my-container my-image:1.1
```

Create the image my-image with the tag 1.1 from the container my-container
-a defines the author metadata of the image as John Doe
-m this is a commit message to explain what is changed/added

docker rm

Syntax : docker rm [OPTIONS] CONTAINER [CONTAINER...]

Remove or delete one or more containers.
- Only removes stopped containers with this command, not the running ones.

docker rm my-container	Remove my-container
docker rm container1 container2 container3	Remove the specified containers
docker rm –f my-container	Stop my-container and remove it even if it's running

DolfinED
Transforming Lives Through Learning

docker rmi

Syntax : docker rmi [OPTIONS] IMAGE [IMAGE...]

Remove one or more Docker images from the local system.

docker rmi my-image	Remove the image my-image
docker rmi image1 image2 image3	Remove the specified images
docker rmi -f my-image	Force remove my-image that has dependent containers created from it

docker logs

Syntax : docker logs [OPTIONS] CONTAINER

Use to retrieve the **logs** of a container.
- This can be helpful for troubleshooting, debugging, and monitoring the output of processes running in a container

docker logs my-container

Print the logs of the container named my-container

docker logs -f my-container

Display the logs and keep updating the terminal with new log messages as they are written

docker --help

Syntax : docker --help

Display the help documentation for Docker, including the list of available commands and their descriptions.

docker run --help

List of available options and flags for the docker run command.

docker image prune

Syntax : docker image prune [OPTIONS]

Remove unused docker images on the host to free up space

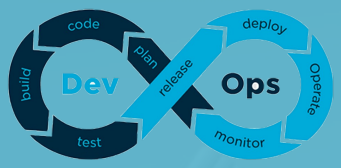
docker image prune	Remove unused and untagged images (images without a tag)
docker image prune –a (or –all)	Remove unused images even if they are tagged

Hands-on Labs (HoLs)

Basic Docker Commands



© DolfinED All rights reserved





Container Start-up Explained

CMD & Container Behavior

Docker Container Lifecycle Depends on the command to be executed when the container starts

- When a Docker container is started, it **runs the command defined by CMD** in the image (or the one passed to it with the docker run command)
- That command becomes the **main process (PID 1)** inside the container.
  - **If the CMD process keeps running →** the container stays alive
  - **If the CMD process exits →** the container stops automatically

Nginx

```
"Cmd": [
        "nginx",
        "-g",
        "daemon off;"
    ],
```

Ubuntu

```
"Cmd": [
        "/bin/bash"
    ],
```

© DolfinED All rights reserved

CMD & Container Behavior – Nginx (default CMD)

Docker Container Lifecycle Depends on the command to be executed when the container starts

- Usually, a CMD in the image defines the default command to be executed
- For example, with Nginx image, the default CMD is to run nginx in the foreground which keeps the container Up

```
docker run nginx
```

Create a container with default nginx image settings. The container will continue to be Up

```
"Cmd": [
            "nginx",
            "-g",
            "daemon off;"
        ],
```

What it does:
nginx: runs the Nginx web server
-g: passes a global configuration directive to Nginx
"daemon off;": tells Nginx not to run as a background process (daemon)

© DolfinED All rights reserved

CMD & Container Behavior – Nginx Interactive (Overriding default CMD)

Docker Container Lifecycle Depends on the command to be executed when the container starts

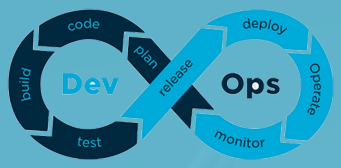
```
docker run –it nginx /bin/bash
```

```
"Cmd": [
            "nginx",
            "-g",
            "daemon off;"
        ],
```

Create a container and overrides the default CMD in the image with /bin/bash in interactive mode

- Bash will continue to run because of the “-it” and the container continues to run
- Once exited, the container stops

© DolfinED All rights reserved



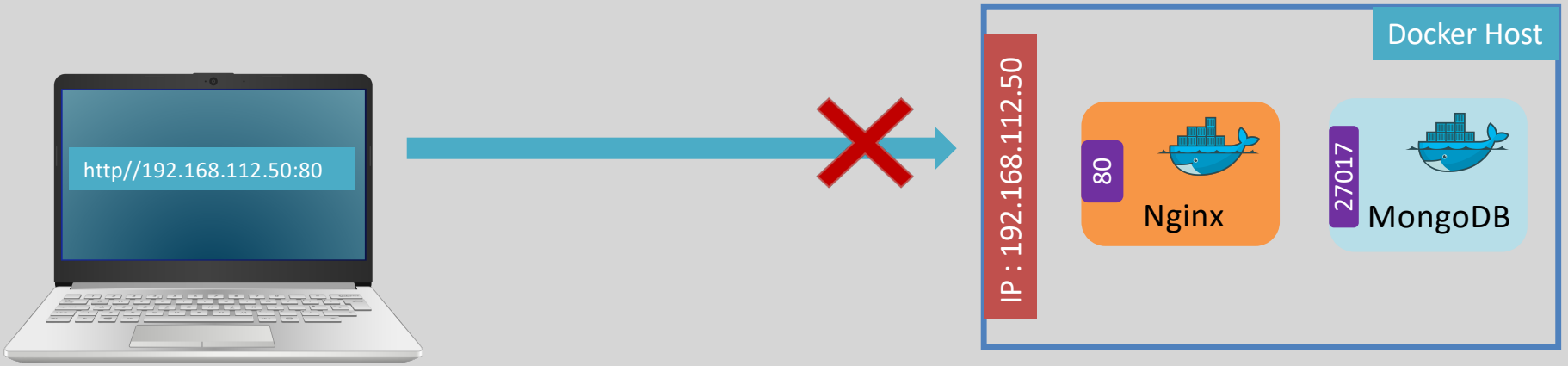


Container Port Default Behavior

Docker Container Port

When we create a container for an application,

- This does not automatically expose or publish the container port
  - ➢ The container is not reachable from the host and external networks
- We need to expose the container port to allow external (inbound) access to the container



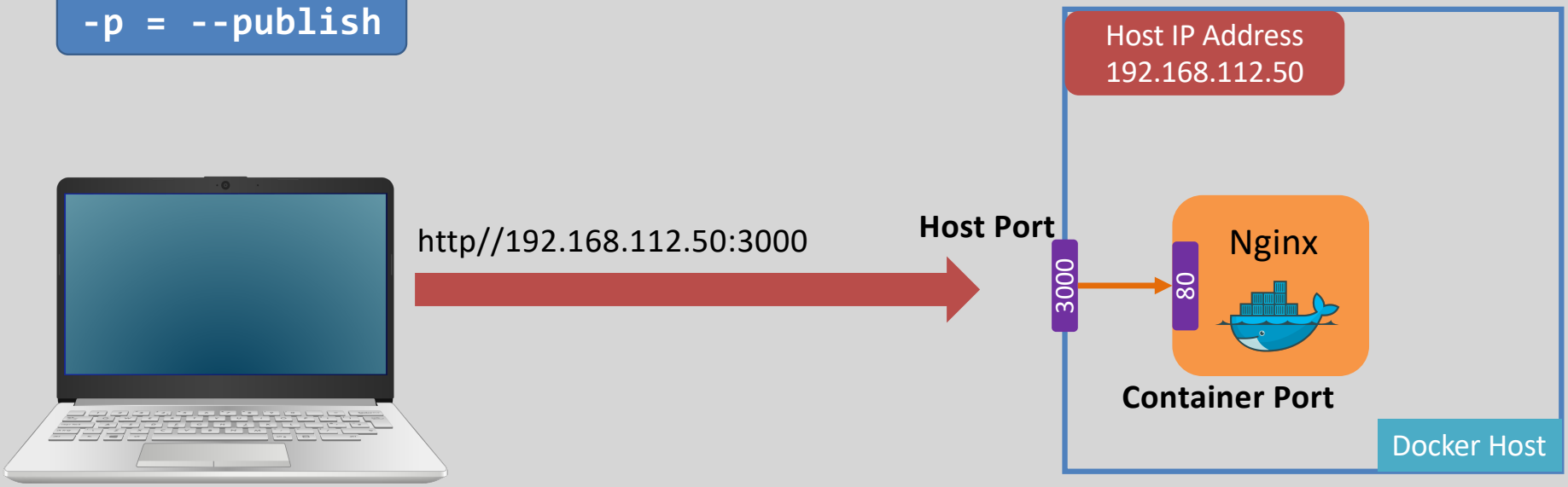
© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Static Container Port Mapping (Binding) - docker run Command

docker run –p 3000:80 nginx

`-p = --publish`



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Static Container Port Mapping (Binding) - docker run Command (cont.)

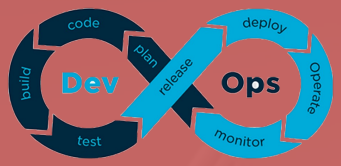
```
docker run –p 3000:80 nginx
```

--publish or –p Map or publish a container's port (80) to the specified host port (3000)

```
docker run -d -p 8080:80  -p 8443:443 nginx
```

Map or publish container port 80 to host port 8080. Also, map container port 443 to host port 8443

© DolfinED All rights reserved





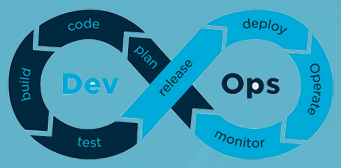
Docker Files

Hands-on Labs (HoLs)

Setup VS Code (The IDE) – Installing the Docker Extension



© DolfinED All rights reserved





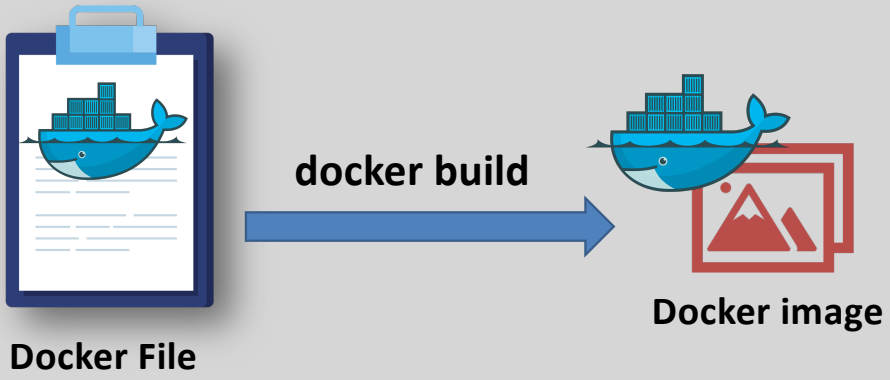
Dockerfile
FROM and RUN Instructions

Docker File

- A Dockerfile is a text document that contains all the commands a user could use manually on the CLI to build an application
- Docker can build images automatically by reading the instructions from a Dockerfile
- The command ”docker build” reads the instructions in a docker file and creates a docker image

Dockerfile

```
FROM ubuntu
MAINTAINER John Doe
RUN apt-get update
CMD ["echo", "Hello from DolfinED"]
```



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Docker File Structure - FROM

The FROM instruction will set the base image for the Dockerfile

- Every Dockerfile instruction that follows FROM applies to this base image.

Syntax : FROM <image>[:tag]

Example: FROM ubuntu

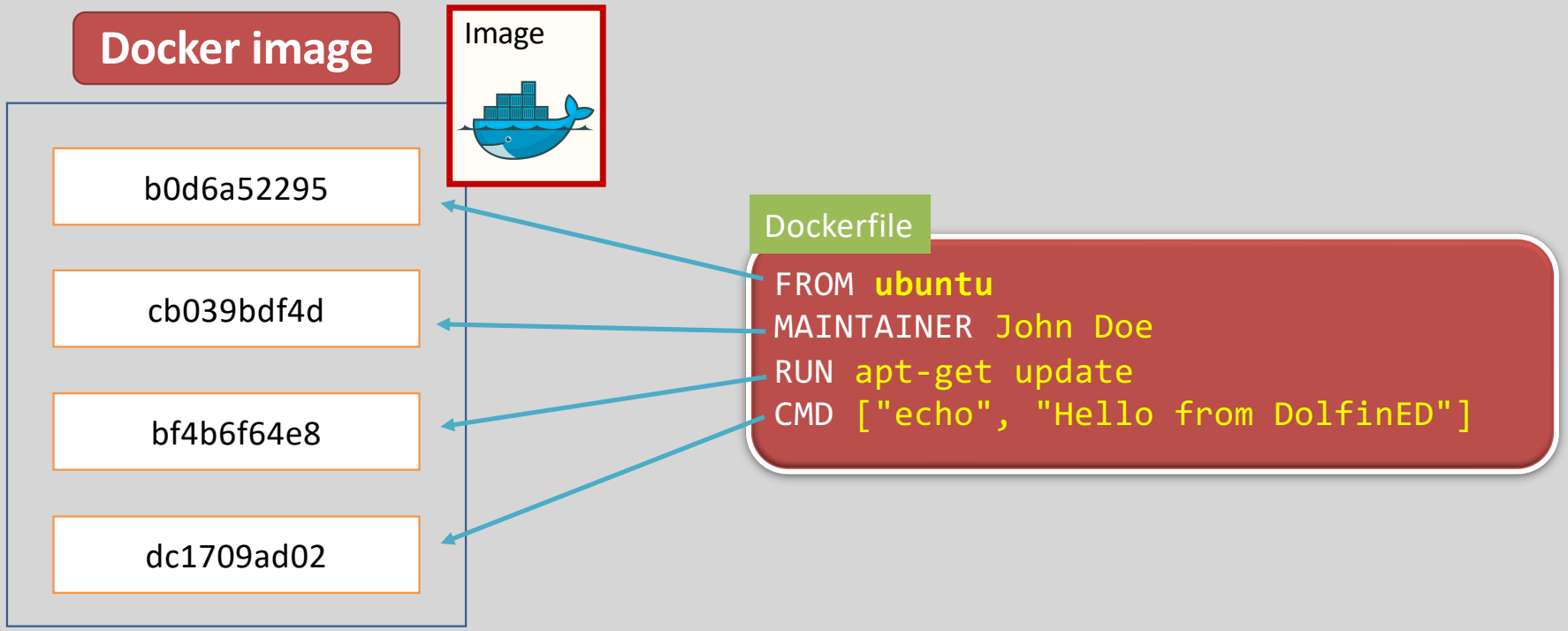
Pull the latest base Ubuntu image

© DolfinED All rights reserved

Docker Image Layers

The docker image consists of layers

- Each layer maps to a docker instruction in the docker file



© DolfinED All rights reserved

Docker File Structure - RUN

RUN - runs one or more commands

- The RUN instruction will execute any commands in a new image layer and commit the results.

Syntax : RUN <command(a)>

RUN apt-get update

RUN apt-get install –y vim

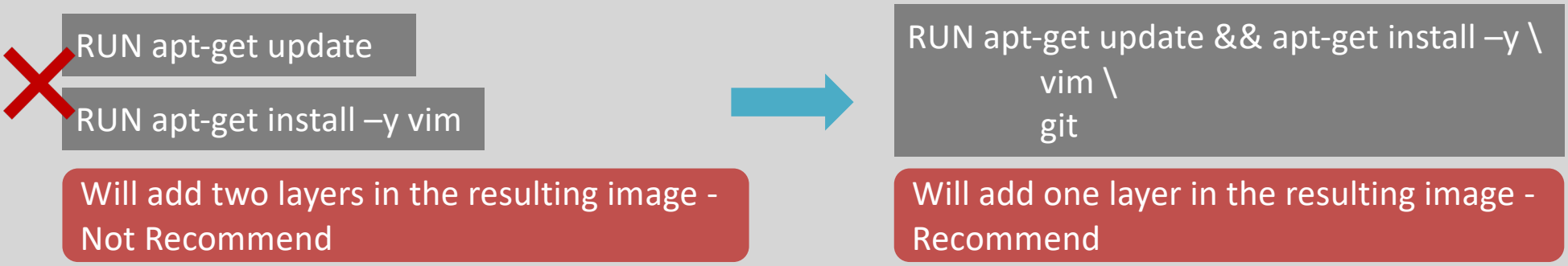
Will add two layers in the resulting image - Not Recommend

© DolfinED All rights reserved

Optimizing Image Layers

It is possible to perform multiple commands using multiple lines with a single RUN statement.

- This will result in fewer image layers

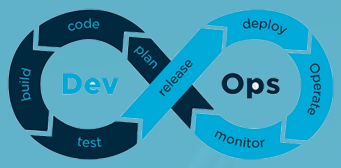


© DolfinED All rights reserved

Inspecting Image Layers – docker history command

```
docker history <image name>
```

Use this command to display image layers and their details





Dockerfile
CMD and docker build

Docker File Structure - CMD

CMD is used to specify the command to be executed when the container is started

- Only one CMD instruction should exist in a Dockerfile
- If there are more than one CMD instructions in a Dockerfile, only the last CMD instruction will take effect
- The CMD can be overridden at container start time

Syntax : CMD ["executable","param1","param2"]

```
CMD ["echo", "Hello from DolfinED"]
```

OUTPUT: Hello from DolfinED

```
CMD ["python", "app.py"]
```

This will execute “python app.py” at the container start

© DolfinED All rights reserved

Docker File Structure – CMD vs. RUN

CMD	RUN
Executes at Runtime	Executes at Build time
Runs a command in a container when it is started	• Adds layers to images • Used to install apps

© DolfinED All rights reserved

docker build command explained

Syntax : docker build [OPTIONS] PATH | URL | -

- PATH: The build context — usually the current directory .
- OPTIONS: Flags you can pass to customize the build
- Dockerfile: By default, Docker looks for a file named Dockerfile, but you can specify a different one

```
docker build -t myapp .
```

Build the image myapp from the dockerfile in the current directory
The current directory is the build context

docker build command explained – Different file name and build context path

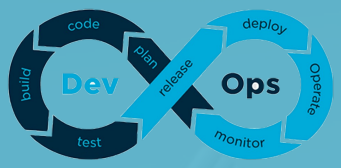
Syntax : docker build [OPTIONS] PATH | URL | -

PATH: The build context — usually the current directory .

```
docker build -f files/test.txt -t image3 /imagefiles
```

- Build the image myapp from the test.txt file in the /files directory
- The /imagefiles directory is the build context (any build files need to be here)

```
docker build -f /home/ubuntu/apps/imagefiles/test.txt -t image3 /home/ubuntu/build/contextfiles/app1
```





Docker Files –
docker commit vs. docker build

docker commit vs. docker build

Feature	`docker commit`	`docker build`
Purpose	Save changes from a container into an image	Create an image from a Dockerfile with specific instructions
Use Case	Ad-hoc, manual changes to a running container	Automated, reproducible builds with a `Dockerfile`
Reproducibility	Not easily reproducible	Fully reproducible and version-controlled
Workflow	Interactive, manual	Structured, automated
Version Control	No version control of changes made to container	Version-controlled via `Dockerfile`
Best Practice	Not recommended for production or complex use cases	Best practice for creating production-ready and reusable images
Image Layers	Single layer image, no layer history	Multi-layer image with layer history

Hands-on Labs (HoLs)

Docker File



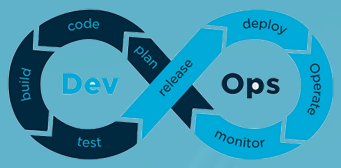
© DolfinED All rights reserved

Hands-on Labs (HoLs)

Docker File – Optimizing Image Layers



© DolfinED All rights reserved





Docker Files - ENTRYPOINT

Docker File Structure - ENTRYPOINT

ENTRYPOINT it is used to specify the starting command to be executed when the container is started

- Anything added to the end of the **“docker run”** command (at Runtime) will be used as a parameter input (appended) to the ENTRYPOINT instruction

Syntax : ENTRYPOINT ["executable", "param1", "param2"]

Dockerfile

```
FROM ubuntu
……
ENTRYPOINT ["echo"]
```

docker build –t myimage .

docker run myimage welcome from dolfined

The OUTPUT: welcome from dolfined

© DolfinED All rights reserved

Docker File Structure – CMD vs. ENTRYPOINT

	CMD	ENTRYPOINT
Purpose	Executes when the container is started if not overridden	The main command to run when the container is started
Can be overridden?	Yes, by running a different command at container start time	Can be overridden using the “--entrypoint” option of the docker run command

© DolfinED All rights reserved

CMD & ENTRYPOINT Used Together

If both CMD and ENTRYPOINT exist in a Dockerfile, they are combined into a single command to start the container

- Make sure that CMD is used to pass parameters or default parameters to ENTROYPOINT

Dockerfile

```
FROM python:3.10-slim
COPY app.py /app.py
ENTRYPOINT ["python3"]
CMD ["/app.py"]
```

```
docker build –t myimage .
```

```
docker run myimage
```

This will run “python3 /app.py” at the container start

© DolfinED All rights reserved

CMD & ENTRYPOINT Used Together (cont.)

Anything added to the end of the **"docker run"** command (at Runtime) will be used as a parameter input (appended) to the ENTRYPOINT instruction

Dockerfile

```
FROM python:3.10-slim
COPY app.py /app.py
ENTRYPOINT ["python3"]
CMD ["/app.py"]
```

```
docker build –t myimage .
```

```
docker run myimage  app123.py
```

This will run "python3 app123.py" at the container start and the CMD default instruction (/app.py) is overridden

© DolfinED All rights reserved

CMD & ENTRYPOINT Used Together – ERRORS?

Do not use CMD to execute a different command when used with ENTRYPOINT

Dockerfile

```
FROM python:3.10-slim
COPY app.py /app.py
ENTRYPOINT ["python3"]
CMD [”echo”, “Hello”]
```

```
docker build –t myimage .
```

```
docker run myimage
```

This will throw an error as it will be translated into “python3 echo Hello”

```
docker run myimage app123.py
```

This will run “python3 app123.py” at the container start and the CMD [“echo’, “Hello”] will be overridden

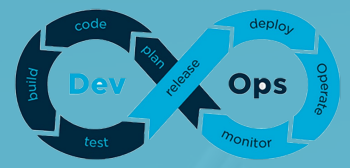
© DolfinED All rights reserved

Hands-on Labs (HoLs)

Docker File – ENTRYPOINT



© DolfinED All rights reserved





Docker Files –
EXPOSE & COPY Instructions

Docker File Structure - EXPOSE

The EXPOSE instruction informs Docker that the container listens on the specified network ports at runtime

Syntax : EXPOSE <port>

EXPOSE 80

Exposes port 80 or http

EXPOSE 80 443

Exposes ports 80 and 443 (HTTP and HTTPS)

© DolfinED All rights reserved

Docker File Structure – EXPOSE (cont.)

EXPOSE is informational only; it does not actually publish or expose the port or open it to the host

- You still need to publish the port(s)
- It is useful for tools that auto generate configurations (Docker Compose, Swarm, K8s)

© DolfinED All rights reserved

Docker File Structure - COPY

The COPY instruction copies new files or directories from <src> and adds them to the filesystem of the container at the path <dest>

- It can copy files from the build context only (cannot copy files from anywhere)
  - When you specify the source path it has to be relative path & within the build context

Syntax : COPY <src> <dest>

© DolfinED All rights reserved

Docker File Structure – COPY (cont.)

```
---
COPY app/utils    /app/code/utils
---
COPY app/main.py   /app/code/
```

```
# Copy only the 'app/utils' directory to  /app/code/utils' inside the image
# Copy just the app/main.py file into  /app/code
```

Project Structure (Host)

```
.
├── Dockerfile
├── app/
│   ├── main.py
│   └── utils/
│       └── helper.py
```

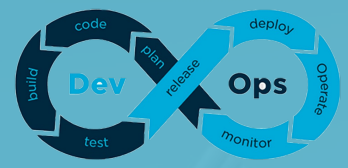
© DolfinED All rights reserved

Hands-on Labs (HoLs)

Docker File –
EXPOSE and COPY Instructions



© DolfinED All rights reserved





Docker Files – WORKDIR, ENV, LABEL Instructions

Docker File Structure - WORKDIR

WORKDIR is used to define the working directory of the docker container at any given time

- It creates the directory specified by “path” and runs all the following container commands (RUN, CMD, ADD, COPY..etc) into this directory

Syntax : WORKDIR </path>

```
WORKDIR /app
```

Create this directory /app in the image and any container when it is created from the image

© DolfinED All rights reserved

Docker File Structure – WORKDIR (cont.)

- This </path> is a container path, not a docker host path
- The </path> will be in the resulting image during build time and in the container during runtime

Syntax : WORKDIR </path>

```
WORKDIR /app
```

Create this directory /app in the image and any container when it is created from the image

© DolfinED All rights reserved

Docker File Example – COPY & WORKDIR Together

```
FROM python:3.10-slim

# Create a working directory
WORKDIR /usr/src/app

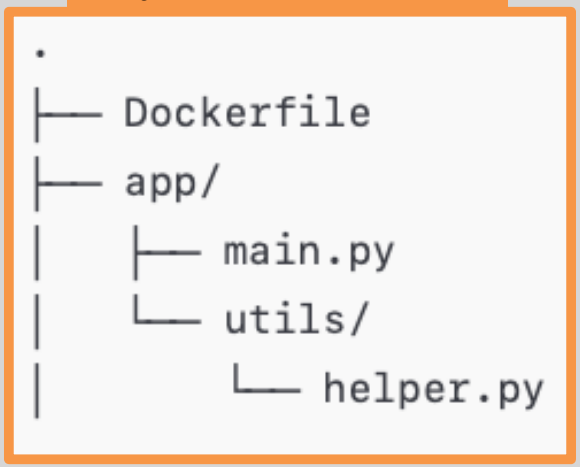
# Copy only the 'app/utils' directory to '/usr/src/app/code/utils'
inside the image
COPY app/utils   /usr/src/app/code/utils

# Copy just the main.py file into /usr/src/app/code
COPY app/main.py   /usr/src/app/code/

# Set the working directory to where your app's main script lives
WORKDIR /usr/src/app/code

CMD ["python3", "main.py"]
```

Project Structure (Host)



The container's initial working directory will be **/usr/src/app/code**

© DolfinED All rights reserved

Docker File Structure - ENV

The ENV instruction allows for setting environment variables for the Docker container

Syntax : ENV <key> <value>

```
ENV name=DolfinED
CMD ["sh", "-c", "echo $name"]
```

OUTPUT: DolfinED

© DolfinED All rights reserved

Docker File Structure – ENV – Overriding Defaults at Runtime

You can use the ENV to set the default values of the variables.

- You can override these default values at runtime

```
FROM alpine
# Install ping and curl
RUN apk add --no-cache iputils curl
# Set default environment variables
ENV action=ping
ENV target=8.8.8.8

# Use shell to allow variable expansion
CMD ["sh", "-c", "exec $action $target"]
```

```
docker build –t myimage  .
```

```
docker run -e action=curl -e target=10.10.1.1 myimage
```

OUTPUT: curl 10.10.1.1 (without –e option at run time – the Output would be ping 8.8.8.8)
sh : Invokes a shell inside the container
-c : tells the shell to run the following string as a command

© DolfinED All rights reserved

Docker File Structure - LABEL

- The LABEL instruction in Docker is a way to add metadata to an image
- This metadata can include information about the image, such as the author, version, description, environment, or other custom data

Syntax : LABEL key=“value"

```
LABEL Owner="DolfinED Academy Team <team@support.dolfined.com>"
```

© DolfinED All rights reserved

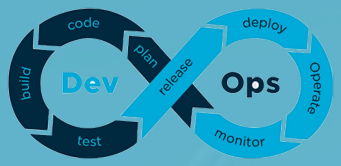
Hands-on Labs (HoLs)

Docker File –
WORKDIR, ENV, and LABEL Instructions



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning





Docker Files – ADD Instruction

Docker File Structure - ADD

ADD is like COPY used to copy files from source to a destination but has additional capabilities:

- Can fetch files from **remote URLs** (not recommended)
- ADD can automatically extract compressed data (.tar, .tar.gz, ..tgz, .bz2…etc)

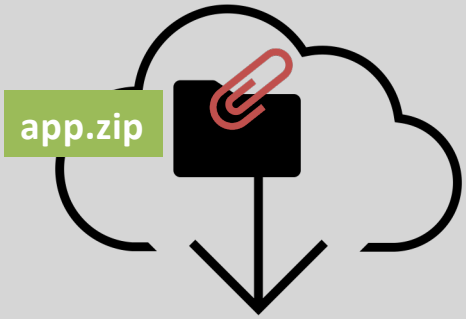
Syntax: ADD <src> <dst>

```
ADD ./archive.tar.gz /app/
```

- This will extract the content of the archive.tar.gz into the /app directory
- Docker Best Practice : Use COPY unless you specifically need ADD's tar auto-extraction feature

© DolfinED All rights reserved

Docker File Structure - ADD



Docker host

```
.
├── archive.tar.gz
├── data/
│   ├── file1.txt
│   └── file2.txt
└── test.txt
```

Image/Container

```
/app
├── test-copy.txt
├── extracted-files/
│   └── data/
│       ├── file1.txt
│       └── file2.txt
└── remote/
    ├── [contents of app.zip...]
```

```
FROM alpine
RUN apk add --no-cache unzip
WORKDIR /app
COPY test.txt  ./test-copy.txt
ADD archive.tar.gz   ./extracted-files/
RUN mkdir -p /app/remote
ADD https://dolfined-rds-lab-aug2023.s3.us-east-1.amazonaws.com/app.zip   ./remote/app.zip
RUN unzip ./remote/app.zip -d ./remote && rm ./remote/app.zip
CMD ["ls", "-l", "/app"]
```

© DolfinED All rights reserved

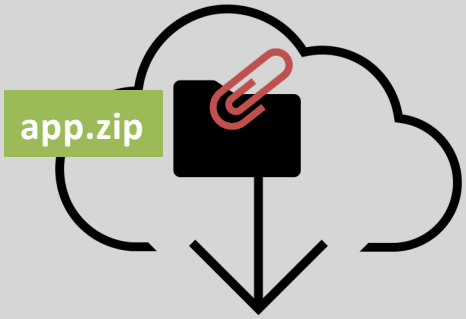
Hands-on Labs (HoLs)

Docker File –
ADD Instruction



© DolfinED All rights reserved

Docker File Structure - ADD



Docker host

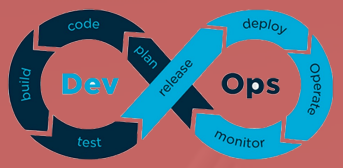
```
.
├── archive.tar.gz
├── data/
│   ├── file1.txt
│   └── file2.txt
└── test.txt
```

Image/Container

```
/app
├── test-copy.txt
├── extracted-files/
│   └── data/
│       ├── file1.txt
│       └── file2.txt
└── remote/
    ├── [contents of app.zip...]
```

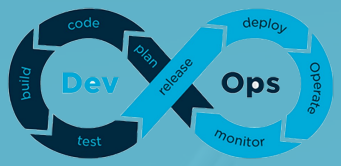
```
FROM alpine
RUN apk add --no-cache unzip
WORKDIR /app
COPY test.txt  ./test-copy.txt
ADD archive.tar.gz   ./extracted-files/
RUN mkdir -p /app/remote
ADD https://dolfined-rds-lab-aug2023.s3.us-east-1.amazonaws.com/app.zip   ./remote/app.zip
RUN unzip ./remote/app.zip -d ./remote && rm ./remote/app.zip
CMD ["ls", "-l", "/app"]
```

© DolfinED All rights reserved





Docker Images & Docker Registry



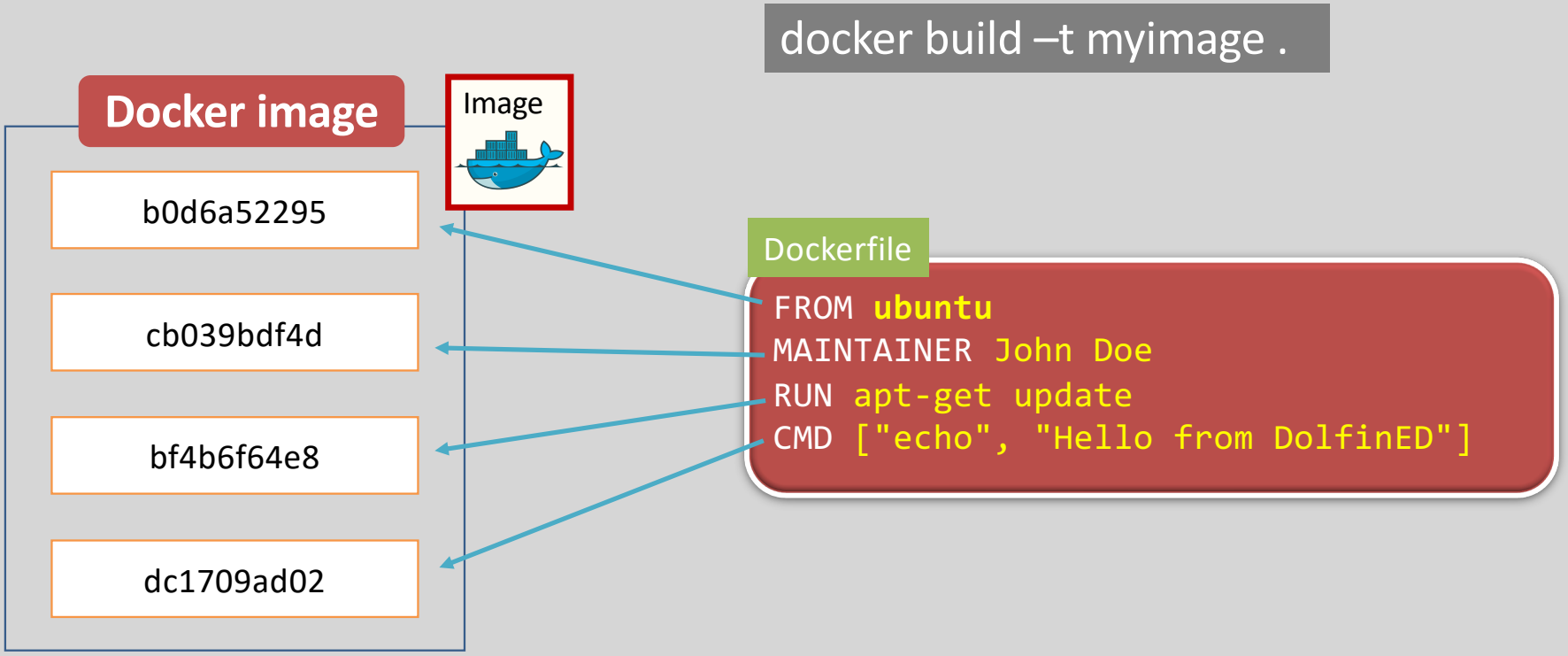


Docker Images

Docker Image – Build Time

Image build time is the phase when Docker reads the Dockerfile line by line and turns it into a Docker image

- Image layers are read-only



© DolfinED All rights reserved

Docker Image Layers – Best Practice Tips

- Group COPY and RUN wisely to improve layer caching
- Put less frequently changed lines early (like installing dependencies)
- Avoid unnecessary RUN commands to reduce layer count and image size

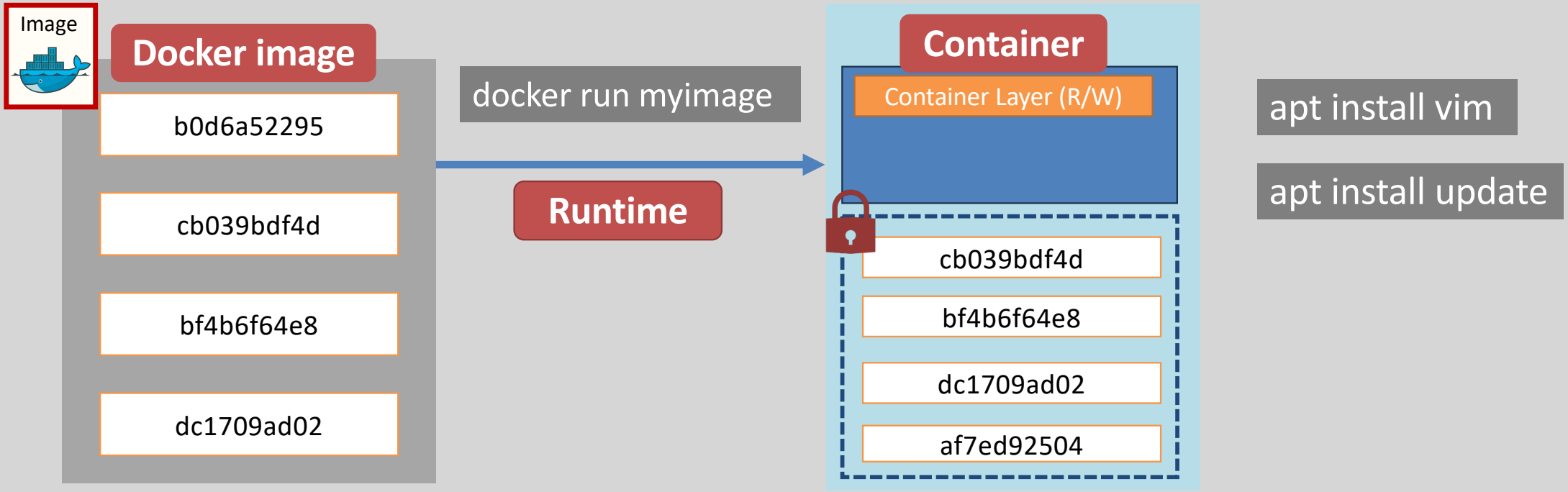
Goal	What's Better?	Why?
🏎️ Fast builds	**More layers (smart caching)**	Docker can skip unchanged layers
💾 Smaller images	**Fewer layers, clean builds**	Fewer intermediate files/layers
🛠️ Maintainability	**Readable layers**	Easier debugging and edits
📦 Layer reusability	**Split steps logically**	Base images can be reused/cached

© DolfinED All rights reserved

Docker Containers - Runtime

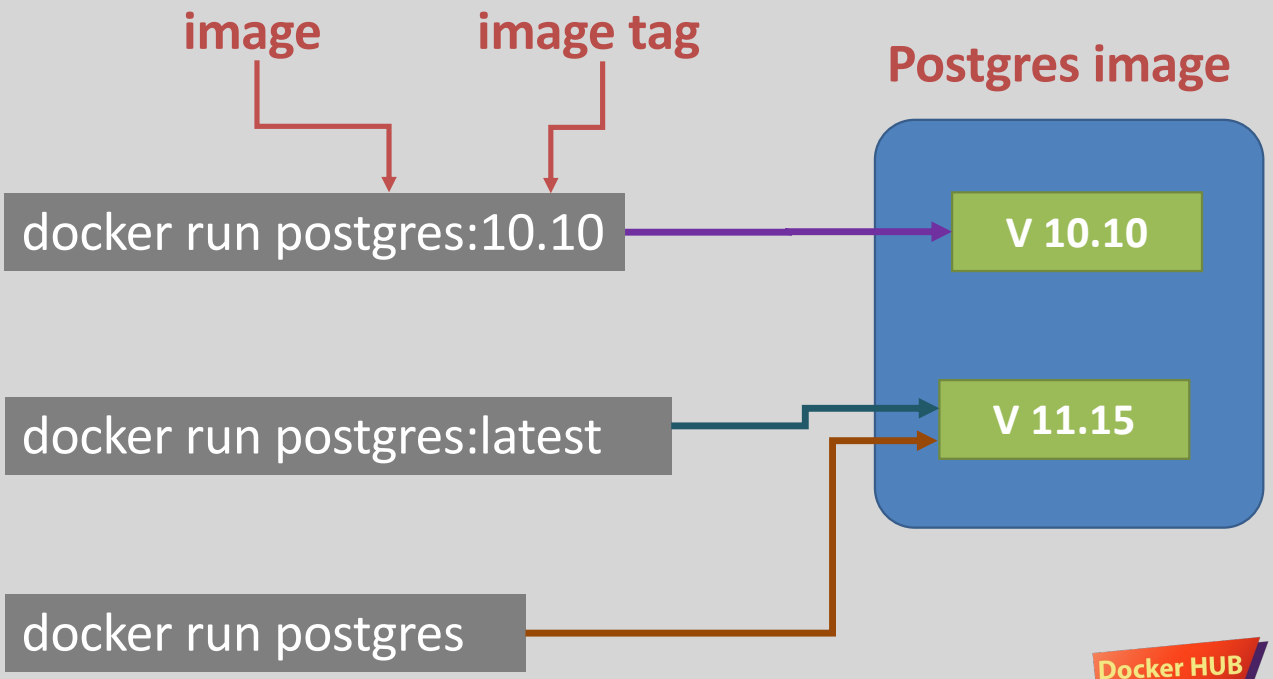
Container runtime is the container execution phase

- The image is used to create the container and execute the CMD or ENTRYPOINT command



© DolfinED All rights reserved

Image tag



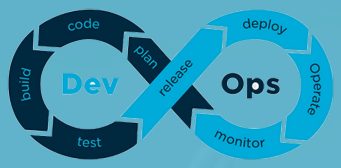

```
[root@localhost ~]# docker run -d postgres:10.10
Unable to find image 'postgres:10.10' locally
10.10: Pulling from library/postgres
80369df48736: Pull complete
b18dd0a6efec: Pull complete
5c20c5b8227d: Pull complete
c5a7f905c8ec: Pull complete
5a3f55930dd8: Pull complete
ffc097878b09: Pull complete
3106d02490d4: Pull complete
88d1fc513b8f: Pull complete
f19250dffc5e: Pull complete
756351b7a443: Pull complete
0d24b08575ba: Pull complete
31babd3be108: Pull complete
16f2724dc303: Pull complete
69802daaa561: Pull complete
```

```
[root@localhost ~]# docker run -d postgres:latest
Unable to find image 'postgres:latest' locally
latest: Pulling from library/postgres
42c077c10790: Already exists
3c2843bc3122: Pull complete
12e1d6a2dd60: Pull complete
9ae1101c4068: Pull complete
fb05d2fd4701: Pull complete
9785a964a677: Pull complete
16fc798b0e72: Pull complete
f1a0bfa2327a: Pull complete
fd2d68720749: Pull complete
83b23beac012: Pull complete
7962517582d4: Pull complete
6b4a569b8013: Pull complete
ad029fbc8984: Pull complete
```



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning





Docker Registry

Docker Registry – Pushing Images



Docker image

Host

docker push

Docker HUB



Public Repo

Private Repo

© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Docker Image Naming Convention

Docker Hub has a **specific naming convention** for Docker images that helps with organization, versioning, and discovery.

```
[registry/][username_or_org/]repository[:tag]
```

- Registry : (Optional) the domain of the container registry – default to docker.io
- Username or organization : The name of the user or organization (required for non-official images only)
- Repository : the name of the image or the project
- Tag (Optional) : The version – defaults to Latest if not included

© DolfinED All rights reserved

Image Rename – docker tag Command

The docker tag command is used to **add a new name (or alias)** to an existing Docker image (add a custom name to an image)

Syntax : docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]

```
docker tag python:3.10 my-python-app:latest
```

Give the image python:3.10 a custom name my-python-app:latest

© DolfinED All rights reserved

Docker Registry – docker tag Command

Images must be correctly tagged with the intended registry & image name before they are pushed

Registry / User/Account / Imag/Repo

Generic Syntax : docker tag SOURCE_IMAGE REGISTRY_HOST/USERNAME/REPO:TAG

```
docker tag myapp  registry.mycorp.com/devops/myapp:1.0.0
```

This will tag the image myapp with the name registry.mycorp.com/devops/myapp:1.0.0 in preparation to be pushed to the private registry registry.mycorp.com, under account devops, under repo/image myapp with the tag 1.0.0

© DolfinED All rights reserved

Docker Registry – docker login Command

Registry

docker login Registry_Host

Log in to a Docker registry

© DolfinED All rights reserved

DockerHub Registry – docker tag Command

For Dockerhub, you do not need to specify the registry as it defaults to docker.io

- The local image name and the repo name can be the same (they do not have to)

User/Account / Imag/Repo

```
docker tag LOCAL_IMAGE_NAME:tag USERNAME/REPO:TAG
```

```
docker tag myimage myusername/myapp:latest
```

myimage is tagged in preparation for pushing to docker hub registry, under account myusername, under the repo myapp with the tag latest

```
docker tag myimage myusername/myapp:v1
```

© DolfinED All rights reserved

Docker Registry – docker push Command

docker push uploads a locally tagged image to a Docker registry (like Docker Hub, GitHub Container Registry, AWS ECR, etc.)

Registry / User/Account / Imag/Repo

Syntax : docker push [REGISTRY_HOST]/USERNAME/REPO:TAG

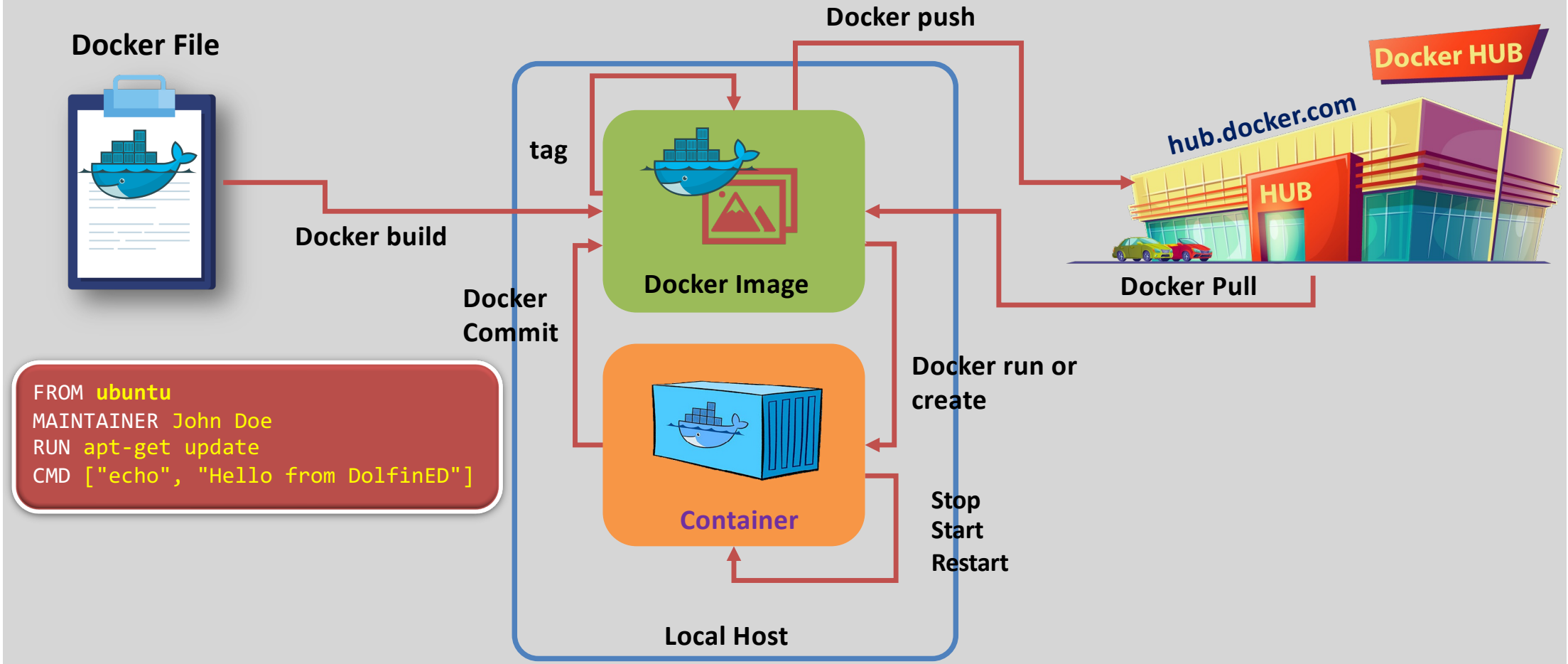
Docker Hub : docker push myapp myusername/myapp:latest

This pushes the image to Docker Hub under the account myusername, repo myapp, with the tag “latest”

Generic Registry : docker push registry.mycorp.com/devops/myapp:1.0.0

© DolfinED All rights reserved

Docker Image Lifecycle




```
FROM ubuntu
MAINTAINER John Doe
RUN apt-get update
CMD ["echo", "Hello from DolfinED"]
```

© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Hands-on Labs (HoLs)

Creating a Docker Hub Account



© DolfinED All rights reserved

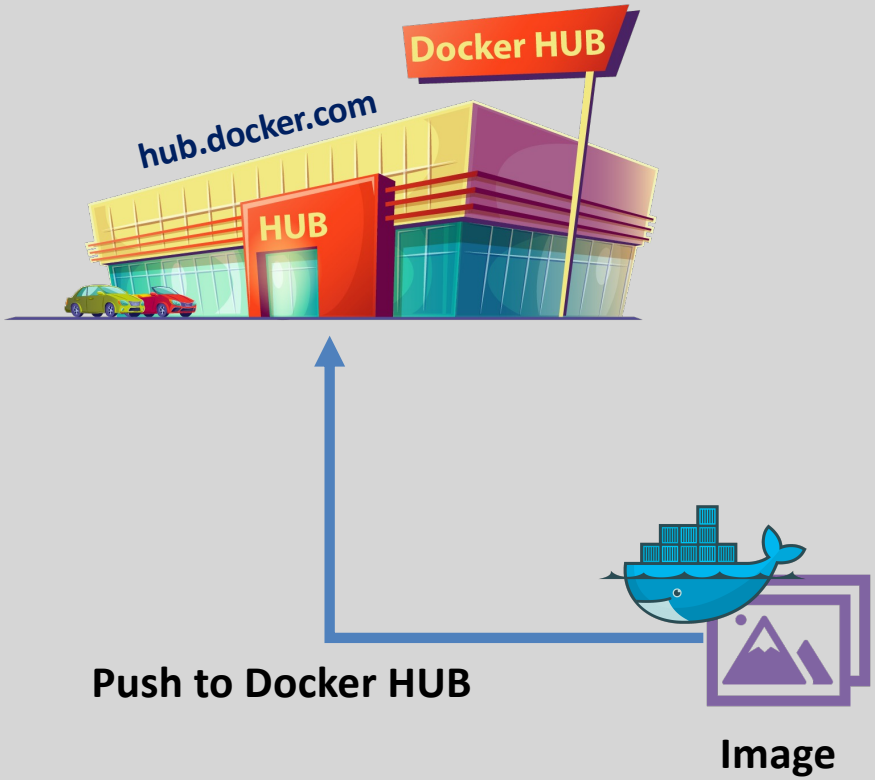
Hands-on Labs (HoLs)

Docker Registry – Public Repository



© DolfinED All rights reserved

Demo - Docker Registry



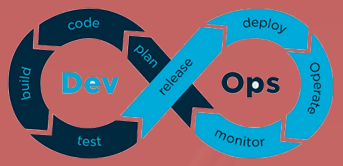
© DolfinED All rights reserved

Hands-on Labs (HoLs)

Docker Registry – Private Repository

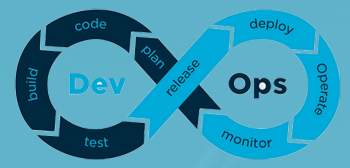


© DolfinED All rights reserved





Docker Networking





Docker Networking Overview

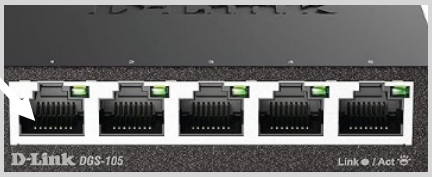
Network Bridges - Overview

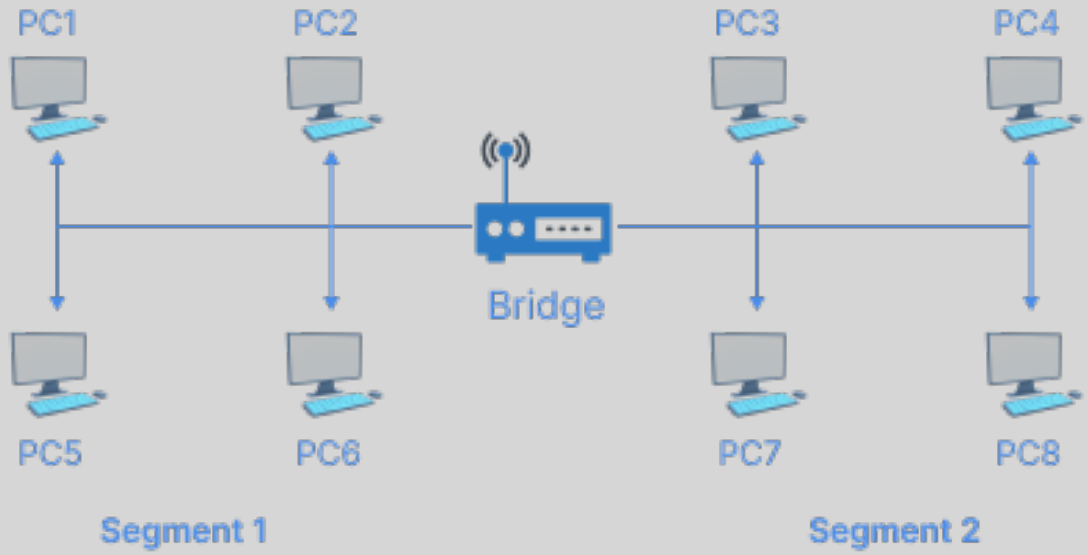
A bridge is a data link-layer (OSI layer 2) physical or virtual device that forwards traffic between two or more network segments.





Ethernet port

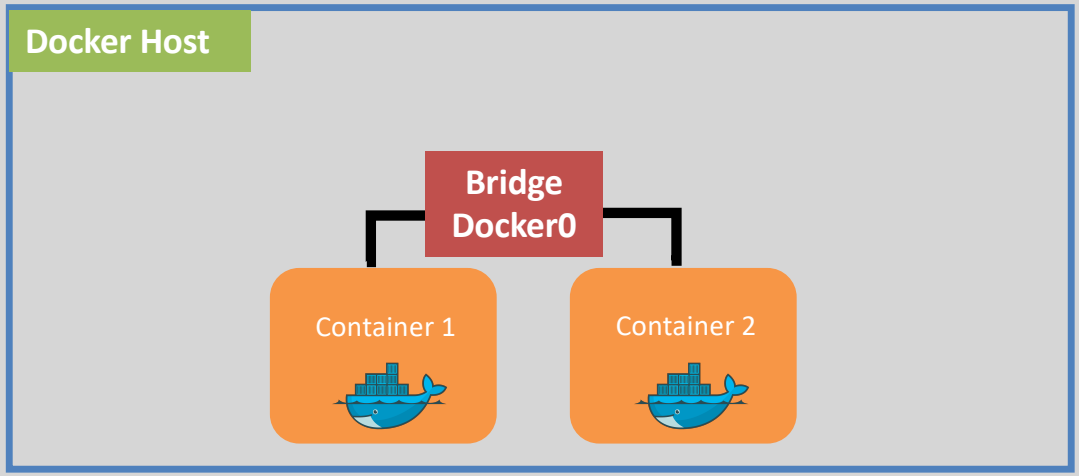




© DolfinED All rights reserved

Docker Bridge Networks

- In Docker, a bridge network uses a software bridge to allow containers to communicate inside the docker host.
- By default, when you start Docker, a bridge (docker0) is created automatically.

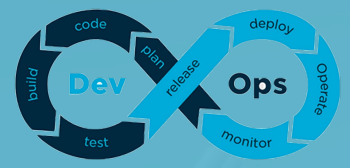


© DolfinED All rights reserved

Docker Network Modes (Drivers)

- Bridge (default) - The default network driver.
- Host - Remove network isolation between the container and the Docker host.
- None - Completely isolate a container from the host and other containers.
- Overlay - Overlay networks connect multiple Docker daemons together.
- Ipvlan - IPvlan networks provide full control over both IPv4 and IPv6 addressing.
- Macvlan - Assign a MAC address to a container.
- Userdefined – Custom network

© DolfinED All rights reserved

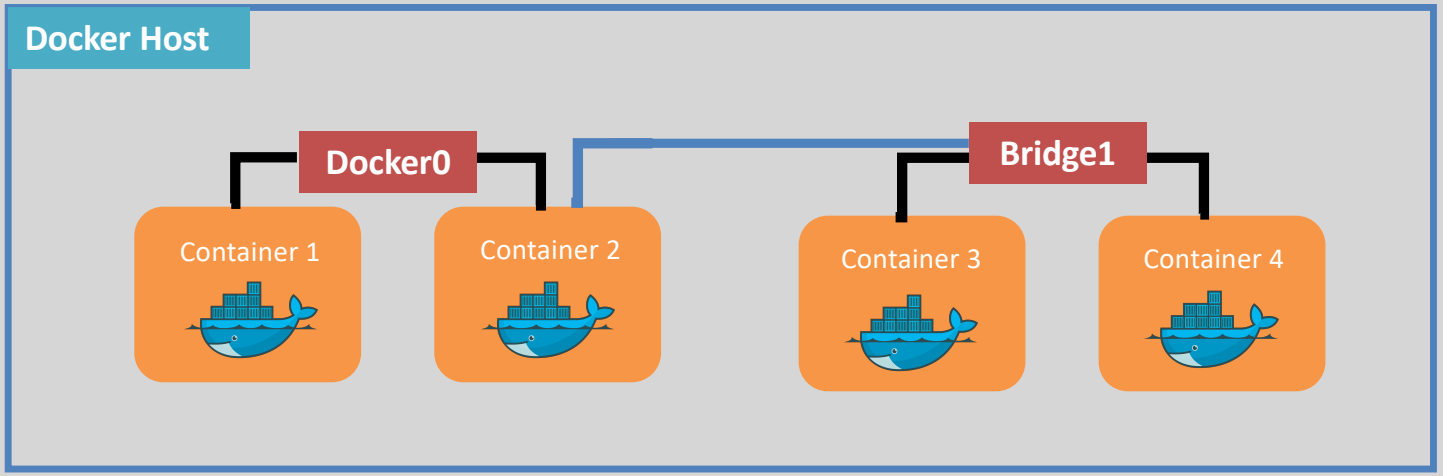




Docker Bridge Mode (Driver)

Bridge Driver/Mode - Containers and Docker Bridge Networks

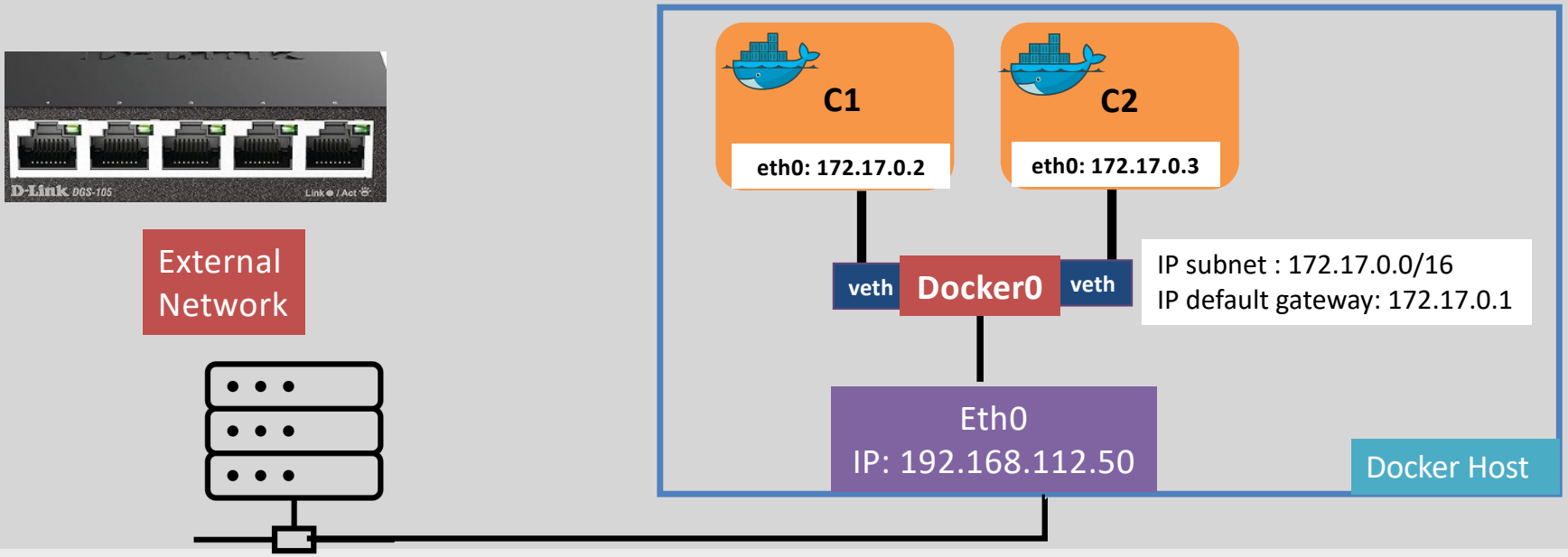
- Containers connected to the same bridge network can communicate
- A bridge network isolates its containers from other containers on other bridges
- Bridge networks apply within the same Docker host only
- A container can connect to multiple bridge networks at the same time



© DolfinED All rights reserved

Docker Default Bridge (docker0)

- The default docker bridge network (docker0) IP subnet is 172.17.0.0/16
- 172.17.0.1 is the IP default gateway between bridge Docker0 and the Docker host
- Unless otherwise specified, all newly created containers attach to Docker0
- Each container created in bridge mode has an eth0 with an assigned IP address
- A new bridge virtual interface (veth) is created with every running container



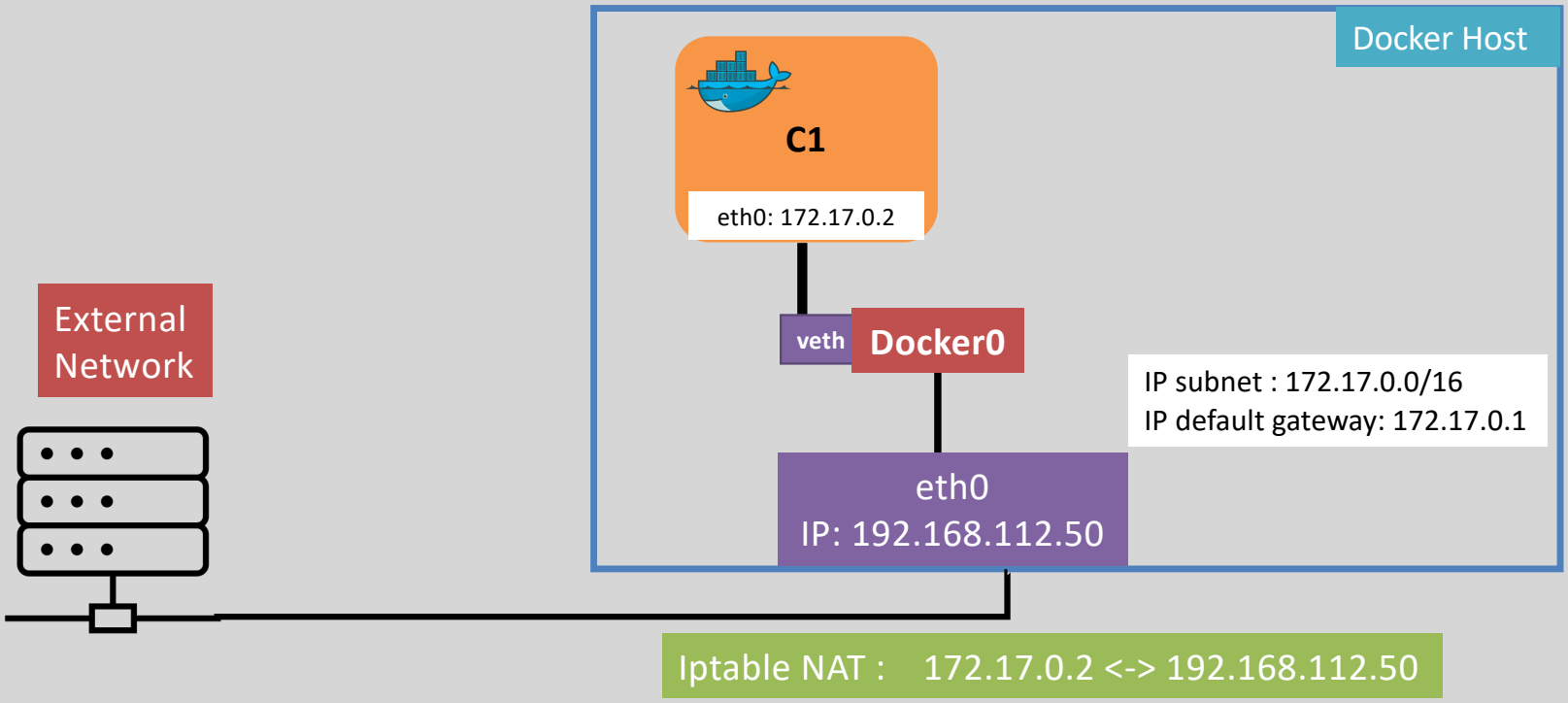
© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Bridge Mode - Containers Outbound Connectivity To External Networks

In bridge network mode, when a container communicates outbound with external networks, the Docker host performs Network Address Translation (NAT)

- The Docker host maintains NAT entries in IPtables.



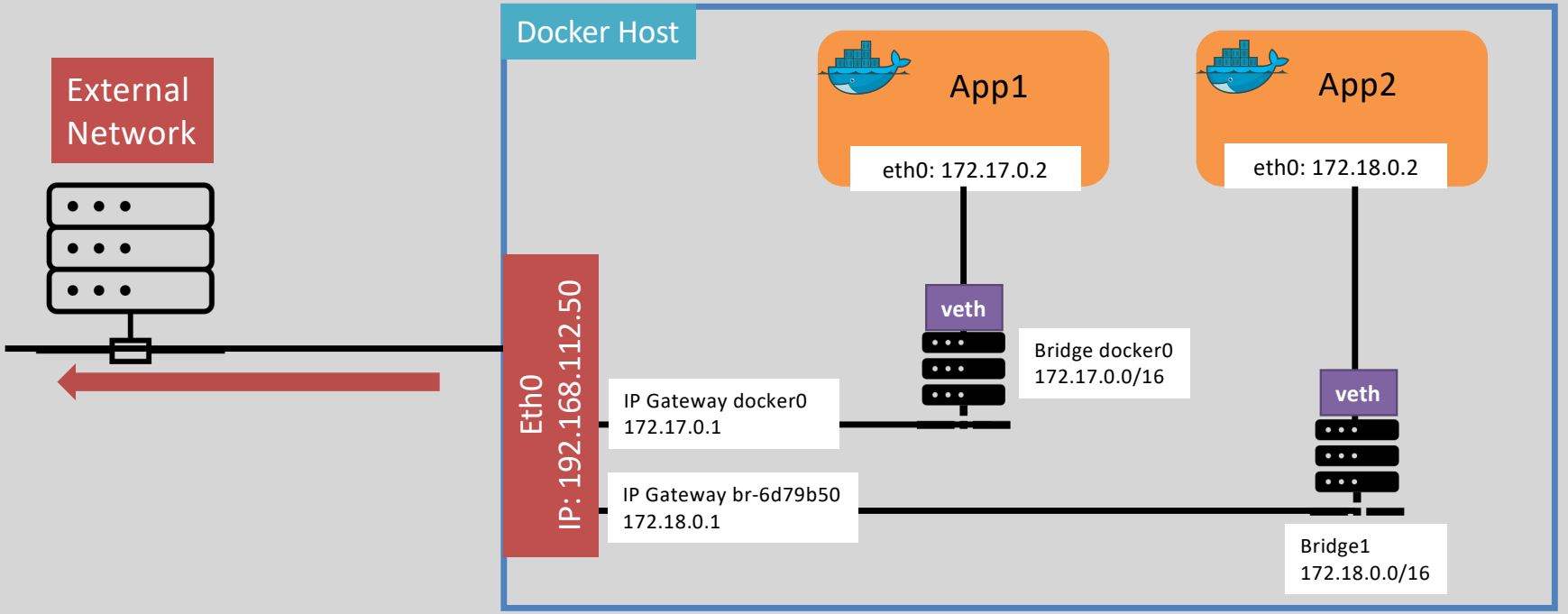
© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Bridge Mode – Multiple Bridge Networks

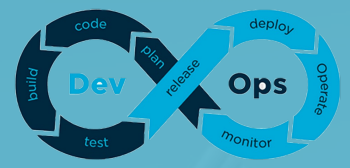
Multiple bridge networks can co-exist inside a docker host

- We can create custom bridge networks as needed (aside from the default bridge)
- A bridge network isolates its containers from other containers on other bridges



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning





Docker Networking Commands

Docker Network Commands

ip a	Display network interfaces and their associated IP addresses on a Linux-based system
docker network ls	List all available networks
docker network create <name>	Create a new network with a specific name
docker network create --subnet 10.0.0.0/16 <network-name>	Create a new network with a custom subnet range
docker **network** inspect <network-name or ID>	Inspect a docker network

DolfinED
Transforming Lives Through Learning

Docker Network Commands (cont.)

docker run --network <network-name> <image-name>	Run a new container and connect it to a Docker network
docker network connect <network-name> <container_name>	Connect a running container to a Docker network
docker network disconnect <network-name> <container_name>	Disconnect a running container from a Docker network (Does not work with none and host network modes)

Hands-on Labs (HoLs)

Docker Networking Modes – Bridge Mode



© DolfinED All rights reserved

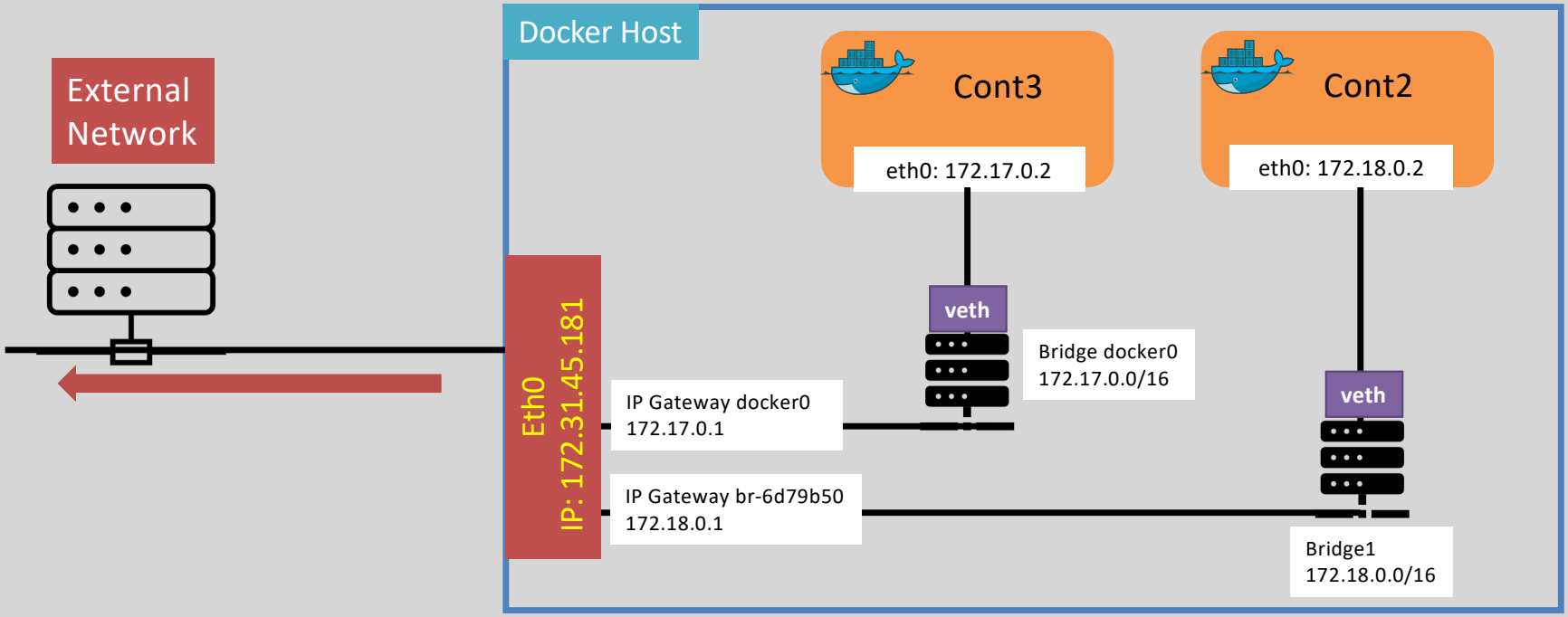
Hands-on Labs (HoLs)

Docker Networking Modes – Custom Networks

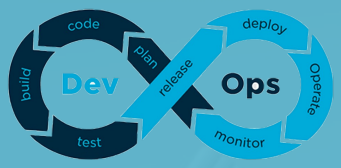


© DolfinED All rights reserved

Docker Bridge Mode – Custom Networks



© DolfinED All rights reserved





Docker Bridge Mode – Container Port

Port Mapping - Recap

Static Port binding

```
docker run –p 3000:80 nginx
```

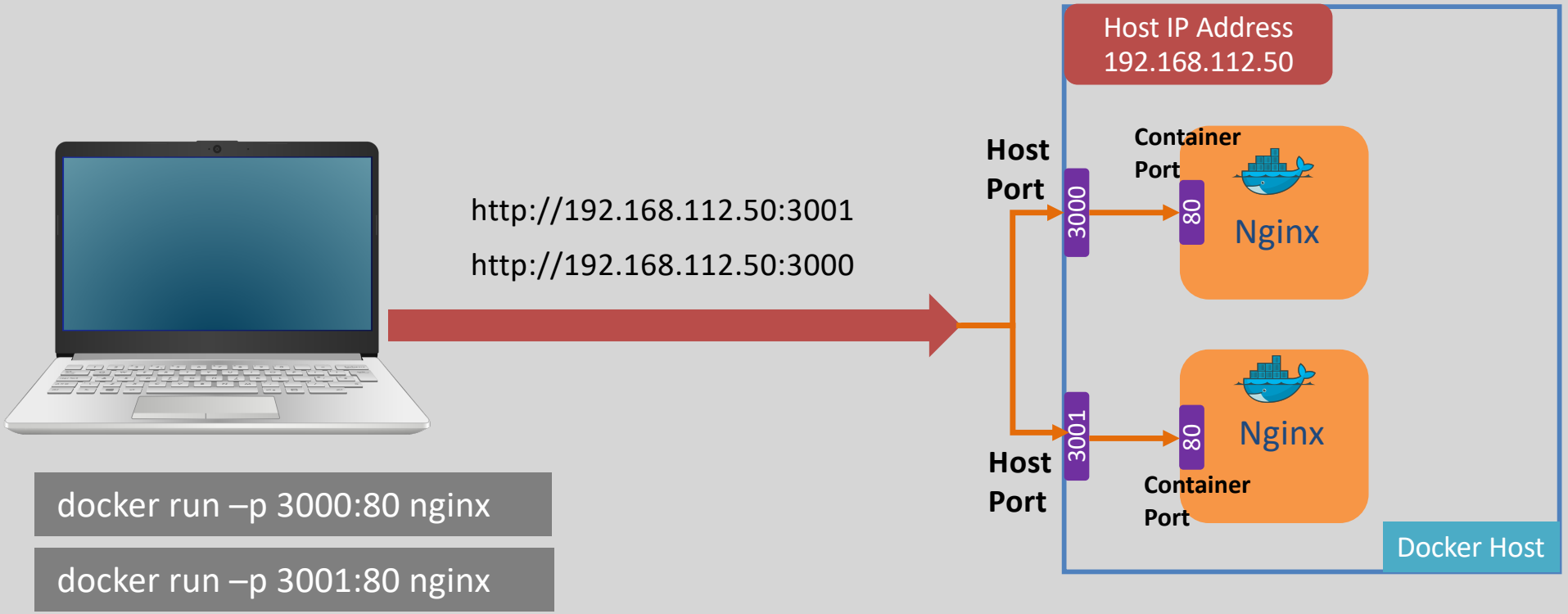
--publish or –p Map or publish a container's port (80) to the specified host port (3000)

```
docker run -d -p 8080:80  -p 8443:443 nginx
```

Map or publish container port 80 to host port 8080. Also, map container port 443 to host port 8443

© DolfinED All rights reserved

Static Port Mapping – Multiple Containers Using the Same Port



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Dynamic Port Mapping - docker run Command Options

Dynamic Port binding

```
docker run –P nginx
```

--publish-all or, -P Map or publish all exposed container ports to random host ports from the range 32768 to 61000

```
docker port <container_id>
```

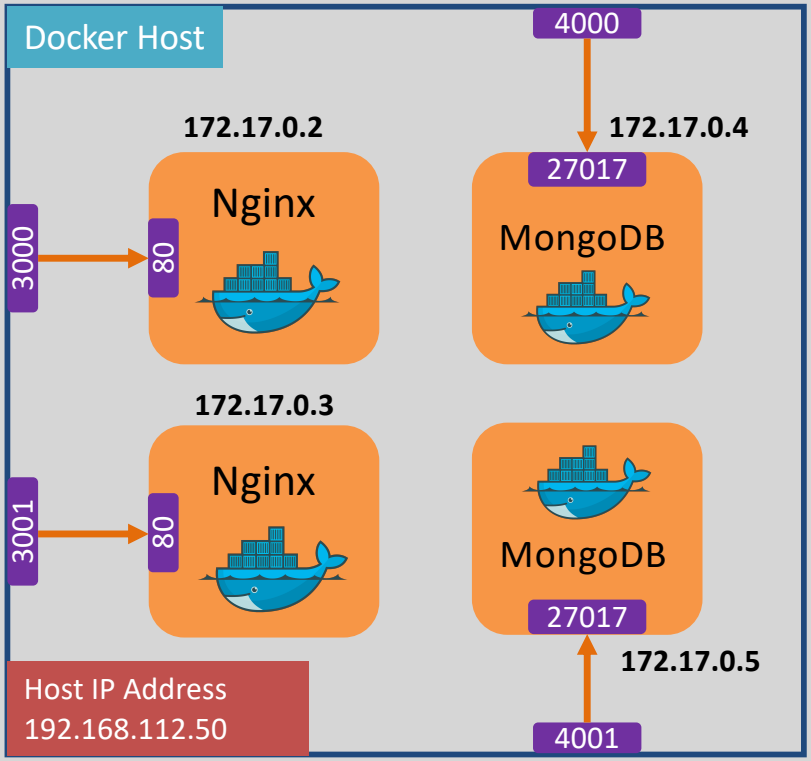
Check the container port mappings

© DolfinED All rights reserved

Container Port – IP Tables

Docker Engine does Network and Port Address Translation (NAT and PAT) and maintains the information in Iptables.

Host Port	Container Port	Container IP
3000	80	172.17.0.2
3001	80	172.17.0.3
4000	27017	172.17.0.4
4001	27017	172.17.0.5



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

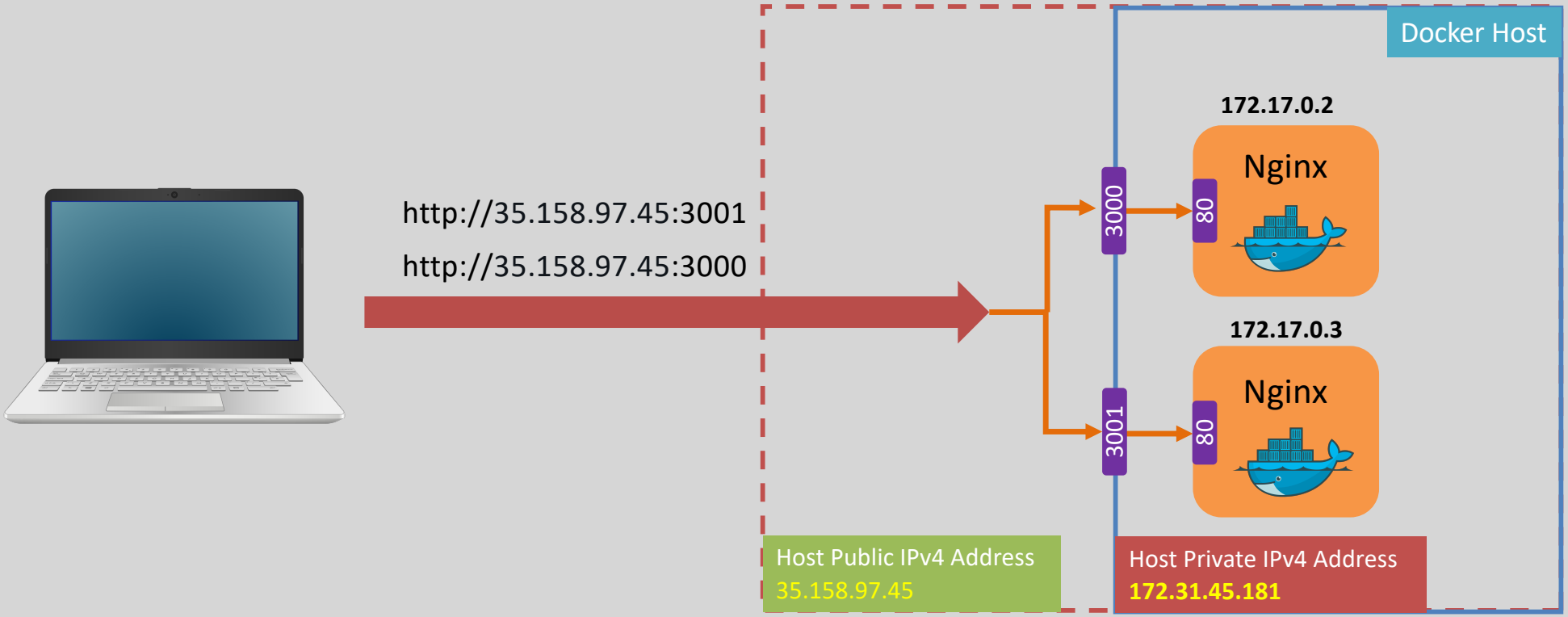
Hands-on Labs (HoLs)

Docker Bridge Network – Container Port Mapping



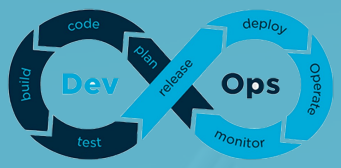
© DolfinED All rights reserved

Container Port Mapping



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning



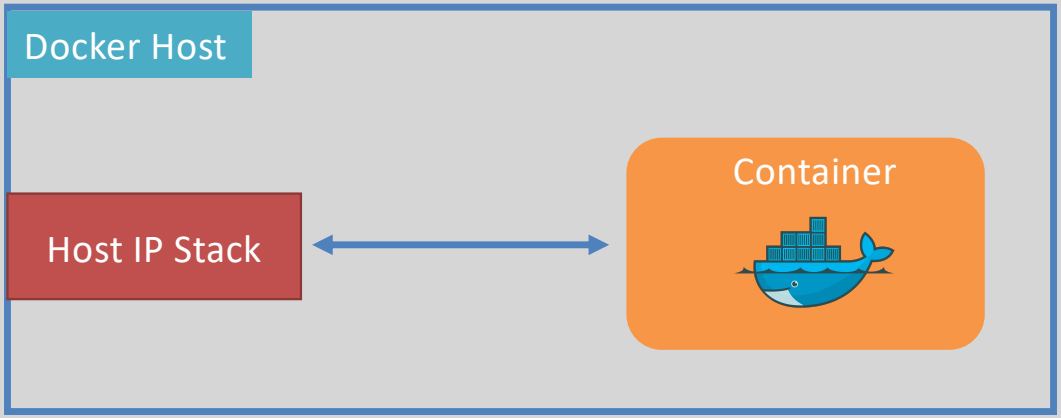


Docker Networking Modes – Host and None

Container Networking – Host Network Mode

The container shares the host's networking namespace

- The container binds directly to the Docker host's
- The container has no eth0 interface or IP address
- No need for Dynamic NAT



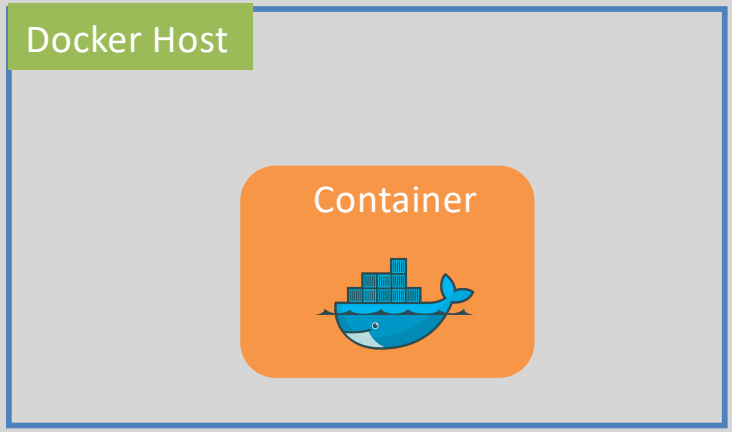
© DolfinED All rights reserved

Container Networking – None Network Mode

In this mode, the container is completely isolated

Use cases:

- For containers that don't need a network (batch or file processing)
- If you want to set up your own network stack



© DolfinED All rights reserved

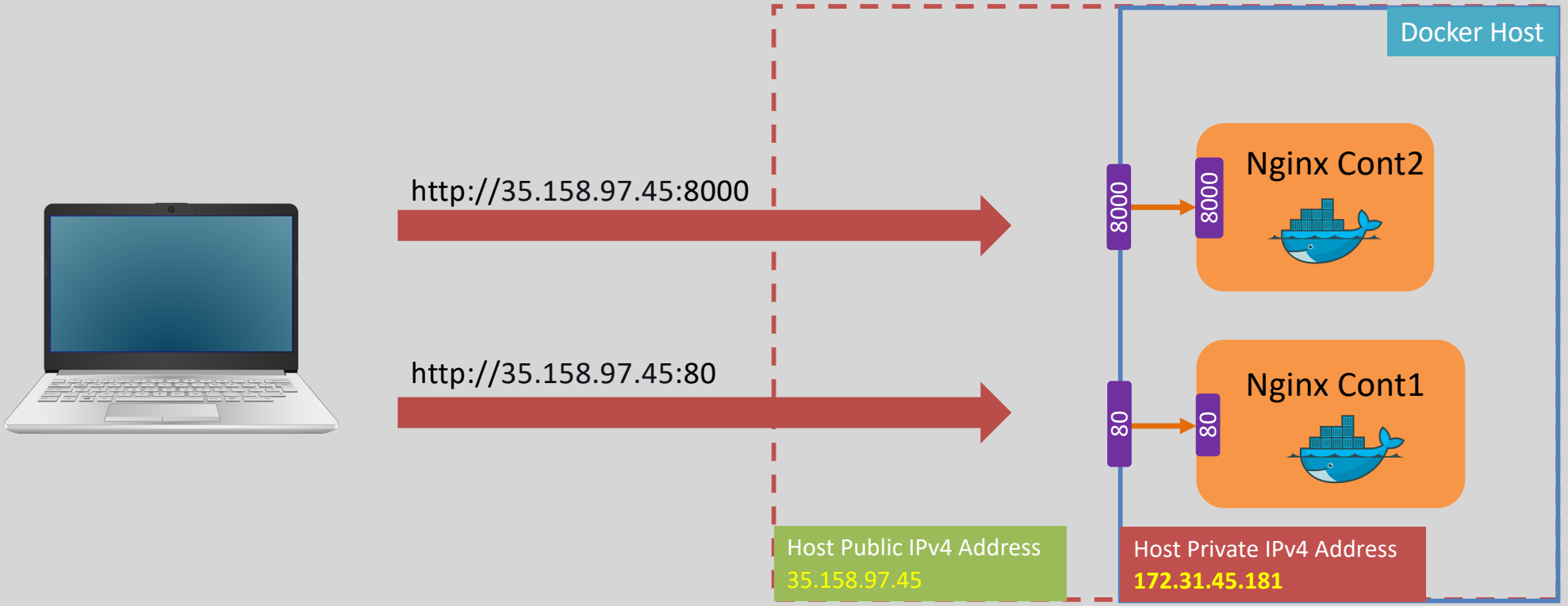
Hands-on Labs (HoLs)

Docker Networking Modes: Host and None



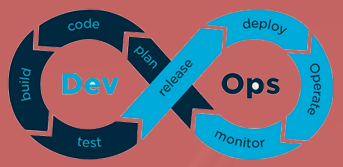
© DolfinED All rights reserved

Container Networking – Host Mode



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

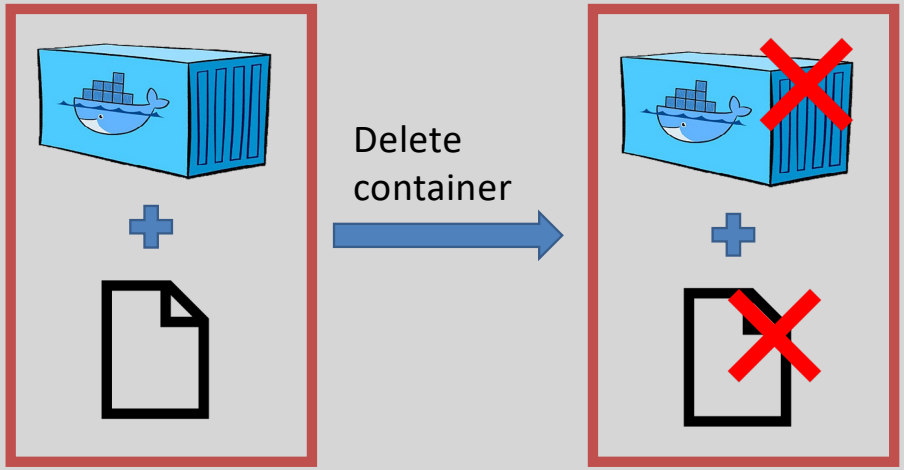




Persisting Data In Docker

The Need For Persisting Data in Docker Containers

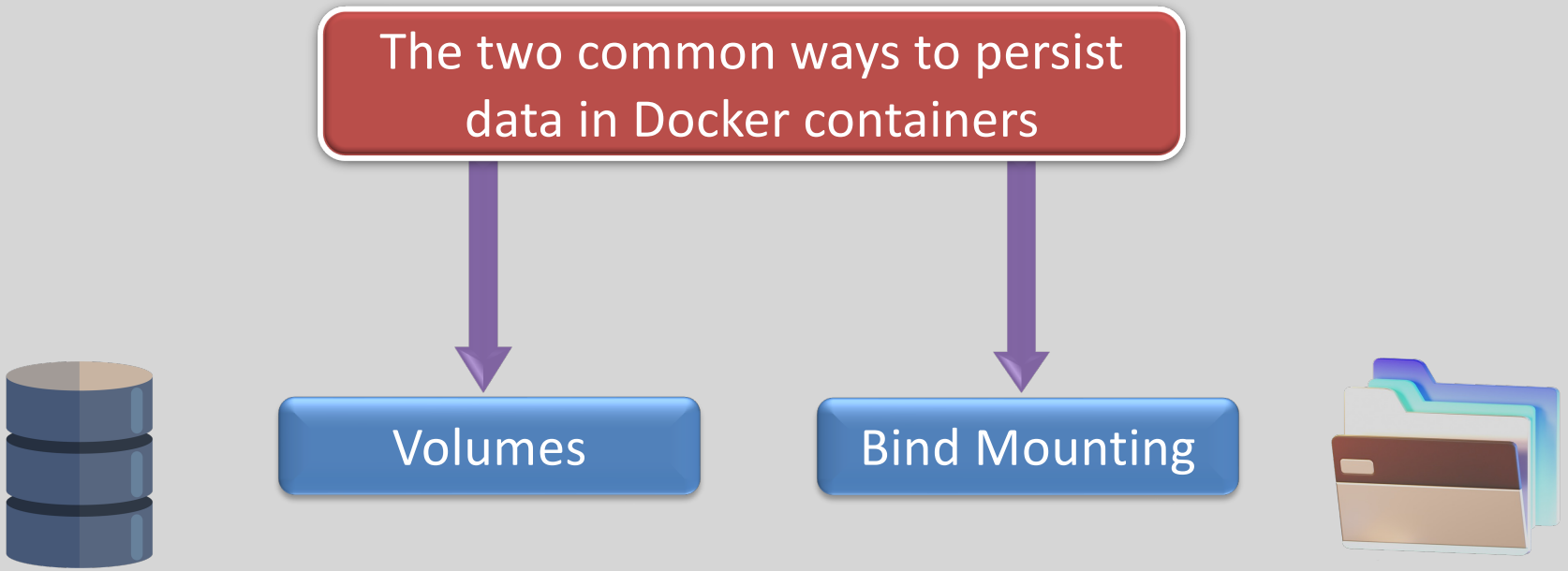
Data in a container is lost if the container is deleted



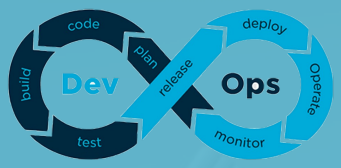
© DolfinED All rights reserved

Persisting Data in Docker - Benefits

- Persist data (e.g., database files, logs)
- Share data between containers
- Separate app code from user data
- Avoid losing data when a container stops or is deleted



© DolfinED All rights reserved



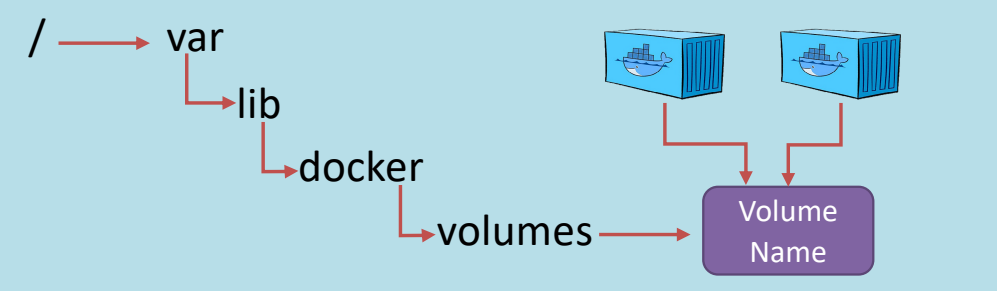


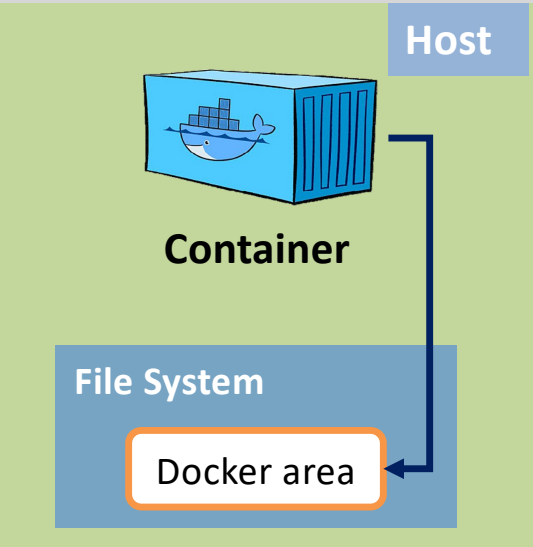
Docker volumes

Docker Volumes

Volumes are stored in the **host filesystem** in the Docker internal storage area which is managed by Docker

- Each volume gets its own volume path as follows:
  /var/lib/docker/volumes**/<volume-name>/**_data/



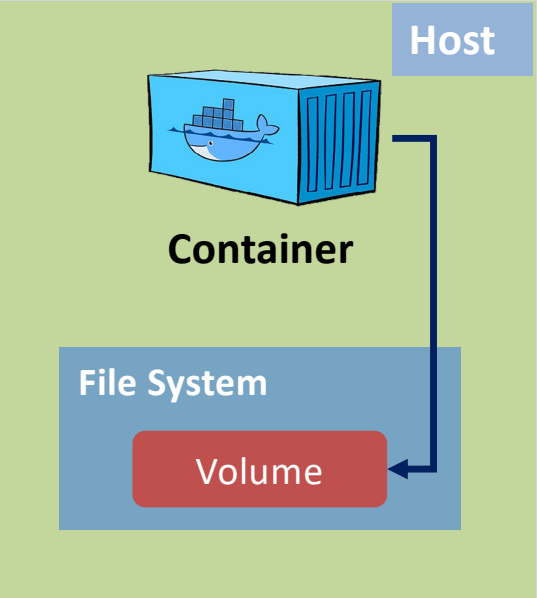


© DolfinED All rights reserved

Docker Volumes (cont.)

A docker volume:

- Is **isolated** from container layers
- **Persists independently** of containers
- Is **mounted into containers** at runtime
- It allows data to survive container restarts, deletion, and image rebuilds



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

docker volume Commands

docker volume create <volume-name>	Create a new “named” docker volume
docker volume ls	List existing docker volumes
docker volume inspect <volume-name>	Show detailed info about a volume
docker volume rm <volume-name>	Delete a docker volume

© DolfinED All rights reserved

docker run Command Options & Docker volumes

-v [volume name]:[container directory]

When used with docker run, this will create and attach a volume to a container at a specific container directory or attach the volume if created beforehand

Example: docker run -v mydata:/app/data ubuntu

Create an ubuntu container and a volume mydata
Mount the volume at the /app/data folder in the container

© DolfinED All rights reserved

docker run Command Options & Docker volumes (cont.)

-v [volume name]:[container directory]:ro

When used with docker run, this will create and attach a volume to a container at a specific container directory.
The container has read only access to the volume (no write access)

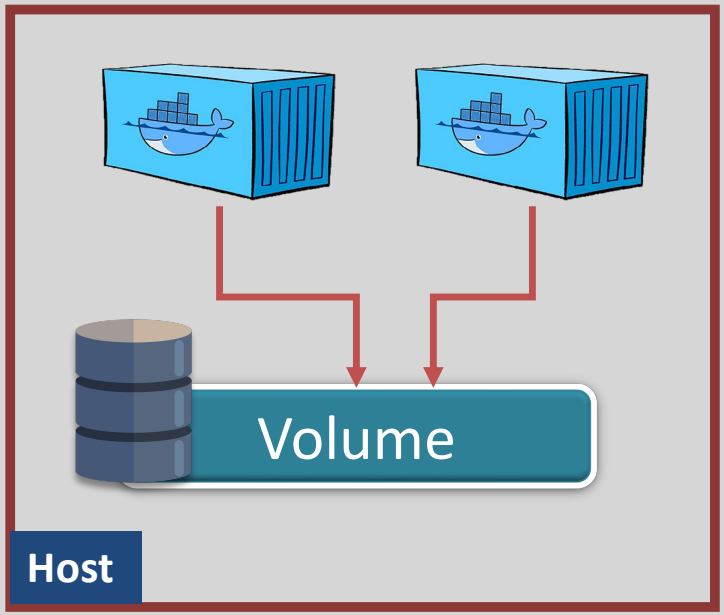
Example: docker run -v mydata:/app/data:ro ubuntu

Create an ubuntu container and a volume mydata.
Mount the volume at the /app/data folder in the container.
The container has read only access to the mydata volume

© DolfinED All rights reserved

Sharing Docker Volumes

Multiple containers can share/use the same volume



© DolfinED All rights reserved

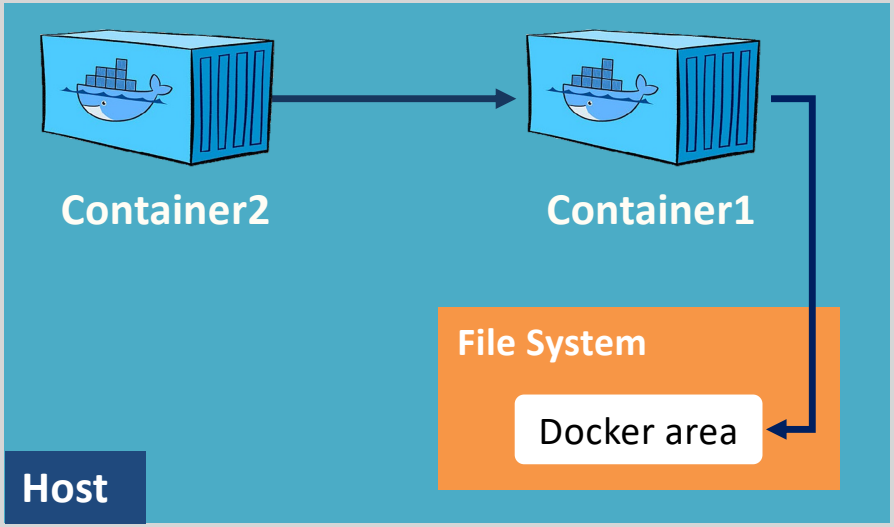
Sharing Data Between Multiple Docker Containers

The --volumes-from flag allows you to **inherit all the volume mounts** from another container.

- It is not as common today

--volumes-from container1

--volumes-from container1:ro



docker run -it **--volumes-from** cont1 ubuntu

Create an ubuntu container and allow it to inherit all volume mounts from cont1

© DolfinED All rights reserved

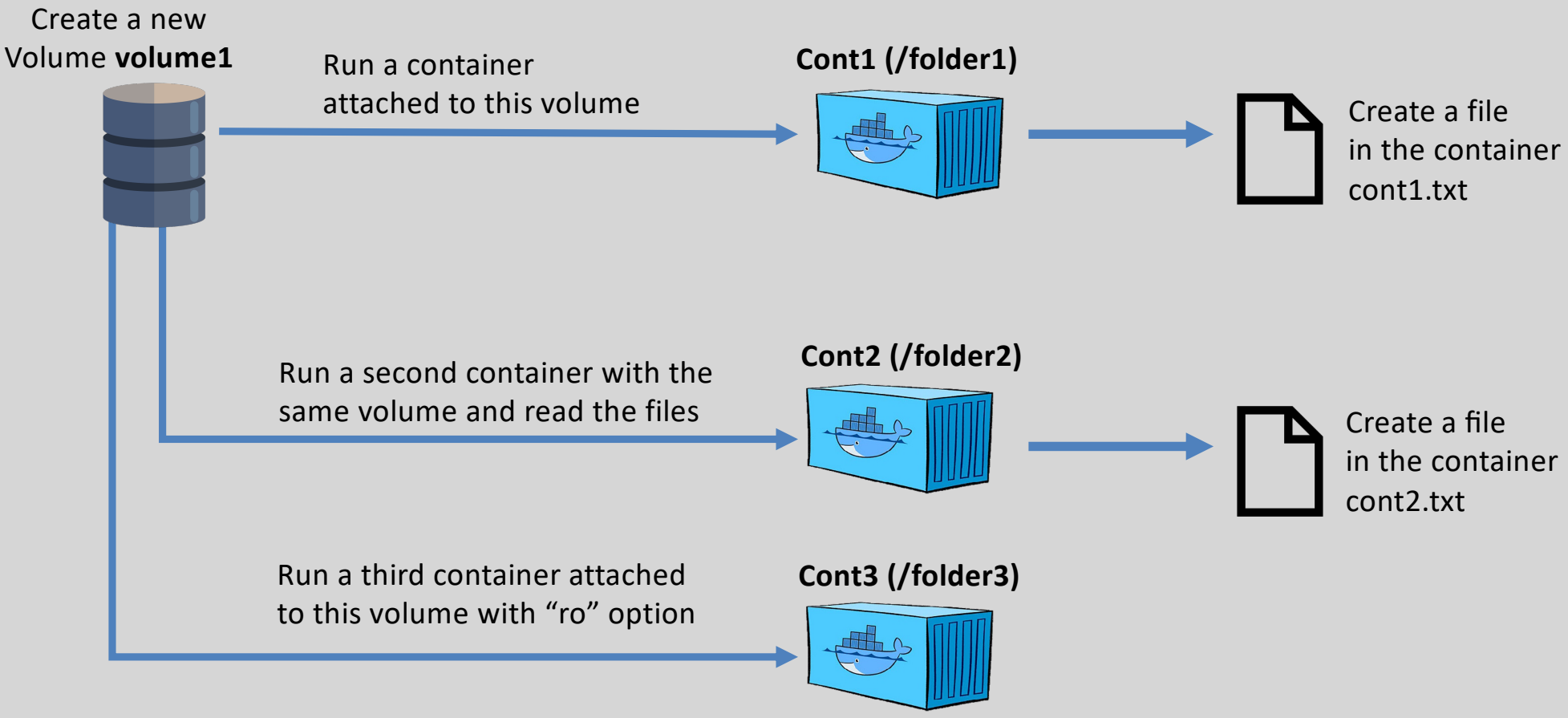
Hands-on Labs (HoLs)

Named Docker Volumes

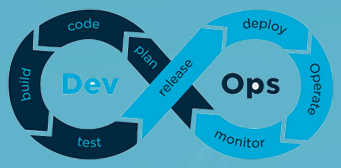


© DolfinED All rights reserved

Docker Volumes



© DolfinED All rights reserved





Creating a Docker volume using a Dockerfile

Docker File Structure – VOLUME

It instructs Docker to treat a specific directory inside the container **as a volume** and store this directory's data **outside of the container's writable layer**

Syntax : VOLUME [path in the container]

```
FROM ubuntu
.....
VOLUME /data
```

- Create a container from ubuntu latest
- Create an anonymous volume and map it to the specified container path
- We can't specify the volume name in the Dockerfile)
- Mount the /data directory in the container to that volume

© DolfinED All rights reserved

Docker File Structure – VOLUME (cont.)

The volume created will be anonymous since we cannot specify a name for it in the Dockerfile

- You can choose to mount it to a named volume that you previously created

```
FROM ubuntu
.....
VOLUME /data
```

```
docker volume create volume1
docker run -v volume1:/data <image-name>
```

Create an ubuntu container and an anonymous (we can't specify the name in the Dockerfile). Mount the /data directory in the container to that volume

© DolfinED All rights reserved

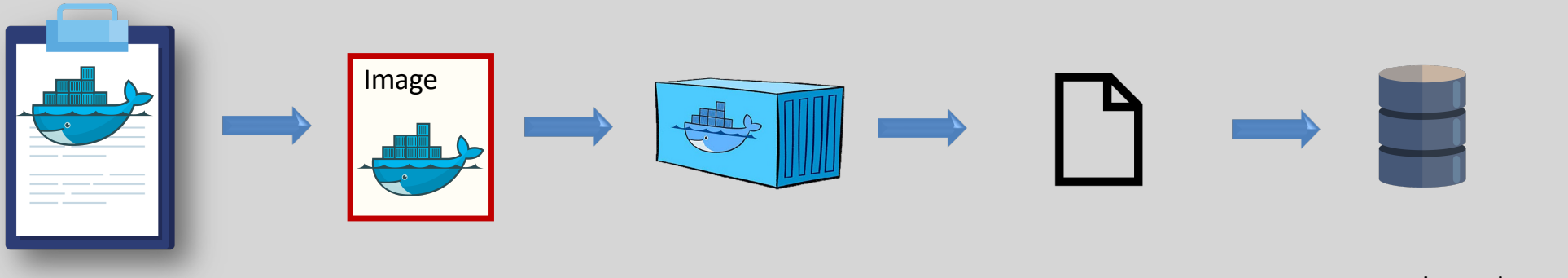
Hands-on Labs (HoLs)

Creating a Docker Volume Using a Dockerfile



© DolfinED All rights reserved

Creating Docker Volumes in Dockerfiles



Create a Docker file with volume

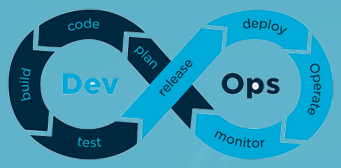
Create an image from docker file

Create a container from the image

Add a file in the volume

access the volume from the host and see the changes

© DolfinED All rights reserved



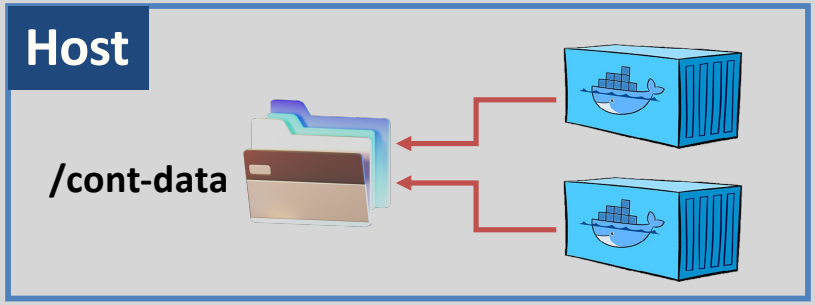


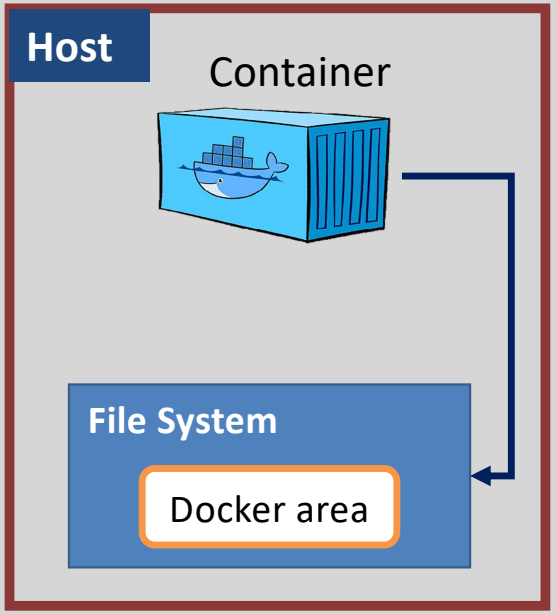
Docker Bind Mounts

Persisting Data Using Bind Mounting

Bind mounting is simply linking a host path or file to a container path or file

- Bind mounts may be stored anywhere in the host filesystem
- It is used with docker run commands (not in a Dockefile)
- Perfect for development and testing (container can directly access host files)





© DolfinED All rights reserved

docker run Command & Bind Mounting Option

```
docker run -v /home/user/project:/app myimage
```

/app in the container is mapped directly to the host path /home/user/project

```
docker run –v  /home/user/nginx.conf:/etc/nginx/nginx.conf  nginx
```

Injecting a **host-specific config** into the container without rebuilding an image.

```
docker run -v $(pwd)/html: /usr/share/nginx/html nginx
```

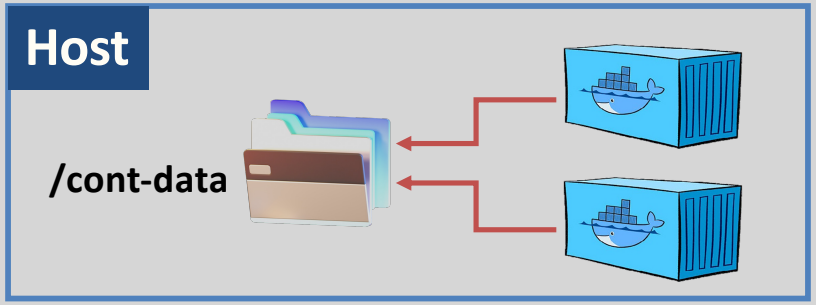
Mount the html folder under the current directory to the container's /usr/share/nginx/html

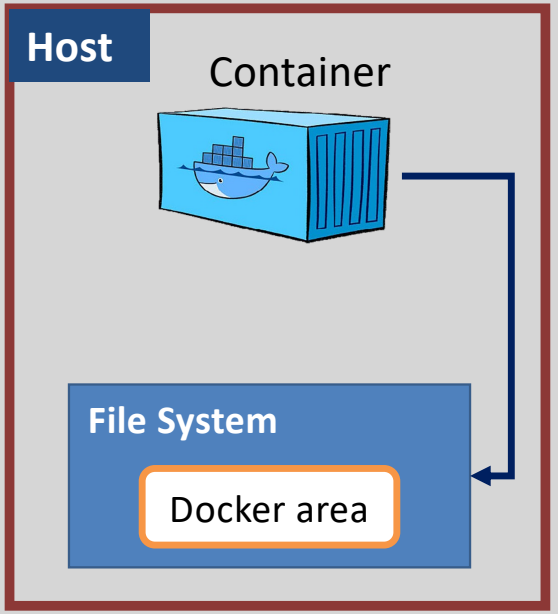
© DolfinED All rights reserved

Docker Volumes or Bind Mounts

Volumes are better for production because they're :

- Safer (Docker manages them)
- Less error-prone (no accidental host overwrite)
- Portable (no hard-coded paths)
- Easier to back up
- Better integrated with Docker tooling





© DolfinED All rights reserved

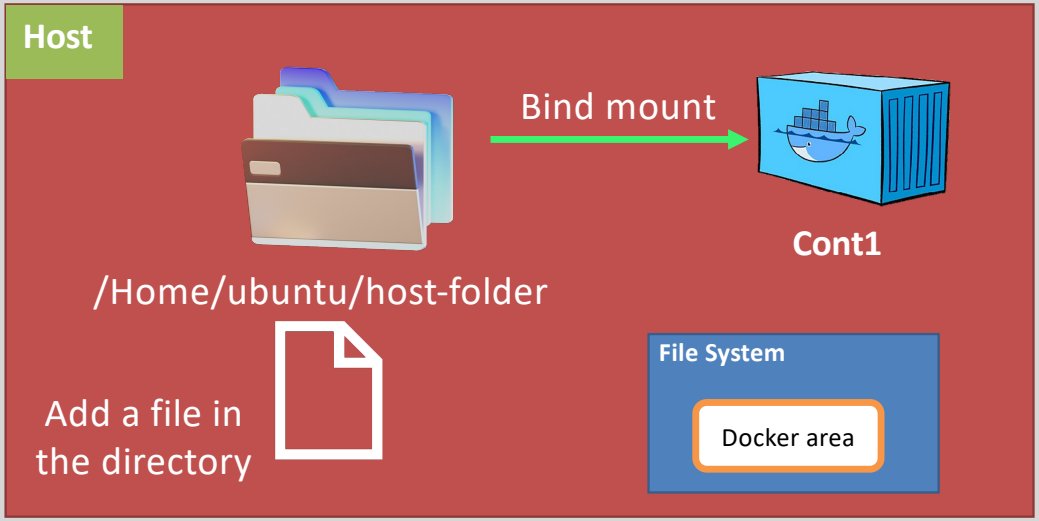
Hands-on Labs (HoLs)

Bind Mounts



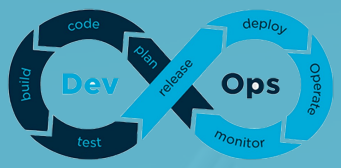
© DolfinED All rights reserved

Docker Bind Mounts



-v [host directory]:[container directory]
-v [host directory]:[container directory]:ro

© DolfinED All rights reserved





docker run –mount Option

docker run Command --mount Option & Docker volumes (cont.)

```
docker run --mount type=<type>,source=<source>,target=<target> <image>
```

This is the new way, and an alternative to –v (--volume). It is verbose and more readable
Type : volume, bind or tmpfs
Source: Name of the volume or host path to use
Target: Path inside the container where the mount will appear

Example: `docker run --mount type=volume,source=mydata,target=/app/data nginx`

Create an nginx container and a volume mydata
Mount the volume at the /app/data folder in the container

© DolfinED All rights reserved

docker run Command --mount option & Bind Mounting Option

```
docker run --mount type=<type>,source=<source>,target=<target> <image>
```

```
docker run -d --mount type=bind,source=/home/ubuntu,target=/app nginx
```

Create a bind mount between the host's /home/ubuntu directory to the container's /app directory

```
docker run --mount type=bind,source="$(pwd)"/html,target=/usr/share/nginx/html nginx
```

Create a bind mount between the host's html folder under the current directory to the container's /usr/share/nginx/html folder

- Any changes made to files in your host in the current directory's html folder (e.g. with VS Code, Nano, or Vim) are immediately visible inside the container.

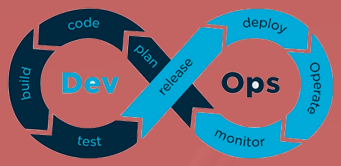
© DolfinED All rights reserved

Hands-on Labs (HoLs)

Named Docker Volumes – Using --mount



© DolfinED All rights reserved



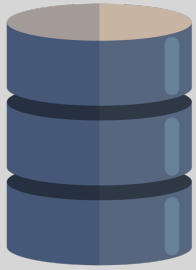


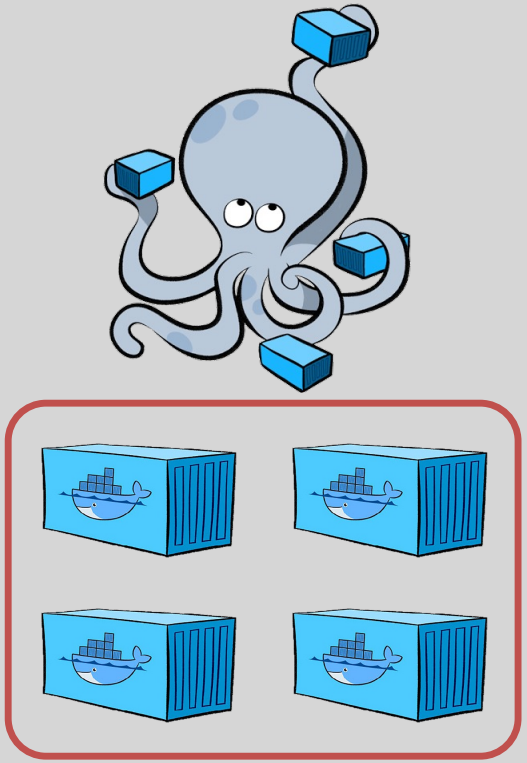
Docker Compose

Introduction To Docker Compose

Docker compose can simultaneously create, start, stop, or delete multiple containers on the same docker host







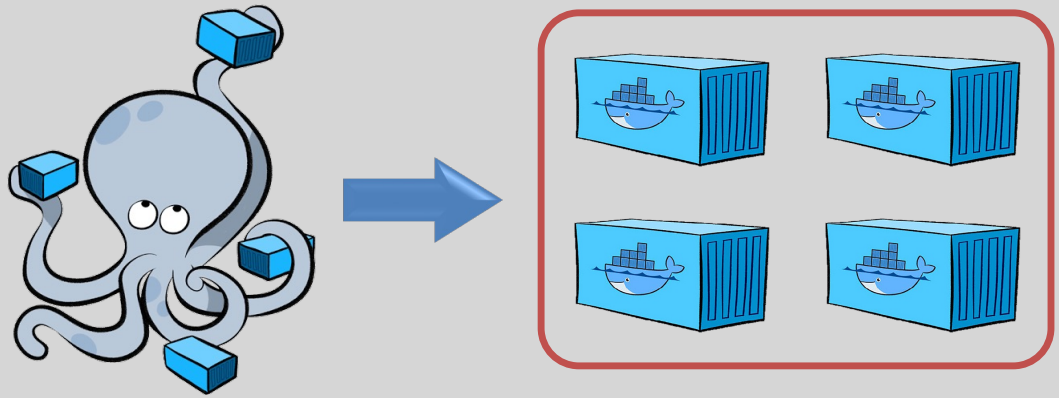
Multi-Container Application

© DolfinED All rights reserved

What Is Docker Compose

Docker Compose is a tool that lets you define and manage multi-container Docker applications

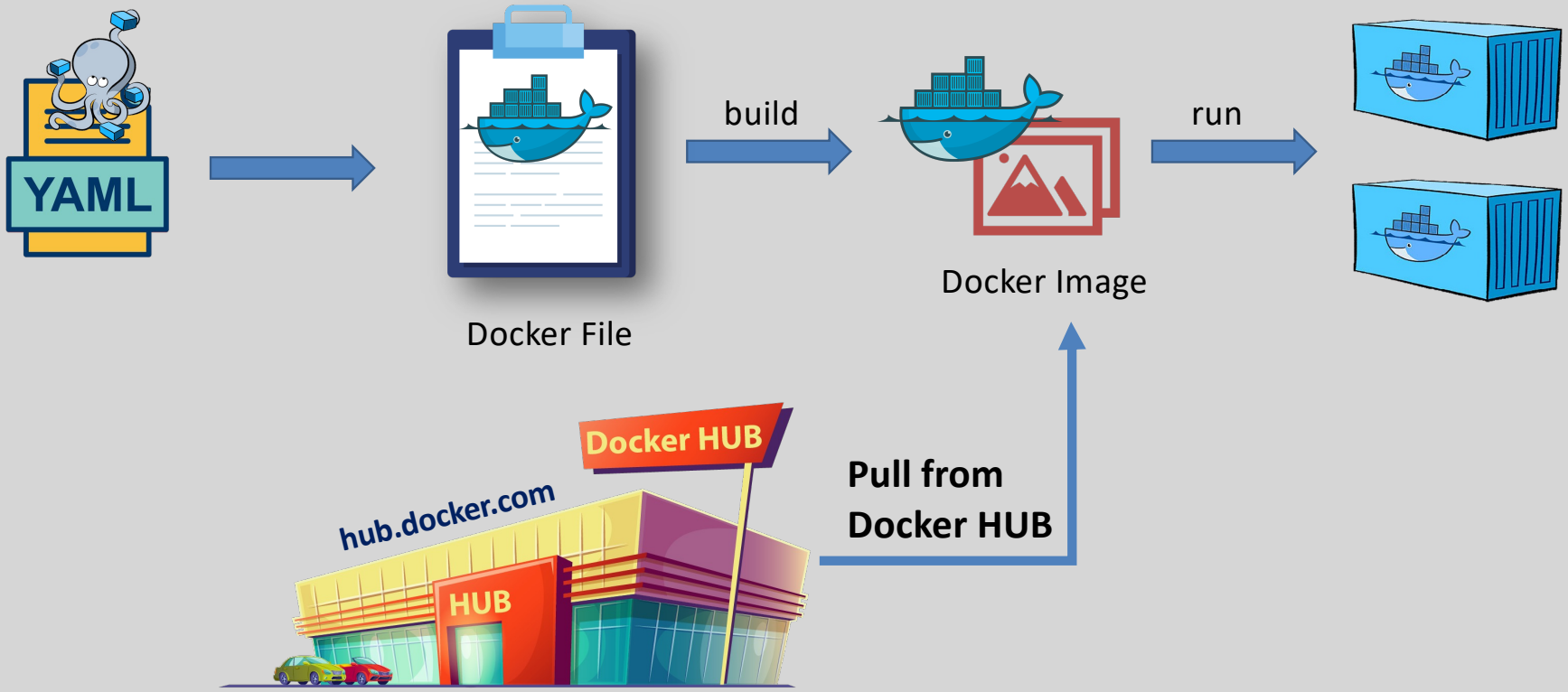
- It allows you to define the entire environment (services, networks, volumes, dependencies, etc.) in a simple YAML file (docker-compose.yml)
- Using a single docker command, you can execute the YAML file to run a project of multiple containers (on a single docker host)
- Docker Compose can build the component images using their Dockerfiles or pull them from a registry



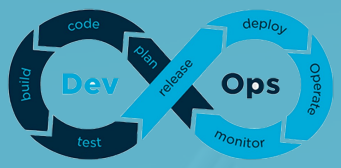
Multi Container Applications

© DolfinED All rights reserved

Docker Compose in Action



© DolfinED All rights reserved





Docker Compose File Structure

Docker Compose File Structure - Services

This section defines all the different containers we need to create

- In the example, we have three services or containers: database, frontend, and backend
- These names have local significance to Docker compose configuration only

docker-compose.yml

```
services:
   database:


   frontend:


   backend:
```

© DolfinED All rights reserved

Docker Compose File Structure – Service Image

For the database service (container), we need to create it from a mysql image (from DockerHub)

- This is why we added the image: mysql statement

docker-compose.yml

```
services:
  database:
    image: mysql

  frontend:

  backend:
```

© DolfinED All rights reserved

Docker Compose File Structure – Build

For the frontend service (container), we need docker compose to build the image from a file named Dockerfile available in the same directory as docker-compose.yml file

- This is why we added the build: .

docker-compose.yml

```
services:
  database:
    image: mysql


  frontend:
    build: .


  backend:
```

© DolfinED All rights reserved

Docker Compose File Structure – Build (cont.)

For the backend service (container), we need docker compose to build the image from a file named fileconfig available in the /app directory

- This is why we added the build:
- The context: /usr/project/app defines the full path where the docker file is located
- The dockerfile: fileconfig defines the file's name to use

docker-compose.yml

```
services:
   database:
     image: mysql


   frontend:
     build: .


    backend:
      build:
        context: /usr/project/app
        dockerfile: fileconfig
```

© DolfinED All rights reserved

Docker Compose File Structure – container_name

We can use the container_name to define a name for the services (container)

Docker-compose.yml

```
services:
   database:
     image: mysql
     container_name: dolfined


   frontend:
     build: .


   backend:
     build:
       context: /usr/project/app
       dockerfile: fileconfig
```

© DolfinED All rights reserved

Docker Compose File Structure – ports

ports: defines the host-port:container-port mapping

Docker-compose.yml

```
services:
   database:
     image: mysql
     container_name: dolfined



   frontend:
     build: .
     ports:
       - “3000:5000”


   backend:
     build:
      context: /usr/project/app
      dockerfile: fileconfig
```

© DolfinED All rights reserved

Docker Compose File Structure – volumes (bind mount)

The volumes: under a service is used to create a bind mount (HostPath) mapping between a hostPath and a containerPath

Docker-compose.yml

```
services:
   database:
     image: mysql
     container_name: dolfined
     volumes:
       - hostPath:containerPath

   frontend:
     build: .
     ports:
       - "3000:5000"


   backend:
     build:
      context: /usr/project/app
      dockerfile: fileconfig
```

© DolfinED All rights reserved

Docker Compose File Structure – volumes (named Volume)

The volumes: can also be used to mount a named volume (volumeName) to a containerPath

- This requires the volume to be created separately as shown using the volumes: section

Docker-compose.yml

```
services:
   database:
     image: mysql
     container_name: dolfined
     volumes:
       - volumeName:containerPath

   frontend:
     build: .
     ports:
       - "3000:5000"
   backend:
     build:
      context: .
       dockerfile: Dockerfile
volumes:
  volumeName:
```

© DolfinED All rights reserved

Docker Compose File Structure - volumes (Anonymous volume)

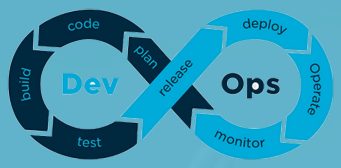
The volumes: can be used to mount an anonymous volume to a service (container)
In this case we do not need the volumes section outside the services

Docker-compose.yml

```
services:
  database:
    image: mysql
    container_name: dolfined
    volumes:
      - :containerPath

  frontend:
    build: .
    ports:
      - "3000:5000"
  backend:
    build:
     context: .
      dockerfile: Dockerfile
```

© DolfinED All rights reserved





Basic Docker Compose Commands

Docker Compose Commands

docker compose build	Builds the images for the services
docker compose run <service-name>	Creates a specific service (container) included in the docker-compose.yml file
docker compose up	Builds the images if they are not located locally and starts the containers.
docker compose –f <file-name> up	Specifies a different compose file, builds the images, and starts the containers
docker compose up -d	Similar to ‘docker compose up’ but runs the containers in detached mode.

© DolfinED All rights reserved

Docker Compose Commands (cont.)

docker compose stop	Stops all containers created by ‘docker compose up’
docker compose start	Starts all containers created by ‘docker compose up’
docker compose exec	Execute a command in a running container
docker compose ls	List running Compose projects

© DolfinED All rights reserved

Docker Compose Commands (cont.)

docker compose down	Stops and removes all containers created by 'docker compose up'
docker compose down --volumes	Stops and removes all containers and volumes (named and anonymous) created by 'docker compose up'
docker compose down --remove-orphans	Stops and removes all current Compose containers and any old (orphan) containers that are no longer referenced by Compose yaml file
docker compose down --volumes --remove-orphans	Does all the above combined

© DolfinED All rights reserved

Docker Compose Network

By default, Docker Compose sets up a bridge network for your app

- Each container for a service joins the default network and is reachable by other compose services (containers) on that network

```
docker compose up -d
```

```
docker network ls
```



```
NETWORK ID     NAME              DRIVER    SCOPE
880f38a30a3e   bridge            bridge    local
b7dd88f96729   host              host      local
d6e109ba1caa   ubuntu_default    bridge    local
```

© DolfinED All rights reserved

Hands-on Labs (HoLs)

Install Docker Compose



© DolfinED All rights reserved

Hands-on Labs (HoLs)

Docker Compose Configuration File



© DolfinED All rights reserved

Hands-on Labs (HoLs)

Docker Compose Configuration File – Multiple Containers



© DolfinED All rights reserved

Hands-on Labs (HoLs)

Docker Compose – Adding Persistent Storage (Volumes and Bind Mounts)



© DolfinED All rights reserved

Hands-on Labs (HoLs)

Docker Compose – Using the Image Build Capability



© DolfinED All rights reserved

Hands-on Labs (HoLs)

Assignment – Creating a Docker Compose Configuration File Based On Given Requirements



© DolfinED All rights reserved

Docker Compose File Example - Requirements

Create a docker compose yaml file that will:

- Create an app service,
- The resulting container name is **cont1**
- Build the image from a docker file located at /app/bin/project/ and the docker file name is docker2
- Map container port **3000** to host port **5551**
- The app service will require a bind mount at hostpath /usr/home/project to a containerPath /data/app
- The app service requires a named volume myvol mapped to /data/logs/ at the container

© DolfinED All rights reserved

Docker Compose File Example – Requirements (cont.)

- Add a database service, **db-service**, using redis database image
- The resulting container name is **redis-db**
- Use the latest redis image
- The db-service will receive communication on host port 7326 and redis port **6379**
- Add a volume db_vol attached to the db-service at the container /data path
- Add a bind mount (host path) /usr/logs mapped to container path /var/log/redis

© DolfinED All rights reserved

Docker Compose File – app Service

Create a docker compose yaml file that will:

- Create an app service,
- The resulting container name is **cont1**
- Build the image from a docker file located at /app/bin/project/ and the docker file name is docker2
- Map container port **3000** to host port **5551**
- The app service will require a bind mount at hostpath /usr/home/project to a containerPath /data/app
- The app service requires a named volume myvol mapped to /data/logs/ at the container

```
services:
  app:
    container_name: cont1
    build:
      context: /app/bin/project/
      dockerfile: docker2
    ports:
      - "5551:3000"
    volumes:
      - /usr/home/project:/data/app
      - myvol:/data/logs/

volumes:
  myvol:
```

© DolfinED All rights reserved

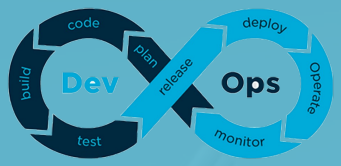
Docker Compose File – db-service Service

- Add a database service, **db-service**, using redis database image
- The resulting container name is **redis-db**
- Use the latest redis image
- The db-service will receive communication on host port 7326 and redis port **6379**
- Add a volume db_vol attached to the db-service at the container /data path
- Add a bind mount (host path) /usr/logs mapped to container path /var/log/redis

```
services:
  app:
    container_name: cont1
    build:
      context: /app/bin/project/
      dockerfile: docker2
    ports:
      - "5551:3000"
    volumes:
      - /usr/home/project:/data/app
      - myvol:/data/logs/

  db-service:
     container_name: redis-db
     image: redis:latest
     ports:
       - "7326:6379"
     volumes:
       - db_vol:/data
       - /usr/logs:/var/log/redis
volumes:
   myvol:
   db_vol:
```

© DolfinED All rights reserved





Docker Compose File Structure (cont.)

- depends_on
- healthcheck

Docker Compose Attributes – depends_on:

- Specifies the startup order of services
- Does not wait for redis to be **ready**—only that the container has started
- Combine with healthcheck for true readiness checks

Docker-compose.yml

```
services:
  web:
    .....
    depends_on:
        - redis
  redis:
     image: redis: alpine
    .......
```

© DolfinED All rights reserved

Docker Compose Attributes – healthcheck:

- Monitors whether a container is “healthy” (application-level check)
- Useful for ensuring a service is truly ready before others depend on it

Docker-compose.yml

```
services:
  web:
    depends_on:
      redis:
        condition: service_healthy
  redis:
    image: redis:alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 3
```

© DolfinED All rights reserved

Docker Compose Attributes – depends_on: & healthcheck:

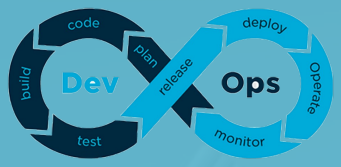
- Avoids race conditions (e.g., web service starting before DB is ready)
- Improves resilience and reliability of service orchestration
- Delays startup of dependent services until health checks pass

No Hyphen next to redis

Docker-compose.yml

```
services:
  web:
    depends_on:
      redis:
        condition: service_healthy
  redis:
    image: redis:alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 3
```

© DolfinED All rights reserved





Docker Compose File Structure (cont.)

- networks

Docker Compose Attributes – networks: default network

By default, Docker Compose creates a network named default, prefixed with the project (or folder) name.

- Any service that does not explicitly define a networks: section will automatically be connected to this default network.

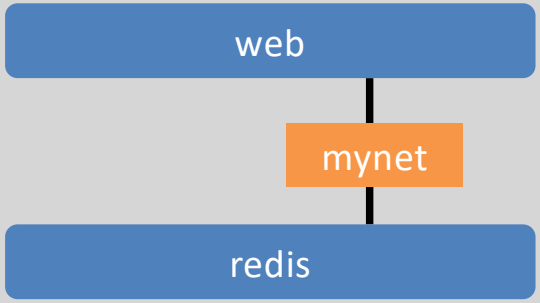
Docker-compose.yml

```
services:
  web:
    image: myapp
    # No networks section = uses 'default'
  visitor-counter:
     build: .
     ports:
       - "9000:5000"
     # No networks section = uses 'default'
```

© DolfinED All rights reserved

Docker Compose Attributes – networks:

- Docker Compose creates a default network; you can define your own
- Enables communication between services using service names



Docker-compose.yml

```
services:
  web:
    networks:
      - mynet
  redis:
    networks:
      - mynet

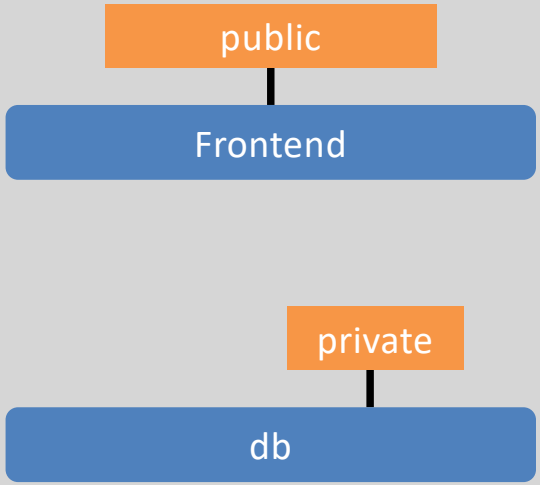
networks:
  mynet:
```

© DolfinED All rights reserved

Docker Compose Attributes – networks: - Isolate Services

Explicit networks allow you to **segment your architecture**.

- Example: a frontend service shouldn't be able to talk to your internal DB.
  - This means frontend and db **won't** be able to talk unless you assign both to a common network.



Docker-compose.yml

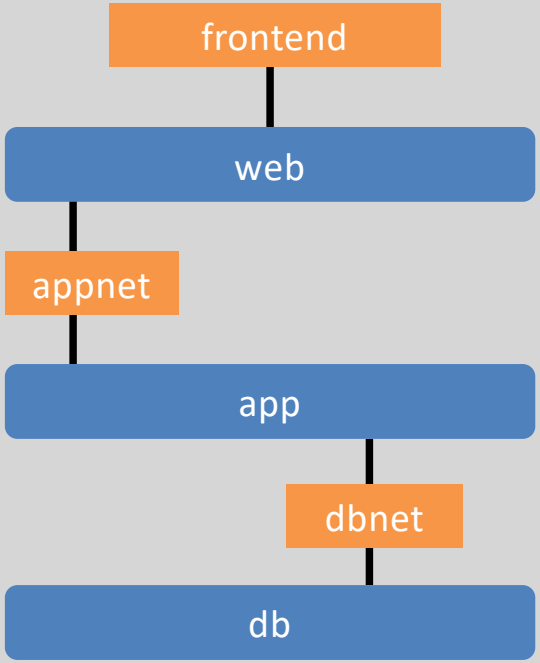
```
services:
  frontend:
    networks:
      - public
  db:
    networks:
      - private

networks:
  public:
  private:
```

© DolfinED All rights reserved

Docker Compose Attributes – networks: - Multiple Network Attachments

Some services might need to connect to **more than one network**



Docker-compose.yml

```
services:
  web:
    .......
    networks:
      - frontend
      - appnet  # only talks to app
  app:
    ......
    networks:
      - appnet
      - dbnet
  db:
    ......
    networks:
      - dbnet  # only app can talk here
networks:
   frontend:
   appnet:
   dbnet:
```

© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Docker Compose Attributes – networks: - Connect to External Containers or Services

You'd use an external network when you want multiple Docker Compose projects or standalone containers to **communicate across project boundaries**.

- Tells Docker not to create a default network; instead use an external network (pre-existing, not created by this docker compose project).
- This network (shared-network must pre-exist)

```
docker network create shared-network
```

Docker-compose.yml

```yaml
services:
  web:
    image: myapp
    # No networks section = uses 'default'
  visitor-counter:
    build: .
    ports:
      - "9000:5000"
    networks:
      - default       # uses 'default'
networks:
    default:
      external:
        name: shared-network
```

© DolfinED All rights reserved

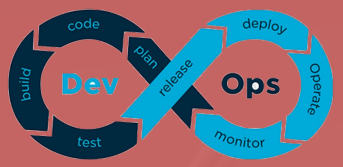
Docker Compose Attributes – networks: - Customize Network Settings

You can customize the network driver, IP address range, and default gateway among other settings for your custom network

Docker-compose.yml

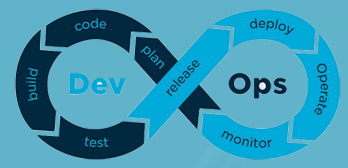
```
services:
  web:
    image: myapp
    networks:
        - mynetwork
networks:
  mynetwork:
    driver: bridge
    name: custom-network
    ipam:
      config:
        - subnet: 172.30.0.0/16
          gateway: 172.30.0.1
```

© DolfinED All rights reserved





Bonus / Advanced Docker Topics





Bind Mounts – HostPath Data & Default Container Behavior

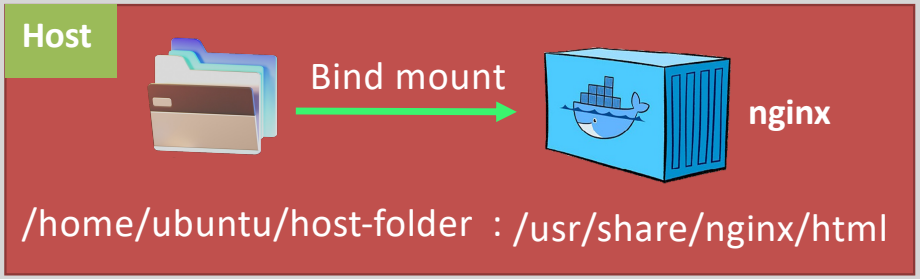
The Issue : Bind Mount Can Hide File in the Container

When you mount a directory (or file) from your host machine into a container, the host path content (empty or not) prevails over that in the container's mount point

What this means:

Any files or directories already present in the container at the mount point become invisible while the mount is active

- These files won't be accessible to the container during runtime



Dockerfile

```
FROM nginx:latest
COPY ./index.html /usr/share/nginx/html/
EXPOSE 80
```

```
docker run –v /home/ubuntu/host-folder:/usr/share/nginx/html nginx
```

© DolfinED All rights reserved

The Issue : Bind Mount Can Hide File in the Container (cont.)

⚠ **Why is this a problem?**

- You may assume certain files (added via COPY in the Dockerfile) are there, but they're not
- Your app may fail to run, missing configs, templates, or dependencies
- Debugging this can be confusing because it works **without** the volume mount

© DolfinED All rights reserved

Bind Mount – Best Practices

- ✅ Avoid bind mounting entire directories unless necessary
- ✅ Mount specific files instead of folders when possible
- ✅ Use bind mounts mainly in development, not in production
- ✅ Always verify that expected files exist after mounting

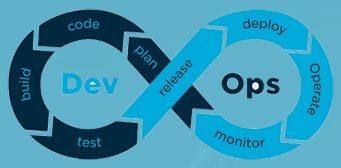
© DolfinED All rights reserved

Hands-on Labs (HoLs)

Bind Mounts – HostPath Data & Default Container Behavior



© DolfinED All rights reserved



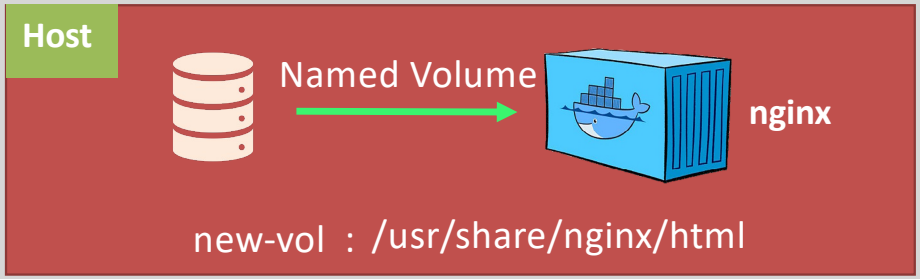


Docker Volumes – Volume Data & Default Container Behavior

The Issue : Named Volume Content Can Overwrite The Content in the Container

When you mount a named Docker volume to a container:

- If the volume is empty, Docker copies the content of the /usr/nginx/share/html to the volume only once. The volume content then takes over
- If the volume has content, The volume content replaces (overwrites) what is in /usr/share/nginx/html
- Always check whether the volume is fresh or already populated



Dockerfile

```
FROM nginx:latest
COPY ./index.html /usr/share/nginx/html/
EXPOSE 80
```

```
docker run –v new-vol:/usr/share/nginx/html nginx
```

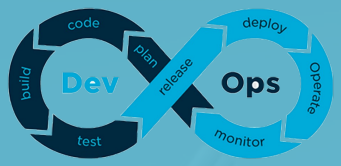
© DolfinED All rights reserved

Hands-on Labs (HoLs)

Docker Volumes – Volume Data & Default Container Behavior



© DolfinED All rights reserved





DockerHub Image Digest

Image Tag

A tag is a mutable label that can point to different images over time.

- For the same tag, the actual contents of that image might change in the future if the image is updated or rebuilt by the maintainers.
- Example: python:3.11-slim today might have different layers or security patches than python:3.11-slim 3 months from now — even though the tag is the same.

© DolfinED All rights reserved

Docker Image Digest

- A digest is a unique, immutable hash that represents the exact image contents (all layers, metadata, etc.)
- Once created, the image cannot change without changing the digest
- This guarantees that the image you build with today is bit-for-bit identical to what you'll get tomorrow — crucial for production consistency, audits, and security

```
docker pull python:3.11-slim
```

```
docker image inspect python:3.11-slim --format='{{index .RepoDigests 0}}'
```

```
python@sha256:4f6c8650a845d64a3e81e3602d0e5f5c31e232ff42d95a5b7fd3605bda1a9185
```

© DolfinED All rights reserved

Tag is a pointer. Digest is a fingerprint.

✅ **Tags can break your app**

- A tag might point to a new image with changes, causing unexpected behavior or bugs.

🔐 **Digests ensure security & reproducibility**

- Lock your base image to a known-good, tested version.

© DolfinED All rights reserved

Docker Image Digest - Benefits

- Guarantees that the **base image will not change** even if the :tag gets updated
- Helps in **security audits** and **build reproducibility**
- Prevents accidental breaking changes from upstream images

© DolfinED All rights reserved

Docker Image Digest in Dockefiles

Dockerfile

```
FROM nginx@sha256:c15da6c91de8d2f436196f3a768483ad32c258ed4e1beb3d367a27ed67253e66
COPY ./index.html  /usr/share/nginx/html/
EXPOSE 80
```

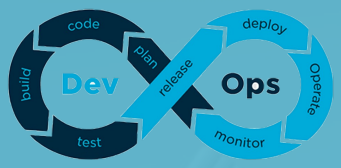
© DolfinED All rights reserved

Hands-on Labs (HoLs)

Docker Image Digest



© DolfinED All rights reserved





docker diff Command

docker diff Command

The docker diff command shows the **changes** made to a container's filesystem compared to its original image

Syntax : docker diff <container_name or container_id>

The output will list the changed files and directories wih the following prefixes:

A Added file or directory

C Changed file or metadata

D Deleted file or directory

© DolfinED All rights reserved

docker diff Command – Use Cases

- Debugging what files were added/modified in a running container
- Auditing changes before committing a new image
- Verifying filesystem side effects from container activity

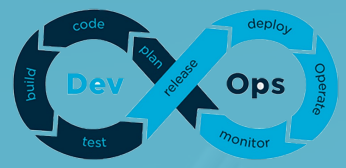
© DolfinED All rights reserved

Hands-on Labs (HoLs)

docker diff Command



© DolfinED All rights reserved





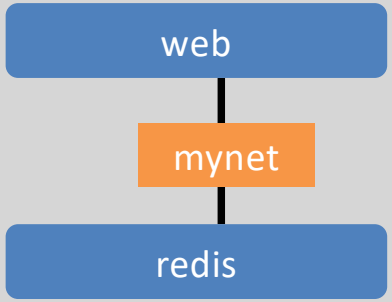
Name/DNS Resolution in Docker

How Do Containers Communicate?

When containers are on the **same user-defined bridge network**, they can **communicate by name**

- Docker **automatically sets up an internal DNS server** for each network
  - ➢ Docker's embedded DNS responds to lookups
- This is true for the default network created by Docker Compose
- This is not applicable for the default bridge (br0) in Docker

curl http://redis



Docker-compose.yml

```
services:
  web:
    ……
    networks:
      - mynet
  redis:
    ……
    networks:
      - mynet
networks:
  mynet:
```

© DolfinED All rights reserved

Internal DNS Server per (user-defined) Network

When you create a **user-defined bridge (or overlay)** network, Docker spawns an **internal DNS server** at 127.0.0.11 inside each container

- This DNS server:
  - Registers each container/service by its name and IP address
  - Resolves **container/service names** to IPs
  - Updates dynamically when services are restarted or replaced
- Each network has its **own DNS namespace**, ensuring **isolation** and no name collisions
  - Services are only discoverable by name **within the same network**

cat /etc/resolv.conf → nameserver 127.0.0.11

© DolfinED All rights reserved

🧠 Docker DNS Resolution Summary

Scenario	What Happens
`127.0.0.11` inside container	Queries Docker DNS embedded in container runtime
Resolving other containers	Docker DNS returns the IP of container (if on same user-defined network)
Resolving external domains	Docker forwards query to host's DNS server
Using external Docker network	Works if services share the same user-defined network
Using default bridge	No service discovery — must use IP manually

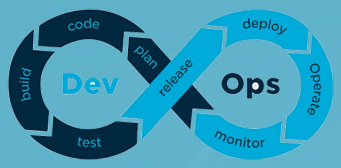
© DolfinED All rights reserved

Hands-on Labs (HoLs)

Name Resolution in Docker



© DolfinED All rights reserved





Dockerfile – Multi-Stage Builds

Dockerfile – Multi-Stage Builds

A **multi-stage build** uses **multiple FROM statements** in a single Dockerfile.

Benefits:

- Use one stage to **build or compile** the app
- Use a second (smaller) stage to **run** the final application

✅ The result:
A **lean, secure, and production-ready image** without build-time tools or source code.

All-in-One

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "app.py"]
```

Multi-Stage

```
# ===== Stage 1: Build =====
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ===== Stage 2: Final image =====
FROM python:3.11-slim
WORKDIR /app
# Copy only installed packages and source code
COPY --from=builder /install /usr/local
COPY . .
CMD ["python", "app.py"]
```

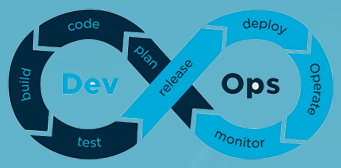
© DolfinED All rights reserved

Hands-on Labs (HoLs)

Docker Multi-Stage Builds



© DolfinED All rights reserved





.dockerignore
Optimize Docker Builds

What is .dockerignore?

A file used to **exclude files and directories** from being copied into the Docker build context

- Works **just like .gitignore**, but specifically for Docker
- Placed in the **root of your build context** (typically next to your Dockerfile)

.dockerignore

```
# Ignore Python cache and virtual envs
__pycache__/
*.pyc
*.pyo
venv/
.env
# Ignore Git files
.git
.gitignore
# Ignore local development tools
*.log
*.swp
.DS_Store
node_modules/
```

© DolfinED All rights reserved

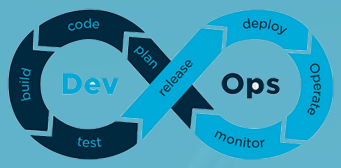
Why use .dockerignore?

🚀 **Speeds up builds** by reducing the build context size

🔒 **Avoids leaking secrets**, credentials, or local config files

📦 **Keeps images smaller** by skipping unnecessary files

© DolfinED All rights reserved





Docker Best Practices

Docker Best Practices

✅ Use Small Base Images

- When available use slim packages such as alpine, python:3.11-slim, or Debian:bullseye-slim

✅ Avoid installing unnecessary packages

- e.g., avoid curl, vim and other dev tools unless required for debugging

✅ Use .dockerignore

- Like .gitignore, this prevents copying unnecessary files into the image (e.g., .git, __pucache__, .env…etc.)

✅ Use WORKDIR instead of RUN cd

© DolfinED All rights reserved

Docker Best Practices (cont.)

✅ Leverage Layer Caching

- Order the Docker instructions from the least to most likely to change
- e.g., install dependencies before copying app code

```
FROM python:3.11-slim
# Set working directory
WORKDIR /app
# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# Copy application files
COPY . .
ENV FLASK_RUN_HOST=0.0.0.0
EXPOSE 5000
CMD ["flask", "run"]
```

© DolfinED All rights reserved

Docker Compose Best Practices

✅ Use .env for Configuration

- Do not hardcode values in the Compose YAML file, use .env instead

✅ Use Named Volumes for Persistence

✅ Use depends_on and Health Checks

- To control startup order and ensure service readiness

© DolfinED All rights reserved

Image Creation and Management Best Practices

✅ Tag Images Properly

- Use meaningful tags

✅ Clean Up Unused Resources

✅ Use image version as opposed to latest to ensure consistent operation

© DolfinED All rights reserved

Docker Security Best Practices

✅ Use Non-Root Users

- For security, avoid running as root. Create and switch to a non-root user

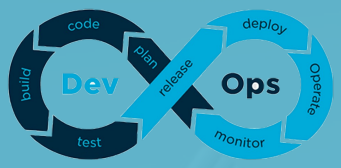
✅ Never Store Secrets in Images

- Use Environment Variables or Secret Managers (not .env in production)

✅ Use Multi-Stage Builds

- Especially for compiled languages (e.g., C, C++, Java, Go, Rust)— compile in one stage, run in another
- Reduces image size and attack surface

© DolfinED All rights reserved



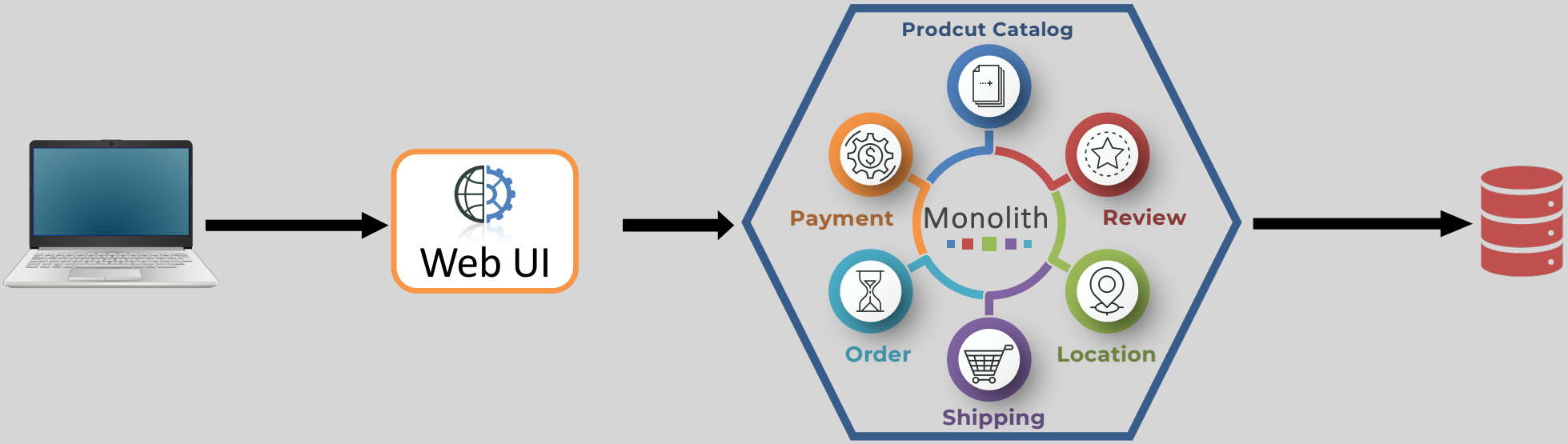


Monolith vs Microservices Applications

Monolith Applications

The entire application is built as a single code project.

- Can be harder to scale and troubleshoot.



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Monolith vs. Microservices



Monolith



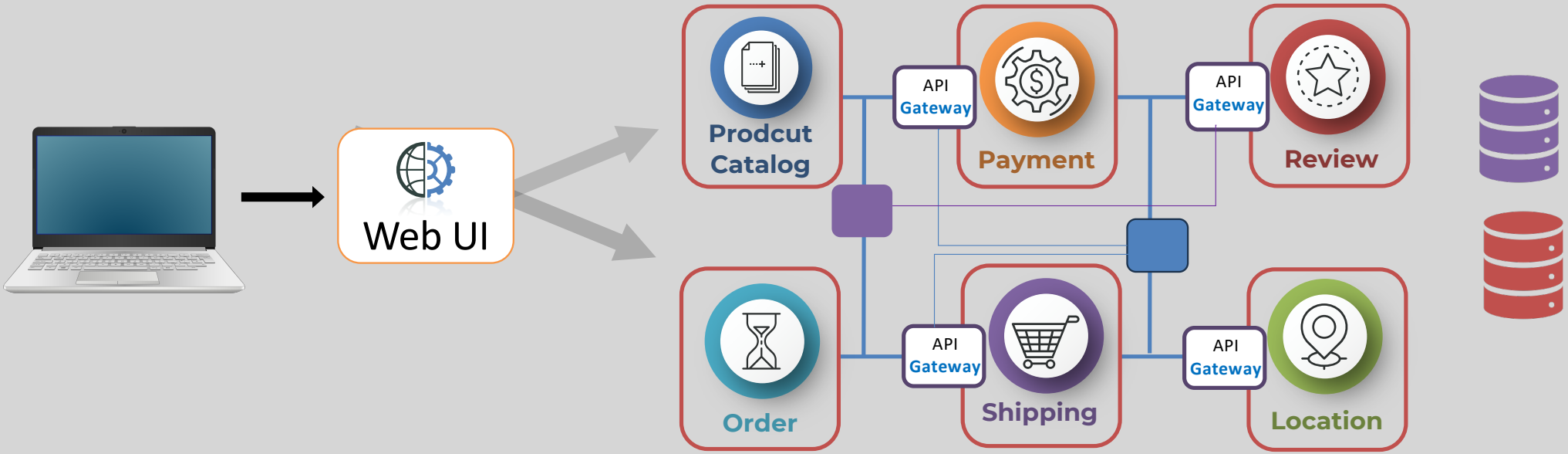
Microservices

© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Building Microservices Applications

- Application code processes or microservices are built as separate code projects
- Easier to scale and troubleshoot
- Each backend service exposes a set of APIs through an API Gateway



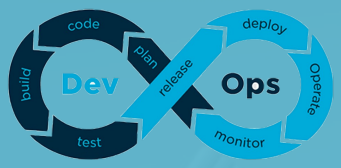
© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Microservices VS. Monolith (cont.)

	Microservices	Monolith
Architecture	Collection of small services	Single build of code
Scalability	Easy scaling and better usage of resources	Hard to Scale
Time to Market	Easy to build and deploy	Complex and difficult to deploy
Reliability	Very reliable, if a service fails, the whole application will not go down	If a service fails, the entire application goes down

© DolfinED All rights reserved

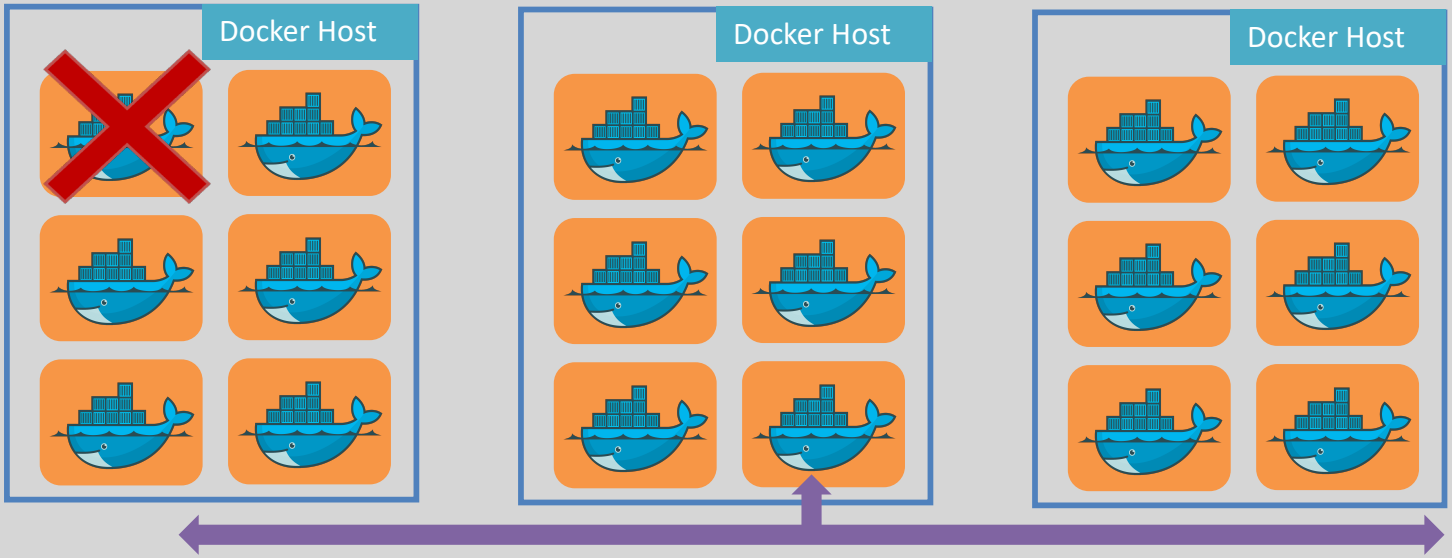




Docker SWARM Overview

Microservices Environments – Deployment, Management and Scaling

- How do we automate container lifecycle?
- How can we scale microservices on-demand?
- How can we recreate containers when they fail?
- How can we update services without downtime?
- How can we allow and track communications among containers?
- How can we control where containers are deployed?



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Docker Swarm – Docker Orchestration Tool

What is it?

- Swarm mode is an advanced Docker Engine feature for managing a cluster of Docker daemons
- It is a native clustering and orchestration tool built into Docker
- It turns multiple Docker hosts into a single virtual Docker engine called a Swarm
- It uses the **docker swarm** and **docker service** commands

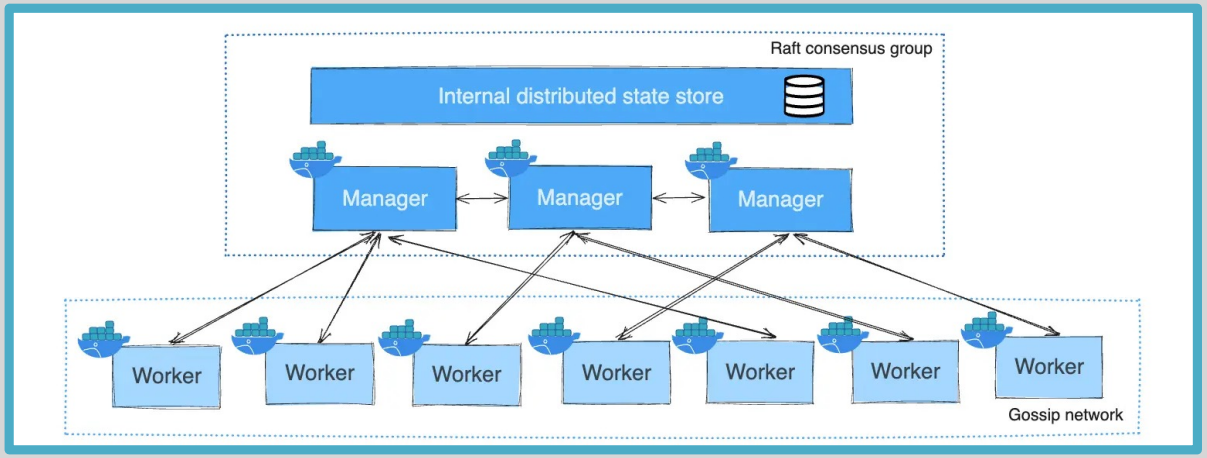


Swarm of Bees

© DolfinED All rights reserved

Docker Swarm – Architecture

- Manager nodes handle cluster management including cluster state maintenance, scheduling tasks
- You can have one or more managers (odd number is recommended)
- Worker nodes are also instances of Docker Engine whose sole purpose is to execute containers

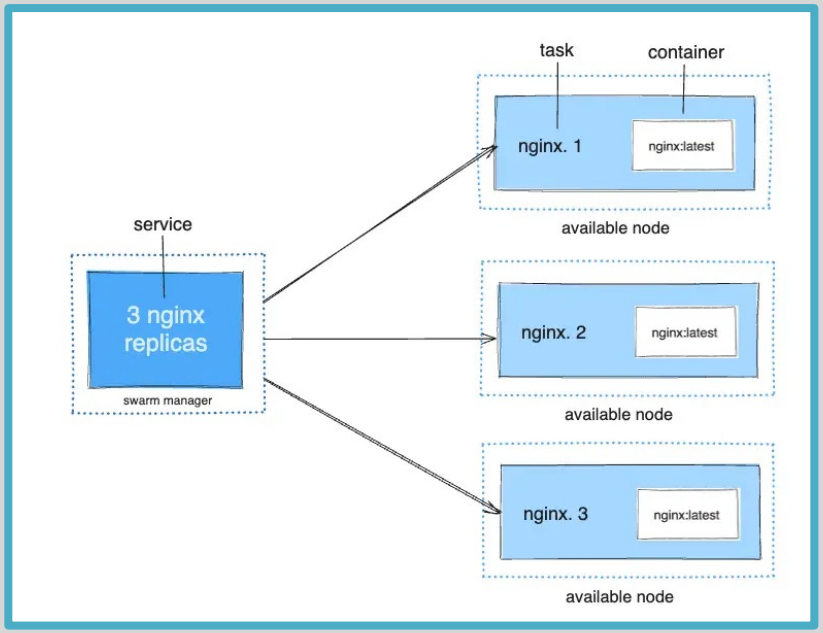


Source: docs.docker.com

© DolfinED All rights reserved

Docker Swarm – Services, Tasks, and Containers

- Services are used to deploy an application image when Docker Engine is in Swarm mode
- The service defines the image, commands to execute in the running containers, ports to expose, networking details, CPU and memory limits, update policy, and the number of replicas to create
- The replicas are deployed as tasks on the worker nodes
- Tasks run independently of each other
- Each task includes one container



Source: docs.docker.com

© DolfinED All rights reserved

Docker Swarm – Use Cases

- Small to medium **teams or projects**
- When **simplicity & fast setup** are important
- Ideal for **development, PoCs**, and lightweight production

Use Swarm mode if you intend to use Swarm as a production runtime environment

© DolfinED All rights reserved

Why Use Swarm?

🧠 Easy to Learn : Uses same Docker CLI & Compose Native

🔁 Rolling Updates : Built-in service update strategy

🔒 Secure by Default : TLS Encryption between nodes

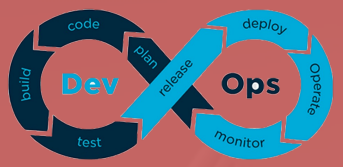
🚀 Lightweight : Less resource-intensive than Kubernetes

© DolfinED All rights reserved

Swarm vs. Kubernetes vs. OpenShift

Feature	Docker Swarm	Kubernetes	OpenShift
🧠 Learning Curve	Easiest	Moderate to steep	Steep (based on Kubernetes)
📦 Setup	Simple CLI	Complex but flexible	Very complex (enterprise-focused)
🔄 Rolling Updates	Supported	Advanced strategies	Advanced + Pipelines
🔒 Security	TLS by default	Needs setup	Strong default policies (RBAC, SCC)
🌐 Networking	Simple overlay	Advanced, pluggable	Kubernetes-based
📂 Storage Integration	Basic	Pluggable (CSI)	Strong support
🧩 Ecosystem	Minimal	Huge ecosystem	Enterprise tools, CI/CD
📊 Ideal For	Dev/Test, simple prod	Complex apps, scalability	Regulated industries, enterprises

© DolfinED All rights reserved





Capstone Projects

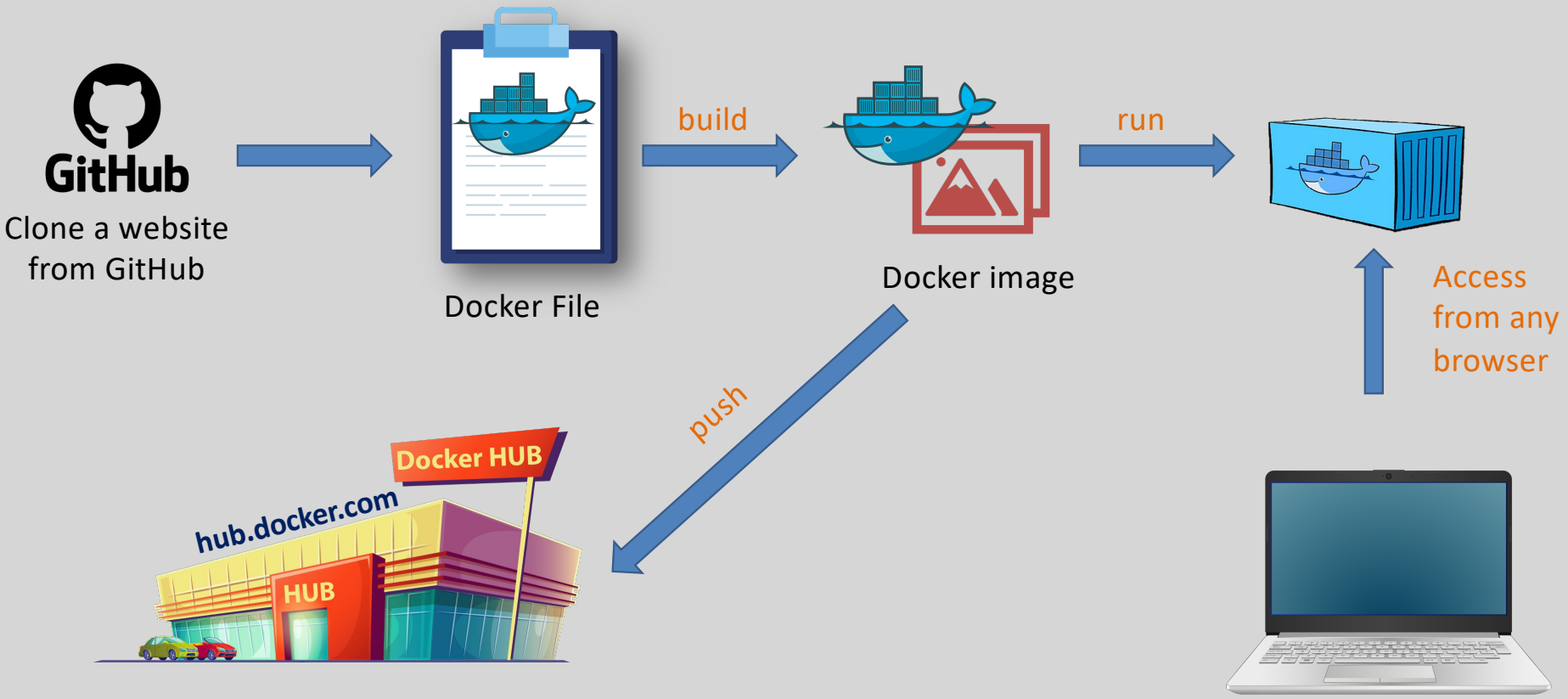
Capstone Project I

Deploying A Website on Nginx Server using Docker



© DolfinED All rights reserved

Deploy a Website on Nginx using a Dockerfile



© DolfinED All rights reserved

DolfinED
Transforming Lives Through Learning

Deploy a Website on Nginx using a Dockerfile

Docker Host

```
Project Code Structure (Simplified)
/home/ubuntu/
 ├── Dockerfile
 ├── docker-compose.yml (optional)
 ├── Docker-Capstone-Project1/
    ├── school-website/
       └── index.html
           error.html
           about.html
          ....
       ├── assets/
       ├── Documentaion/
       ├── src/
```

Container

```
Container Image Structure(Simplified)
usr/share/nginx/html/
                  └── index.html
                      error.html
                      about.html
                     ....
                  ├── assets/
                  ├── Documentaion/
                  ├── src/
```

© DolfinED All rights reserved

Key Docker Skills Demonstrated

- Writing and building a Dockerfile for a static web server
- Understanding container lifecycle: build → run → stop → remove
- Managing ports, volumes, and static content in containers
- Basic use of Docker Compose for service orchestration
- Containerizing front-end workflows for scalable deployment

© DolfinED All rights reserved

What You Will Practice

✅ Docker Fundamentals

- Writing a Dockerfile to containerize a static web app using a lightweight web server (e.g., Nginx or httpd).

✅ Docker Image Creation

- Building custom images for front-end deployment.

✅ Serving Static Content with Nginx

- Configuring Nginx in a container to serve HTML, CSS, and JS assets.

✅ Deployment Readiness

- Testing and running the container locally (docker run).
- Tagging and pushing to Docker Hub.

© DolfinED All rights reserved

What You Will Practice - Optional

✅ Docker Volumes (Optional Enhancement)

- Using bind mounts or named volumes to live-reload content during development.

✅ Docker Compose (Optional for Extension)

- Adding Docker Compose to manage Nginx + potential extensions (e.g., analytics or form backend stubs) with one command.

© DolfinED All rights reserved

Capstone Project II

Website Visitor Counter Application –

- Flask + Redis DB/Cache

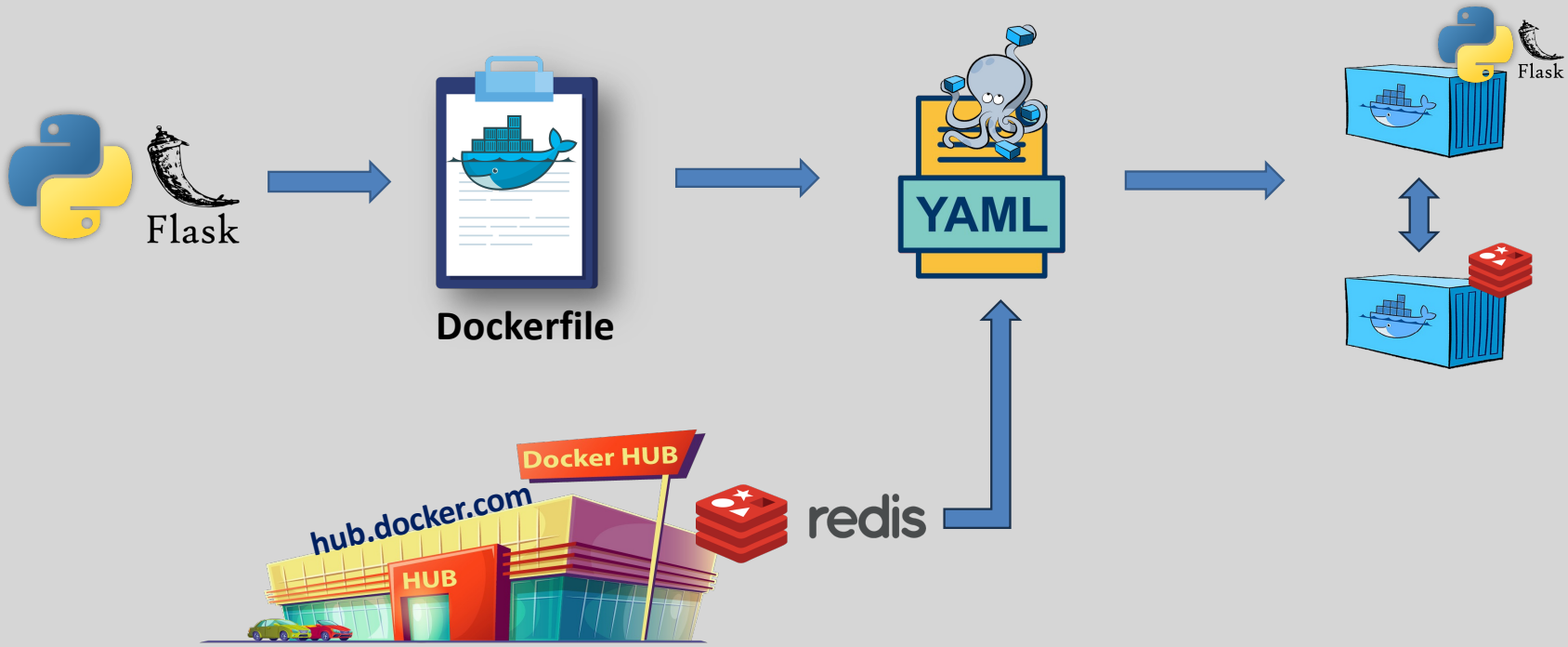
Deployed using Docker Compose



© DolfinED All rights reserved

Creating A Simple Web Application Using Docker Compose

- The application utilizes the Flask framework to build a multi-container web application that counts the number of visitors to a website
- Redis database is used to keep track of the visit count



© DolfinED All rights reserved

The GitHub Repo Project Structure

```
visitor-counter/
│
├── app/                          # Application source code
│   ├── app.py                    # Flask application logic
│   ├── templates/
│   │   └── index.html            # HTML page with embedded counter and visitor table
│   └── static/                   # (Optional) Static assets like CSS/JS
│
├── .env.example                  # Sample environment variables (do not commit .env)
├── .gitignore                    # Ignore Python cache, .env, etc.
├── Dockerfile                    # Docker image setup for Flask app
├── docker-compose.yml            # Docker Compose for Flask + Redis
├── requirements.txt              # Python dependencies
├── README.md                     # Project overview and instructions
└── Project-Requirements.md       # Assignment-specific requirements document
```

Project Link on GitHub

© DolfinED All rights reserved

Key Docker Skills Demonstrated

- Writing and building a Dockerfile to build/package the visitor counter application
- Using environment variables
- Managing ports, volumes, and content in containers
- Using Docker Compose for multi-container application orchestration
- Containerizing and integrating the application with the redis database

© DolfinED All rights reserved

What You Will Practice

✅ Docker Fundamentals
- Writing a Dockerfile to containerize the Flask application

✅ Docker Image Creation
- Using Docker Compose to build the Flask application docker image

✅ Automating Build and Debugging using –build, -d, -p and –no-cache options

✅ Docker Volumes and Bind Mounts
- Using bind mounts to live-reload content during development
- Using named Volumes to persist Redis data

© DolfinED All rights reserved

What You Will Practice (cont.)

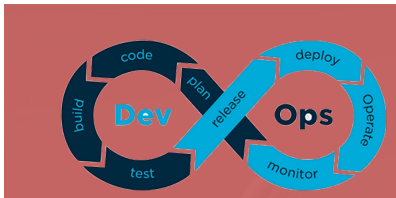
✅ Docker Compose

- Adding Docker Compose to manage and orchestrate the build and deployment lifecycle

✅ Deployment Readiness

- Testing and running the application using Docker Compose

© DolfinED All rights reserved





Congratulations!
What is Next?

Next Learning Step:



kubernetes

© DolfinED All rights reserved